

The Residue of Proof

Sufficiency, History, and Repair in Automated Reasoning

Flyxion

Independent Researcher

October 2026

Preface

Automated reasoning systems have become dependable producers of verified statements: a satisfiability solver emits a refutation that a small checker accepts, a proof assistant elaborates a script into a term that a kernel typechecks, a rewrite engine normalizes an expression and a certificate says the normalization was sound. In each case the output of the process is a verdict together with evidence that a trusted component can examine. This book is about what such an output is *not*, and about what it would have to retain in order to remain useful to a scientist who relies on it.

A verified result can be correct and still fail its user in at least three ways. It may be correct about a formal object that is not the object of interest, so that the bridge from the formal statement to the observation carries obligations no kernel can discharge. It may be correct and fragile, valid for the constants it was checked against and silent about the neighbouring ones. And it may be correct and opaque: the history of the search that found it, the options that were refused, the alternatives that were tried, may be the only thing that would let a later reader repair it when an assumption changes, and that history is precisely what compression throws away. The central claim is that these three failures have a common mathematical form. In each, a function that was needed was not constant on the fibres of the representation that was kept.

Structure. Part I sets out proof systems and search as transition systems and develops the classical machinery used throughout: inference systems and fairness, resolution and unification, rewriting and normal forms. Part II treats the decision procedures and kernels that supply certificates in practice: conflict-driven clause learning and its proof formats, theory combination, the kernel architecture of proof assistants, and the trusted base. Part III is the core. It defines verification obligations, sufficiency of a representation relative to a declared task family, stability regions and repair, and inheritance along transformations. Part IV applies the same structure to the steps of a search, asking when a search may discard possibilities while keeping the guarantees that justify it. Part V turns to scientific use: correspondence with observations, mutation of verified artifacts as an experiment on the claims of Part III, and a negative result treated as a method. Part VI collects the three principles and an honest account of what has been proved.

Standards of proof. Results internal to the argument are proved. Classical external theorems are stated precisely and cited; they are listed in Appendix A. Several statements were also checked by computation, as recorded in Chapter 20, and these checks are marked as checks and not as proofs. No theorem of the book has been mechanized in a proof assistant. Where the book describes experimental work reported elsewhere, the chapters prove what

such reports can and cannot support and do not reproduce the data.

A note on vocabulary. The verbs *pop*, *refuse*, *bind* and *collapse* are used in Part IV for the operations on a search world, following earlier manuscripts of the author on binding and on disposition calculi [35, 36]. Every definition needed is restated, and nothing from those manuscripts is assumed.

Reading. A reader interested only in the classical material can read Parts I and II alone; they are independent of the rest. A reader interested in the argument can begin with Chapter 1, Chapter 10, and Chapter 20. Each chapter closes with a boxed statement of what it establishes and with exercises, some of which are open-ended extensions flagged as such.

Flyxion
Independent Researcher
October 2026

Contents

Preface	ii
I Proof Systems and Search	1
1 Proof Systems as Transition Systems	2
1.1 Proof systems and checkers	2
1.2 Producers and the independence of soundness	3
1.3 Statements as translations	4
1.4 Search as a transition system	4
1.5 The residue of a run	5
1.6 What this chapter fixes	5
2 Search Spaces and Strategies	7
2.1 Inference systems and saturation	7
2.2 Implementing fairness: the given-clause loop	8
2.3 Existence versus discovery	9
2.4 Three limits	9
2.5 Outcomes of a bounded search	10
2.6 Summary	11
3 Resolution, Unification, and Saturation	12
3.1 Propositional resolution	12
3.2 Completeness by elimination	13
3.3 Unification	14
3.4 First-order resolution	15
3.5 Resolution as a proof system, and what extraction discards	16
4 Rewriting and Normalization	18
4.1 Abstract reduction systems	18
4.2 Term rewriting and termination by interpretation	20
4.3 Critical pairs and local confluence	20
4.4 Equational theories and completion	21

4.5	Normal forms as a canonical residue	22
II	Decision Procedures and Kernels	24
5	SAT and Clause Learning as Resolution with Memory	25
5.1	Clauses, trails, and unit propagation	25
5.2	Conflict-driven clause learning as a transition system	25
5.3	Conflict analysis is resolution	27
5.4	Certificates: RUP, RAT, and DRAT	28
5.5	Elimination, and what a reduced formula forgets	30
5.6	The residue of a solver run	30
6	Theories, Congruence, and Combination	32
6.1	Theories and the lazy architecture	32
6.2	Equality: congruence closure	33
6.3	Linear arithmetic: Farkas certificates	34
6.4	Combining theories	35
6.5	Certificates for theory results and the growth of the trusted base	36
6.6	What an SMT result discards	36
7	Kernels, Tactics, and the Trusted Base	38
7.1	The LCF discipline	38
7.2	Dependent type theory as kernel	39
7.3	Proof irrelevance as a collapse	40
7.4	Axiom footprints	41
7.5	The trusted base of a verified statement	42
7.6	Scripts and terms	43
8	Certificates and Checkers	44
8.1	Certifying computation	44
8.2	Pipelines and the direction of preservation	45
8.3	Proof by reflection	46
8.4	How large must certificates be?	47
8.5	Certificates are bound to inputs	47
8.6	Certificates are not explanations	48
III	Sufficiency of Verified Results	49
9	Verification Obligations	50
9.1	A verified result is a tuple	50
9.2	Dimensional homogeneity as a theorem	51

9.3	Correspondence and scope: the free-fall example	52
9.4	The ledger	53
9.5	What discharges what	54
10	Sufficiency of Certificates	56
10.1	Histories, representations and task families	56
10.2	The Galois connection and the scope of a representation	57
10.3	Lower bounds	58
10.4	No free sufficiency	59
10.5	Proof irrelevance as a sufficiency statement	60
10.6	A geometric instance: exterior data and hidden material	60
10.7	Declaring families in practice	63
10.8	What this chapter fixes	63
11	History, Stability and Repair	65
11.1	Stability regions	65
11.2	Linear arithmetic: certificates are half-spaces	65
11.3	Clause sets: regions are up-sets	68
11.4	Repair as a relation	69
11.5	Scripts are functions, terms are points	70
11.6	What this chapter fixes	71
12	Inheritance along Transformations	72
12.1	Correspondence under rewriting is undecidable	72
12.2	Certified fragments and a three-valued verdict	73
12.3	Proof terms inherit along definitional equality	74
12.4	Typed dependency edges and the interface lemma	75
12.5	What this chapter fixes	77
IV	Admissibility of Search	79
13	Search States as Worlds	80
13.1	Closure systems and option spaces	80
13.2	Worlds	81
13.3	The option space moves only under refusal	81
13.4	The extensional state is the minimal state for all futures	82
13.5	Provenance is not in the state	83
13.6	Outcomes, and two guarantees that must not be conflated	84
13.7	What this chapter fixes	85
14	Redundancy as Admissible Collapse	86
14.1	A closure adapted to redundancy	86

14.2 Tautology and subsumption refusal	87
14.3 Completeness with deletion	88
14.4 Admissibility is task-relative	89
14.5 What this chapter fixes	90
15 Guidance and the Verifier Gate	91
15.1 Ordering does not change the option space	91
15.2 The verifier gate	92
15.3 Ungated proposals	93
15.4 Self-contained gates	94
15.5 What this chapter fixes	95
16 Lifecycle and Authority	96
16.1 The machine	96
16.2 Collapse safety	97
16.3 Authority is not verification	97
16.4 Why a failed verification must stay live	98
16.5 Reachability and history	98
16.6 Proposed extension: typed memory consolidation	99
16.7 What this chapter fixes	101
V From Verified Theorem to Scientific Claim	102
17 Correspondence	103
17.1 Correspondence claims and failure conditions	103
17.2 Identifiability is sufficiency of the observation	104
17.3 Local versus global injectivity	104
17.4 A complete example: the Burgers equation	105
17.5 What this chapter fixes	106
18 Mutation of Verified Artifacts	108
18.1 Specimens, operators, outcomes	108
18.2 Preregistration and retention	109
18.3 A flat dependency graph is insufficient	109
18.4 Footprints and the placeholder audit	110
18.5 A model-level example: associative normalization	111
18.6 What a matrix of this size can support	111
18.7 What this chapter fixes	112
19 A Negative Result as Method	113
19.1 Two questions that must not be merged	113
19.2 Incremental predictive value	114

19.3 Underpowered interventions	115
19.4 The ledger of a negative result	116
19.5 What this chapter fixes	116
 VI Synthesis	 118
 20 What a Verified Result Owes	 119
20.1 Three principles	119
20.2 The result card	120
20.3 What is proved, cited, computed and proposed	121
20.4 Open directions	121
20.5 Conclusion	122
 A External Theorems Used	 123
 Bibliography	 126

Part I

Proof Systems and Search

Chapter 1

Proof Systems as Transition Systems

Automated reasoning is usually described by what it produces: a verdict that a statement holds, together with some evidence for the verdict. This chapter fixes the minimal vocabulary needed to say precisely what the evidence is, what checks it, and what the process that found it leaves behind. Everything later in the book is a refinement of three objects introduced here: a *checker*, a *producer*, and the *residue* that remains when the producer's work is compressed into a checked derivation.

1.1 Proof systems and checkers

Definition 1.1 (Proof system). A *proof system* is a quadruple $S = (\Phi, D, \text{Chk}, \text{Val})$ where

- (i) Φ is a set of *statements*, each a finite string over a fixed finite alphabet;
- (ii) D is a set of *derivations*, each a finite string over a fixed finite alphabet;
- (iii) $\text{Chk} \subseteq D \times \Phi$ is a *decidable* relation, the *checking relation*; and
- (iv) $\text{Val} \subseteq \Phi$ is a set of *valid* statements, defined semantically and not required to be decidable or even recursively enumerable.

The system is *sound* if $\text{Chk}(d, \varphi)$ implies $\varphi \in \text{Val}$, and *complete* if $\varphi \in \text{Val}$ implies $\text{Chk}(d, \varphi)$ for some $d \in D$. The set of *theorems* is $\text{Th}(S) = \{\varphi \in \Phi : \exists d \in D. \text{Chk}(d, \varphi)\}$.

Decidability of Chk is the single structural requirement, and it is the reason a proof system is useful at all: whoever holds a candidate derivation can settle its status by a terminating computation, regardless of how the candidate was obtained. Validity, by contrast, is a semantic notion. The central role of the gap between $\text{Th}(S)$ and Val is already visible: soundness says the former is inside the latter, completeness says they coincide.

Proposition 1.2 (Theorems are enumerable). *If D is effectively enumerable then $\text{Th}(S)$ is recursively enumerable. If in addition S is sound and complete, Val is recursively enumerable.*

Proof. Enumerate the pairs $(d, \varphi) \in D \times \Phi$ in a fixed dovetailed order, run the decision procedure for Chk on each pair, and output φ whenever the check succeeds. This lists exactly $\text{Th}(S)$. If S is sound and complete then $\text{Th}(S) = \text{Val}$. ■

Proposition 1.2 is the formal reason that every complete proof system for first-order validity yields a semi-decision procedure and no more; Chapter 2 returns to this with Church's theorem.

Example 1.3 (An implicational Hilbert system). Let Φ be the set of formulas built from propositional variables with the connective $\rightarrow$. Let a derivation be a finite sequence of formulas, each of which is an instance of one of the schemes

$$K : A \rightarrow (B \rightarrow A), \quad S : (A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C)),$$

or follows from two earlier lines A and $A \rightarrow B$ by modus ponens. Define $\text{Chk}(d, \varphi)$ to hold when d is such a sequence ending in φ . The relation is plainly decidable. With Val the set of formulas true under every Boolean valuation, soundness holds: instances of K and S are tautologies, and modus ponens preserves truth under a fixed valuation, so an induction on the length of d shows that every line is a tautology.

A derivation of $p \rightarrow p$:

- | | |
|--|---------|
| 1. $(p \rightarrow ((p \rightarrow p) \rightarrow p)) \rightarrow ((p \rightarrow (p \rightarrow p)) \rightarrow (p \rightarrow p))$ | S |
| 2. $p \rightarrow ((p \rightarrow p) \rightarrow p)$ | K |
| 3. $(p \rightarrow (p \rightarrow p)) \rightarrow (p \rightarrow p)$ | MP 2, 1 |
| 4. $p \rightarrow (p \rightarrow p)$ | K |
| 5. $p \rightarrow p$ | MP 4, 3 |

The five-line certificate is longer than the truth table for $p \rightarrow p$, which has two rows. This is the first instance of a pattern that recurs throughout the book: a derivation is not a compact summary of why a statement holds, but a record in a fixed idiom of one way to establish it.

1.2 Producers and the independence of soundness

Definition 1.4 (Producer). A *producer* for S is any partial function $P : \Phi \rightarrow D$. No assumption of computability, correctness, or termination is made. The set of statements *accepted* from P is

$$\text{Acc}(P) = \{\varphi \in \Phi : P(\varphi) \text{ is defined and } \text{Chk}(P(\varphi), \varphi)\}.$$

Theorem 1.5 (Independence of soundness). *Let S be sound. For every producer P , $\text{Acc}(P) \subseteq \text{Val}$. Conversely, if S is complete then there is a producer P^* with $\text{Acc}(P^*) = \text{Val}$.*

Proof. If $\varphi \in \text{Acc}(P)$ then $\text{Chk}(P(\varphi), \varphi)$, hence $\varphi \in \text{Val}$ by soundness. For the converse, by completeness choose for each $\varphi \in \text{Val}$ a derivation $P^*(\varphi)$ with $\text{Chk}(P^*(\varphi), \varphi)$, and leave P^* undefined elsewhere. ■

The theorem is elementary, but its consequences are the organizing principle of the whole subject. The *assurance* attached to an accepted statement depends only on the checker; the producer may be heuristic, learned, probabilistic, buggy, or adversarial without affecting it. The cost is paid in a different currency. A producer's *failure* to deliver a derivation carries no information about validity.

Discipline of non-certification. Let $\varphi \notin \text{Acc}(P)$. Nothing follows about $\varphi \in \text{Val}$ unless a separate argument shows that P is complete on φ . Failure of certification is not certification of failure.

1.3 Statements as translations

The statement $\varphi \in \Phi$ that a checker reads is rarely the claim a person intends. A physicist or mathematician states a claim ψ in a vocabulary of their own, and a translation τ renders it into the checker's language.

Definition 1.6 (Adequate translation). Let (Ψ, True) be a set of intended claims with a truth predicate $\text{True} \subseteq \Psi$. A map $\tau : \Psi \rightarrow \Phi$ is *adequate* for S if $\tau(\psi) \in \text{Val}$ implies $\psi \in \text{True}$.

Proposition 1.7 (Composition of assurance). *If S is sound and τ is adequate, then $\text{Chk}(d, \tau(\psi))$ implies $\psi \in \text{True}$.*

Proof. Soundness gives $\tau(\psi) \in \text{Val}$, adequacy gives $\psi \in \text{True}$. ■

The proposition has a precise and uncomfortable reading. The checker, however small and trusted, certifies only $\tau(\psi) \in \text{Val}$. Whether this carries over to ψ is a property of τ and of the notion True , neither of which the checker examines. In the vocabulary of later chapters, τ is a *bridge hypothesis*, and a verified result is the pair of a checked derivation and an unchecked bridge. Chapter 9 organizes these obligations; for the moment the point is that adequacy of τ is not a theorem of S , and in general cannot be, since True may refer to a world outside the system.

1.4 Search as a transition system

Producers in practice are searches. A producer begins from a goal, repeatedly applies rules that extend or restrict a partial structure, and stops when a derivation can be read off.

Definition 1.8 (Search procedure). A *search procedure* for S is a tuple $R = (\Sigma, \rightarrow, \iota, F, \text{ext})$ where Σ is a set of *states*, $\rightarrow \subseteq \Sigma \times \Sigma$ a transition relation, $\iota : \Phi \rightarrow \Sigma$ an initialization map, $F \subseteq \Sigma$ the *final* states, and $\text{ext} : F \rightarrow D$ an extraction map. A *run* on φ is a finite path $\iota(\varphi) = s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n$ with $s_n \in F$; its *trace* is the sequence $(s_0, \dots, s_n)$ and its *output* is $\text{ext}(s_n)$.

Define the producer P_R by choosing, for each φ that admits a run, the output of some run (the choice is immaterial for what follows). Then P_R falls under Theorem 1.5. We separate two properties of a procedure.

Definition 1.9 (Internal soundness, trace-faithfulness). R is *internally sound* if for every run on φ ending in s_n we have $\text{Chk}(\text{ext}(s_n), \varphi)$. It is *trace-faithful* if the trace can be recovered from the output together with φ , that is, if the map $(\text{run}) \mapsto (\varphi, \text{ext}(s_n))$ is injective on runs.

Internal soundness is a property of the procedure that makes the checker redundant on its outputs; it is what the LCF architecture of Chapter 7 achieves by construction. Trace-faithfulness is the opposite kind of property, and realistic procedures fail it.

Example 1.10 (Independent subgoals). Let states be pairs (G, T) of a list G of open goals and a partial derivation tree T . Let the goal be $p \wedge q$ from assumptions p, q . The rule $\wedge I$

replaces the goal by the two subgoals p and q , and a closing rule discharges any open goal that is an assumption. After $\wedge I$, two runs are possible: close p first, then q , or close q first, then p . Both end in the same final tree $\wedge I(\text{hyp}_p, \text{hyp}_q)$. The procedure has two distinct traces with equal output, so it is not trace-faithful.

1.5 The residue of a run

Example 1.10 is almost too small to notice. At scale the phenomenon is pervasive: lemmas tried and abandoned, orderings explored, heuristics consulted, and dead branches pruned all appear in the trace and none appears in the output.

Definition 1.11 (Residue). Let $\text{Run}(\varphi)$ be the set of runs of R on φ and $\varrho : \text{Run}(\varphi) \rightarrow D$ the output map. For $d \in D$ the *residue* of d at φ is the fibre $\varrho^{-1}(d)$. Two runs are *output-equivalent* if they lie in the same fibre.

A downstream task that can be served by the pair (φ, d) alone must be invariant on fibres. A task that must distinguish runs inside a fibre cannot be served by the derivation, however carefully the derivation is checked. This is the earliest and simplest statement of the sufficiency criterion that Chapter 10 generalizes: a representation is sufficient for a task precisely when the task factors through it.

Proposition 1.12 (Factorization criterion). Let $\text{Run}(\varphi) \neq \emptyset$ and let $T : \text{Run}(\varphi) \rightarrow Y$ be a task, understood as a function of the full run (so $Y \neq \emptyset$). There is a function $T' : D \rightarrow Y$ with $T = T' \circ \varrho$ if and only if T is constant on every fibre of ϱ .

Proof. If $T = T' \circ \varrho$ and $\varrho(r) = \varrho(r')$ then $T(r) = T'(\varrho(r)) = T'(\varrho(r')) = T(r')$. Conversely, if T is constant on fibres, define $T'(d)$ as the common value on $\varrho^{-1}(d)$ when this fibre is nonempty and arbitrarily otherwise. Then $T' \circ \varrho = T$ by construction. ■

Proposition 1.12 is a restatement of the fact that a function factors through a quotient if and only if it respects the equivalence relation, and its simplicity is deliberate. The substance lies in applying it to tasks that people actually want served by a proof: checking, teaching the method, adapting it to a neighbouring problem, and diagnosing a failed application. These are not functions of d alone, and Chapter 11 shows this with explicit counterexamples.

1.6 What this chapter fixes

Four points are used throughout.

1. Assurance attaches to the checker and the translation, never to the producer (Theorem 1.5, Proposition 1.7).
2. Absence of a derivation is not evidence of invalidity.
3. A search procedure is a transition system, and its traces carry strictly more information than its outputs in general (Example 1.10).
4. A task is served by an output exactly when it factors through the extraction map (Proposition 1.12).

The next chapter studies which transition systems are complete, and what fairness conditions on the transition relation are needed for a search procedure to find every derivation that exists.

Exercise 1.1. Show that the converse of Proposition 1.7 fails: exhibit a sound system, an adequate τ , and a $\psi \in \text{True}$ with $\text{Chk}(d, \tau(\psi))$ false for all d .

Exercise 1.2. In Example 1.3, show that every derivation of $p \rightarrow p$ has at least five lines, or find a derivation with fewer.

Exercise 1.3. Construct a search procedure that is internally sound but not complete, and one that is complete but not internally sound. Explain why neither is ruled out by Theorem 1.5.

Chapter 2

Search Spaces and Strategies

Chapter 1 treated a producer as a black box. This chapter opens it. A producer that is a search has a space it explores, a strategy that orders the exploration, and a guarantee, when there is one, that every derivation that exists will eventually be reached. The guarantee is called *fairness*, and the main theorem below shows that fairness is the only thing a strategy must add to a refutationally complete calculus to obtain a semi-decision procedure. The second half of the chapter records, with proofs from Church's theorem, three limits that no strategy escapes: there is no computable bound on derivation size, there is no terminating completion of the search, and no combination of derivation-finding and finite-countermodel-finding settles every statement.

2.1 Inference systems and saturation

The calculi of automated reasoning share a skeleton that is independent of the logic. Objects are generated from earlier objects by rules, and one distinguished object is contradictory.

Definition 2.1 (Inference system). An *inference system* is a tuple $\mathsf{I} = (X, \mathcal{M}, \models, \perp, \mathcal{R})$ where X is a set of *objects* (formulas, clauses, sequents), $\mathcal{M}$ is a set of *models*, $\models \subseteq \mathcal{M} \times X$ is a satisfaction relation, $\perp \in X$ is an object satisfied by no model, and $\mathcal{R}$ is a set of *rule instances* (Q, c) with $Q \subseteq X$ finite and $c \in X$. A rule instance is *sound* if every model satisfying all of Q satisfies c ; I is *sound* if all its rule instances are.

For $N \subseteq X$, write $m \models N$ if $m \models x$ for all $x \in N$, and call N *unsatisfiable* if no model satisfies it. N is *saturated* if $(Q, c) \in \mathcal{R}$ and $Q \subseteq N$ imply $c \in N$. I is *refutationally complete* if every saturated unsatisfiable set contains $\perp$.

Definition 2.2 (Derivation, fairness). A *derivation* from $N_0 \subseteq X$ is a sequence $N_0 \subseteq N_1 \subseteq N_2 \subseteq \dots$ in which $N_{i+1} \setminus N_i$ consists of conclusions of rule instances whose premises lie in N_i . Put $N_\infty = \bigcup_i N_i$. The derivation is *fair* if every rule instance (Q, c) with $Q \subseteq N_i$ for some i has $c \in N_j$ for some j .

Because premise sets are finite and the chain is increasing, $Q \subseteq N_\infty$ holds if and only if $Q \subseteq N_i$ for some i . Hence fairness is exactly saturation of the limit.

Lemma 2.3 (Fairness is limit saturation). *A derivation is fair if and only if N_∞ is saturated.*

Proof. Suppose the derivation is fair and $(Q, c) \in \mathcal{R}$ with $Q \subseteq N_\infty$. Since Q is finite and the chain increasing, $Q \subseteq N_i$ for some i , so $c \in N_j \subseteq N_\infty$ for some j . Conversely, if N_∞ is saturated and $Q \subseteq N_i$, then $Q \subseteq N_\infty$ so $c \in N_\infty$, hence $c \in N_j$ for some j . ■

Theorem 2.4 (Fair derivations decide refutability). *Let $\mathcal{I}$ be sound and refutationally complete, and let $N_0, N_1, \dots$ be a fair derivation from N_0 . Then N_0 is unsatisfiable if and only if $\perp \in N_i$ for some i .*

Proof. ($\Leftarrow$) By induction on i , every model of N_0 is a model of N_i : the base case is trivial, and for the step each new element is the conclusion of a sound rule instance with premises in N_i , hence satisfied by every model of N_i . If $\perp \in N_i$ then a model of N_0 would satisfy $\perp$, which no model does. So N_0 is unsatisfiable.

($\Rightarrow$) Suppose N_0 is unsatisfiable. Since $N_0 \subseteq N_\infty$, any model of N_∞ would be a model of N_0 , so N_∞ is unsatisfiable. By Lemma 2.3 it is saturated, so by refutational completeness $\perp \in N_\infty$, hence $\perp \in N_i$ for some i . ■

The ($\Rightarrow$) direction is what makes search *complete*: if a contradiction is derivable, a fair strategy reaches it. The ($\Leftarrow$) direction is soundness. Neither direction says anything about how long the search takes, nor about what happens when N_0 is satisfiable.

Corollary 2.5 (Non-refutation is silent). *If N_0 is satisfiable and the derivation is fair, then $\perp \notin N_i$ for all i . If the derivation is stopped at a finite stage k without $\perp$, then nothing follows about the satisfiability of N_0 unless N_k is saturated.*

Proof. The first claim is the contrapositive of the ($\Leftarrow$) direction of Theorem 2.4. For the second, a finite stage that is not saturated is a prefix of a derivation that may still produce $\perp$, so its silence is uninformative. If instead N_k is saturated and $\perp \notin N_k$, then N_k is satisfiable by the contrapositive of refutational completeness, and so is $N_0 \subseteq N_k$. ■

2.2 Implementing fairness: the given-clause loop

Fairness is a property of an infinite derivation. A strategy must achieve it by a local rule for choosing what to do next. The standard device is a *given-clause loop*, with a processed set P and an unprocessed queue U .

Definition 2.6 (Given-clause strategy). Assume that for every finite $Q \subseteq X$ only finitely many rule instances have premises inside Q . Fix $k \geq 1$. Start with $P_0 = \emptyset$ and $U_0 = N_0$ (finite), each element stamped with its time of entry. At step t :

1. select $u \in U_t$: if $k \mid t$, take the element with the oldest time stamp; otherwise any element;
2. set $P_{t+1} = P_t \cup \{u\}$;
3. add to the queue every conclusion c of a rule instance (Q, c) with $u \in Q \subseteq P_{t+1}$ that is not already present, stamped t .

If $U_t = \emptyset$ the loop halts.

Proposition 2.7 (The loop is fair). *The derivation $N_t = P_t \cup U_t$ produced by a given-clause strategy is fair.*

Proof. Each step adds finitely many elements by the finiteness assumption, so for each element e only finitely many elements enter before e . Let $\text{rk}_t(e)$ be the number of queue elements older than e at time t . Newly arriving elements are younger than e and do not increase $\text{rk}_t(e)$; at every k -th step the oldest element is removed, so while $\text{rk}_t(e) > 0$ it decreases by 1 at least once in every k steps. Hence e is selected within $k(\text{rk}_{t_0}(e) + 1)$ steps of its entry at time t_0 , so every element of N_∞ is eventually moved to P .

Now let (Q, c) be a rule instance with $Q \subseteq N_\infty$. Every element of Q is eventually processed. Let u be the element of Q processed last, at some step t . Then $Q \subseteq P_{t+1}$ and $u \in Q$, so c is added to the queue (or was already present) by step 3. Thus $c \in N_\infty$, and N_∞ is saturated. If the loop halts with $U = \emptyset$ the same argument applies to the finite set $N_\infty = P$. By Lemma 2.3 the derivation is fair. ■

The role of the age-based selection is to defeat starvation. A strategy that always prefers the element with the best heuristic score can postpone an old clause forever; the periodic oldest-first selection is a minimal concession to fairness, and it is the reason a heuristic guiding a fair loop can change the *order* in which derivations are found without changing *which* derivations are eventually found. Chapter 15 makes this the basis of a general statement about learned guidance.

2.3 Existence versus discovery

A transition system with a goal may have a run without any particular search finding it. Let $R = (\Sigma, \rightarrow, \iota, F, \text{ext})$ be as in Definition 1.8 of Chapter 1 and call s *live* if there is a path from s to F .

Proposition 2.8 (Iterative deepening finds a run when one exists). *Suppose the set of successors of a state is finite and effectively computable from the state (finite branching together with a decidable edge relation is not enough, since it need not yield a terminating enumeration of the successors), and that F is decidable. Let $\text{DFS}_L(\varphi)$ be depth-first search from $\iota(\varphi)$ that explores all paths of length at most L . Then the procedure that runs $\text{DFS}_1, \text{DFS}_2, \dots$ in turn halts with a run on φ if and only if $\iota(\varphi)$ is live, and then returns a run of minimal length.*

Proof. Finite branching makes the tree of paths of length at most L finite, so DFS_L terminates. If $\iota(\varphi)$ is live with a shortest run of length n , then DFS_L fails for $L < n$ and succeeds for $L = n$, returning a run of length n . If $\iota(\varphi)$ is not live no DFS_L succeeds and the iteration does not halt. ■

Plain depth-first search without a bound is not complete in this sense: on a system with an infinite path from $\iota(\varphi)$ that precedes the useful branch in its ordering it never returns. The proposition also exhibits the asymmetry that Chapter 1 named: on a non-live initial state the procedure does not halt, and cannot, without an additional argument, report that no run exists.

2.4 Three limits

Fix a proof system S for first-order logic: Φ is the set of first-order sentences over a finite signature containing at least one binary relation symbol, Val is the set of valid sentences, and

S is sound and complete with decidable Chk . Gödel's completeness theorem guarantees such systems exist, and we use:

Theorem 2.9 (Church 1936, Turing 1936). [CITED: CHURCH, TURING] *For a signature with at least one binary relation symbol, the set Val of valid first-order sentences is undecidable.*

Proposition 2.10 (No computable bound on derivation size). *There is no computable total function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that every valid φ has a derivation d with $\text{Chk}(d, \varphi)$ and $|d| \leq f(|\varphi|)$.*

Proof. Suppose f exists. Given φ , compute $f(|\varphi|)$, list the finitely many strings d with $|d| \leq f(|\varphi|)$, and run the decidable relation Chk on each pair (d, φ) . Accept if some check succeeds, otherwise reject. If φ is valid then by assumption and completeness some d within the bound passes. If some d passes then φ is valid by soundness. This decides Val , contradicting Theorem 2.9. ■

The proposition has an immediate bearing on the theme of this book. Because no computable function bounds the length of the shortest derivation, no procedure can promise that the compressed object of Chapter 1 is small relative to the statement it certifies. The residue discarded in extraction, or the derivation itself, may be larger than any bound one could state in advance.

Definition 2.11 (Finite countermodel). A sentence φ has a *finite countermodel* if some finite structure for the signature satisfies $\neg\varphi$. Let FM be the set of such sentences.

Proposition 2.12 (Two witnesses, two enumerations). *Both Val and FM are recursively enumerable and disjoint.*

Proof. Val is recursively enumerable by Proposition 1.2 of Chapter 1. For FM , enumerate pairs of a finite structure and a sentence and evaluate the sentence in the structure; evaluation in a finite structure is decidable. Disjointness is immediate: a structure falsifying φ shows φ is not valid. ■

Corollary 2.13 (Neither proved nor finitely refuted). $\text{Val} \cup \text{FM} \neq \Phi$. *Equivalently, there exist sentences that are not valid and have no finite countermodel; every such sentence is rejected by every combination of a sound derivation-finder and a sound finite-model-finder.*

Proof. If $\text{Val} \cup \text{FM} = \Phi$, then given φ run the enumerations of Val and FM in parallel; exactly one eventually produces φ , and the answer is read off. This decides Val , contradicting Theorem 2.9. ■

2.5 Outcomes of a bounded search

The proposition and corollary above justify a three-valued reading of the result of any practical attempt on a first-order goal.

Definition 2.14 (Outcomes). An attempt on φ with a prover and a finite-model finder returns one of

- $\text{Proved}(d)$ with $\text{Chk}(d, \varphi)$, from which $\varphi \in \text{Val}$ follows;
- $\text{Refuted}(m)$ with m a finite structure and $m \models \neg\varphi$, from which $\varphi \notin \text{Val}$ follows;

- Undetermined, from which nothing follows.

By Corollary 2.13, the third outcome cannot be eliminated by waiting longer or by adding a finite-model finder. It is a structural feature of first-order automated reasoning, and the discipline of non-certification of Chapter 1 is its operational consequence. Chapter 12 extends this three-valued vocabulary from statements to *transformations of verified artifacts*, where an analogous undetermined outcome has a different cause (leaving a certified fragment rather than exhausting a budget) and, as we will argue, a different meaning.

2.6 Summary

Fair strategies make refutationally complete calculi into semi-decision procedures (Theorem 2.4). Fairness is implemented, not assumed, by age-based selection (Proposition 2.7). Existence of a run and discovery of a run are different properties (Proposition 2.8). Three barriers survive any such implementation: no bound on derivation size (Proposition 2.10), non-termination on satisfiable inputs (Corollary 2.5), and an irreducible undetermined region (Corollary 2.13).

Exercise 2.1. Show that Theorem 2.4 fails without the soundness hypothesis, and that it fails without refutational completeness. In each case give an inference system and a fair derivation with the stated defect.

Exercise 2.2. In Definition 2.6, replace oldest-first selection every k -th step by selection of the element with minimal heuristic score at every step. Construct an infinite derivation in which some element is never selected, and explain what this does to Lemma 2.3 and to the ($\Rightarrow$) direction of Theorem 2.4.

Exercise 2.3. Prove that Proposition 2.10 remains true if $|\varphi|$ is replaced by any computable measure of φ that takes finitely many sentences to each value.

Chapter 3

Resolution, Unification, and Saturation

Resolution is the calculus on which the most influential automated provers were built, and it is also the cleanest laboratory for the questions of this book. It has one rule, a completeness proof short enough to give in full for the propositional case, and a first-order extension whose only new ingredient is unification, which can also be proved correct from scratch. At the end of the chapter, resolution is placed in the framework of Chapters 1 and 2: it is a proof system, its fair saturation is a producer, and the proof extracted from a saturation is a compression whose residue can be seen with the naked eye.

3.1 Propositional resolution

Fix a countable set A of propositional atoms. A *literal* is an atom p or a negated atom $\neg p$; write $\bar{L}$ for the complement of L . A *clause* is a finite set of literals, read as their disjunction, and the empty clause is written $\square$. A *valuation* $v : A \rightarrow \{0, 1\}$ satisfies a clause if it makes some member true, so $\square$ is satisfied by no valuation. A set of clauses is satisfied by v if every member is. A clause is a *tautology* if it contains both p and $\neg p$ for some p .

Definition 3.1 (Resolution rule). From clauses $C \cup \{p\}$ and $D \cup \{\neg p\}$ infer $C \cup D$. The inferred clause is the *resolvent on p* . Because clauses are sets, repeated literals in $C \cup D$ merge automatically, so no separate factoring rule is needed in the propositional case.

Take X to be the set of clauses, $\mathcal{M}$ the set of valuations, $\perp = \square$, and $\mathcal{R}$ the set of rule instances $(\{C \cup \{p\}, D \cup \{\neg p\}\}, C \cup D)$. This is an inference system in the sense of Definition 2.1.

Proposition 3.2 (Soundness). *Every instance of the resolution rule is sound.*

Proof. Let v satisfy $C \cup \{p\}$ and $D \cup \{\neg p\}$. If $v(p) = 1$ then v does not satisfy $\neg p$, so it satisfies some member of D and hence $C \cup D$. If $v(p) = 0$ then it does not satisfy p and satisfies some member of C . ■

3.2 Completeness by elimination

The completeness proof below eliminates one atom at a time. The elimination is the Davis–Putnam procedure, and Chapter 5 returns to it as the ancestor of conflict-driven learning.

Lemma 3.3 (Elimination preserves satisfiability). *Let N be a finite set of non-tautological clauses and p an atom. Split $N = E \cup P \cup Q$ where P is the set of clauses containing p , Q the set containing $\neg p$, and E the set containing neither. Let $\text{Res}_p(N)$ be the set of resolvents on p of a clause of P with a clause of Q , and put $N_p = E \cup \text{Res}_p(N)$. Then N is satisfiable if and only if N_p is satisfiable, and every clause of N_p is in N or is a resolvent of two clauses of N .*

Proof. The last claim is immediate from the construction. If $v \models N$ then v satisfies every resolvent by Proposition 3.2 and every clause of E , so $v \models N_p$.

Conversely, let $v \models N_p$; we may take v arbitrary on p and modify only that value. Write $P^- = \{C : C \cup \{p\} \in P\}$ and $Q^- = \{D : D \cup \{\neg p\} \in Q\}$, where C and D do not contain p or $\neg p$ because the clauses are non-tautological. The resolvents are exactly the clauses $C \cup D$ for $C \in P^-$, $D \in Q^-$.

Case 1: some $C_0 \in P^-$ is not satisfied by v . Then for every $D \in Q^-$ the resolvent $C_0 \cup D$ is satisfied by v , hence v satisfies D . Let v' agree with v except $v'(p) = 1$. Then v' satisfies all of P through p , all of Q because each D is satisfied by v and therefore by v' (which agrees with v off p), and all of E .

Case 2: every $C \in P^-$ is satisfied by v . Let v' agree with v except $v'(p) = 0$. Then v' satisfies all of Q through $\neg p$, all of P because every C is satisfied, and all of E .

In either case $v' \models N$. ■

Theorem 3.4 (Completeness for finite sets). *If N is a finite unsatisfiable set of propositional clauses, then $\square$ is derivable from N by finitely many resolution steps.*

Proof. By induction on the number k of atoms occurring in N .

Let N° be N with all tautologies removed. A valuation satisfies a tautology, so N° has the same models as N , and a derivation from N° is a derivation from N . So we may assume N contains no tautologies. If $\square \in N$ there is nothing to prove, which covers $k = 0$: with no atoms every clause is $\square$, and an unsatisfiable N must contain it.

Let $k \geq 1$ and $\square \notin N$, and choose an atom p occurring in N . By Lemma 3.3, N_p is finite, unsatisfiable, and mentions only atoms other than p , hence fewer than k atoms. By the induction hypothesis, $\square$ is derivable from N_p . Each clause of N_p is either in N or a resolvent of two clauses of N , so prefixing the derivation by the corresponding steps gives a derivation of $\square$ from N . ■

To pass from finite to arbitrary sets we use the compactness theorem for propositional logic.

Theorem 3.5 (Compactness). [CITED: GÖDEL, MAL'CEV] *A set of propositional clauses is satisfiable if and only if every finite subset is satisfiable.*

Corollary 3.6 (Refutational completeness). *Propositional resolution is sound and refutationally complete in the sense of Definition 2.1: every saturated unsatisfiable set of clauses contains $\square$.*

Proof. Let N be saturated and unsatisfiable. By compactness some finite $N_0 \subseteq N$ is unsatisfiable. By Theorem 3.4 there is a derivation $C_1, \dots, C_n = \square$ in which each C_i is in N_0 or a resolvent of earlier lines. By induction on i , each $C_i \in N$: lines in N_0 lie in N , and a resolvent of two members of N lies in N because N is saturated. Hence $\square = C_n \in N$. ■

Combined with Theorem 2.4, any fair derivation of resolvents from a finite set N_0 of clauses eventually contains $\square$ if and only if N_0 is unsatisfiable. For propositional clauses over finitely many atoms the closure is finite, so the procedure terminates on every input; this is the propositional face of the dichotomy that Church's theorem makes sharp at first order.

3.3 Unification

First-order clauses contain variables, and the resolution rule must find substitutions that make complementary literals equal. Fix a signature of function symbols with arities and a countable set $\mathcal{V}$ of variables. *Terms* are built freely: a variable, or $f(t_1, \dots, t_n)$ with f of arity n . The size $|t|$ of a term is the number of symbol occurrences. A *substitution* σ is a map from variables to terms that is the identity on all but finitely many variables, extended homomorphically to terms and written on the left. Substitutions compose as functions.

An *equation* is a pair $s \doteq t$ of terms. A substitution σ is a *unifier* of a finite set E of equations if $\sigma(s) = \sigma(t)$ for every $(s \doteq t) \in E$. A unifier σ is *most general* if every unifier θ of E factors as $\theta = \theta' \circ \sigma$ for some substitution θ' .

Definition 3.7 (Solved form). A set of equations is in *solved form* if it has the shape $\{x_1 \doteq t_1, \dots, x_m \doteq t_m\}$ with the x_i pairwise distinct variables, none of which occurs in any t_j . Its associated substitution is $\sigma_E = \{x_i \mapsto t_i\}$.

The following transformation rules, due to Martelli and Montanari in this form, act on a finite set E of equations.

- Delete: $E \cup \{t \doteq t\} \Rightarrow E$.
- Decompose: $E \cup \{f(s_1, \dots, s_n) \doteq f(t_1, \dots, t_n)\} \Rightarrow E \cup \{s_i \doteq t_i : 1 \leq i \leq n\}$.
- Conflict: $E \cup \{f(\dots) \doteq g(\dots)\} \Rightarrow \text{fail}$ if $f \neq g$.
- Orient: $E \cup \{t \doteq x\} \Rightarrow E \cup \{x \doteq t\}$ if t is not a variable.
- Occurs: $E \cup \{x \doteq t\} \Rightarrow \text{fail}$ if x occurs in t and $t \neq x$.
- Eliminate: $E \cup \{x \doteq t\} \Rightarrow E[x \mapsto t] \cup \{x \doteq t\}$ if $x \notin \text{Var}(t)$ and x occurs in E .

Constants are function symbols of arity zero, so Decompose with $n = 0$ coincides with Delete.

Theorem 3.8 (Unification). *Every sequence of applications of the rules terminates. It terminates either in fail, in which case E has no unifier, or in a solved form E' , in which case $\sigma_{E'}$ is a most general unifier of E .*

Proof. Rules preserve unifiers. For Delete the equation is satisfied by every substitution. For Decompose, $\sigma(f(\vec{s})) = \sigma(f(\vec{t}))$ if and only if $\sigma(s_i) = \sigma(t_i)$ for all i , because terms are built freely. Orient is symmetric. For Eliminate, if $\sigma(x) = \sigma(t)$ then $\sigma \circ \{x \mapsto t\} = \sigma$ as substitutions (on x both give $\sigma(t) = \sigma(x)$, and elsewhere both give $\sigma(y)$), so σ unifies E if and only if it unifies $E[x \mapsto t]$. For Conflict and Occurs there is no unifier of the displayed equation: heads differ in the first case, and in the second $\sigma(x) = \sigma(t)$ would force a term to be a

proper subterm of itself, which contradicts finiteness of size since $|\sigma(t)| > |\sigma(x)|$ whenever x occurs in $t \neq x$.

Termination. Call a variable *solved* in E if it is the left-hand side of an equation $x \doteq t$ with $x \notin \text{Var}(t)$ and x occurs nowhere else in E . Let $n_1(E)$ be the number of variables occurring in E that are not solved, $n_2(E)$ the total size of all terms in E , and $n_3(E)$ the number of equations of the form $t \doteq x$ with t not a variable. We check that each rule strictly decreases the triple (n_1, n_2, n_3) in the lexicographic order, with n_1 never increasing in the rules that decrease n_2 or n_3 .

- Delete: no variable becomes unsolved (n_1 does not increase) and n_2 drops by $2|t| \geq 2$.
- Decompose: the set of variables is unchanged; a solved variable is not in the decomposed equation, so it stays solved; n_2 drops by 2.
- Orient: the variable set and n_2 are unchanged; n_1 does not increase; n_3 drops by 1.
- Eliminate: after the step x occurs only in $x \doteq t$ and $x \notin \text{Var}(t)$, so x is solved, whereas before it was not solved because it occurred elsewhere. A variable solved before remains solved: it occurred only in its own equation, so it is neither x nor in t , and substituting t for x in the right-hand side of its equation cannot introduce it. Hence n_1 decreases by at least 1.

The lexicographic order on triples of naturals is well-founded, so the rules cannot be applied forever.

Normal forms. Suppose no rule applies and the result is not fail. Every equation has a variable on the left, else Orient, Decompose, Conflict or Delete would apply; none is of the form $x \doteq x$ or has x in its right-hand side, else Delete or Occurs would apply; and no left-hand variable occurs elsewhere in E , else Eliminate would apply. Thus E is in solved form.

Most general unifier. Let $E' = \{x_i \doteq t_i\}$ be in solved form and $\sigma = \sigma_{E'}$. Since no x_j occurs in any t_i we have $\sigma(t_i) = t_i = \sigma(x_i)$, so σ unifies E' . If θ is any unifier of E' then $\theta(x_i) = \theta(t_i)$, so $\theta \circ \sigma = \theta$ on each x_i and trivially elsewhere; thus $\theta = \theta \circ \sigma$ and σ is most general. Since the rules preserve unifiers, σ is a most general unifier of the original E . ■

3.4 First-order resolution

A first-order *literal* is an atomic formula $P(t_1, \dots, t_n)$ or its negation, a *clause* a finite set of literals with all variables implicitly universally quantified, and clauses are identified up to renaming of variables. A structure $\mathfrak{A}$ satisfies a clause if every assignment of the variables into $\mathfrak{A}$ satisfies some member.

Definition 3.9 (First-order resolution). *Binary resolution*: from clauses $C \cup \{A\}$ and $D \cup \{\neg B\}$ with no shared variables, where A, B are atoms with most general unifier σ , infer $\sigma(C \cup D)$. *Factoring*: from $C \cup \{A, B\}$ with A, B atoms of the same polarity and most general unifier σ of A and B , infer $\sigma(C \cup \{A\})$.

Proposition 3.10 (Soundness). *Binary resolution and factoring are sound: any structure satisfying the premises satisfies the conclusion.*

Proof. Let $\mathfrak{A}$ satisfy $C \cup \{A\}$ and $D \cup \{\neg B\}$ and let a be an assignment. Define an assignment b by $b(x) = \llbracket \sigma(x) \rrbracket^{\mathfrak{A}, a}$. By the substitution lemma, every term t satisfies $\llbracket t \rrbracket^{\mathfrak{A}, b} = \llbracket \sigma(t) \rrbracket^{\mathfrak{A}, a}$, and hence the same holds for literals. Since $\sigma(A) = \sigma(B)$, either A and B are both true

under b or both false. If both true, $\neg B$ is false under b , so D has a literal true under b , i.e. $\sigma(D)$ has a literal true under a . If both false, C has a literal true under b , i.e. $\sigma(C)$ has one under a . Either way $\sigma(C \cup D)$ is satisfied under a . The argument for factoring is the same with one premise. ■

Completeness is a consequence of two standard results about grounding, which we cite. A *ground instance* of a clause is the result of replacing all its variables by closed terms.

Theorem 3.11 (Herbrand). [CITED: HERBRAND 1930, SKOLEM, GÖDEL] *A set N of first-order clauses is unsatisfiable if and only if some finite set of ground instances of members of N is unsatisfiable as a set of propositional clauses.*

Lemma 3.12 (Lifting). [CITED: ROBINSON 1965] *Let C_1, C_2 be clauses without shared variables, C'_1, C'_2 ground instances of them, and C' a propositional resolvent of C'_1, C'_2 . Then there is a first-order resolvent C of C_1, C_2 , possibly after factoring C_1 and C_2 , of which C' is a ground instance.*

Theorem 3.13 (Refutational completeness of resolution). *First-order binary resolution with factoring is sound and refutationally complete: if N is unsatisfiable then $\square$ is derivable from N , and every saturated unsatisfiable N contains $\square$.*

Proof. By Theorem 3.11 there is a finite unsatisfiable set N' of ground instances of clauses of N . By Theorem 3.4 there is a propositional resolution derivation $C'_1, \dots, C'_n = \square$ from N' . By induction on i we construct clauses C_i derivable from N by first-order resolution and factoring such that C'_i is a ground instance of C_i . If $C'_i \in N'$, take C_i to be the member of N of which it is an instance. If C'_i is a propositional resolvent of C'_j, C'_k , apply Lemma 3.12 to C_j, C_k . The only ground instance of a clause that equals $\square$ is an instance of $\square$ itself, so $C_n = \square$. The claim about saturated sets follows as in Corollary 3.6: every C_i lies in N by induction. ■

Theorem 3.13 together with Theorem 2.4 gives the usual conclusion: a fair saturation of resolvents, for instance by the given-clause loop of Proposition 2.7, is a semi-decision procedure for unsatisfiability of first-order clause sets, and by Theorem 2.9 it cannot be improved to a decision procedure.

3.5 Resolution as a proof system, and what extraction discards

Resolution is a proof system in the sense of Definition 1.1, a *refutation* system. Let Φ be the set of finite sets of clauses, Val the unsatisfiable ones, D the set of finite sequences of clauses annotated with their justification (an input clause, or a resolution or factoring step citing earlier lines and a unifier), and $\text{Chk}(d, N)$ the relation that holds when each annotation is verified by the unification algorithm of Theorem 3.8 and the last line is $\square$. Then Chk is decidable, and the system is sound and complete by Proposition 3.10 and Theorem 3.13.

The producer is a saturation. A given-clause run generates a possibly enormous set N_∞ of clauses, and extraction keeps only those on which $\square$ depends.

Definition 3.14 (Proof dag). Given a run that generates $\square$, record for each generated clause its parents. The *proof dag* is the sub-dag consisting of $\square$ and all of its ancestors. The extracted derivation d is any linearization of the proof dag consistent with the parent relation.

Example 3.15 (Unused material). Let $N = \{ \{p\}, \{\neg p, q\}, \{\neg q\}, \{r, s\}, \{\neg r, s\} \}$. A saturation derives $\{q\}$ from the first two clauses, $\square$ from $\{q\}$ and $\{\neg q\}$, and also $\{s\}$ from the last two clauses. In a run that first obtains $\square$ through $\{q\}$, the proof dag of $\square$ contains exactly $\{p\}, \{\neg p, q\}, \{\neg q\}, \{q\}, \square$. The clauses $\{r, s\}, \{\neg r, s\}, \{s\}$ are in the run and not in the derivation. A different order of the saturation, which resolves q against $\neg q$ before touching the r, s clauses, yields the same derivation d with a different trace.

Example 3.15 is tiny, but it instantiates Definition 1.11 of Chapter 1 precisely. Two runs on the same N with different traces and the same extracted derivation share a fibre, and the r, s clauses are part of the trace and no part of the output. Nothing in the framework bounds the unused portion in terms of the derivation, and Proposition 2.10 shows that even the derivation's own size admits no computable bound in terms of the statement. A task that depends on what *was tried* (for instance, which lemma candidates were found irrelevant, which instance of a heuristic parameter produced the proof) is not a function of d and cannot be served by it, in the sense of Proposition 1.12.

This is not a defect of resolution. The extraction of Definition 3.14 is exactly the compression a proof checker needs, and it is lossless with respect to the one task the checker serves. The point of the book is that other tasks exist, and that the same compression is lossy for them.

Exercise 3.1. Identify exactly where the proof of Lemma 3.3 uses the assumption that no clause is a tautology, and explain what the splitting $N = E \cup P \cup Q$ would mean for a clause containing both p and $\neg p$.

Exercise 3.2. Prove that Eliminate must carry the side condition “ x occurs in E ”: without it, show the rule system does not terminate.

Exercise 3.3. Show that the clauses $\{P(x, x)\}$ and $\{\neg P(y, f(y))\}$ have no resolvent because unification fails by the occurs check, and verify that no pair of their ground instances admits a propositional resolution step.

Chapter 4

Rewriting and Normalization

Rewriting is the second great engine of automated reasoning, and it differs from resolution in a way that matters for this book. Resolution derives new facts and keeps all of them until a contradiction appears. Rewriting *replaces*: a term is repeatedly simplified, the earlier forms are discarded, and what remains is a canonical representative. A rewriting system is thus a machine that turns a family of equal expressions into a single one, and the structure it forgets in doing so can be stated exactly. This chapter proves the basic theory from the ground up, namely confluence, Newman's lemma and termination by interpretation, cites the critical pair lemma and the correctness of completion, and ends with the observation that a normal form is the extreme case of a compression that is sufficient for one task and insufficient for others.

4.1 Abstract reduction systems

An *abstract reduction system* is a pair $(A, \rightarrow)$ with $\rightarrow \subseteq A \times A$. Write $\rightarrow^+$ and $\rightarrow^*$ for the transitive and reflexive-transitive closures, $\leftarrow$ for the converse, and $\leftrightarrow^*$ for the equivalence generated by $\rightarrow$. Elements a, b are *joinable*, written $a \downarrow b$, if $a \rightarrow^* c \leftarrow^* b$ for some c . An element is a *normal form* if no step leaves it, and b is a normal form of a if $a \rightarrow^* b$ and b is a normal form.

Definition 4.1 (Termination and confluence). The system $(A, \rightarrow)$ is

- *terminating* if there is no infinite sequence $a_0 \rightarrow a_1 \rightarrow a_2 \rightarrow \dots$;
- *confluent* if $b \leftarrow^* a \rightarrow^* c$ implies $b \downarrow c$;
- *locally confluent* if $b \leftarrow a \rightarrow c$ implies $b \downarrow c$;
- *Church–Rosser* if $a \leftrightarrow^* b$ implies $a \downarrow b$;
- *convergent* if it is terminating and confluent.

Lemma 4.2 (Confluence is the Church–Rosser property). $(A, \rightarrow)$ is confluent if and only if it is Church–Rosser.

Proof. ($\Leftarrow$) If $b \leftarrow^* a \rightarrow^* c$ then $b \leftrightarrow^* c$, so $b \downarrow c$.

($\Rightarrow$) Suppose $a \leftrightarrow^* b$, by induction on the length of a zigzag from a to b . Length zero is trivial. Otherwise $a \leftrightarrow^* b'$ by a shorter zigzag, and $b' \rightarrow b$ or $b' \leftarrow b$. By the induction hypothesis $a \rightarrow^* c \leftarrow^* b'$. If $b \rightarrow b'$ then $b \rightarrow b' \rightarrow^* c$, so $a \rightarrow^* c \leftarrow^* b$. If $b' \rightarrow b$ then $c \leftarrow^* b' \rightarrow b$, and confluence gives $c \rightarrow^* d \leftarrow^* b$, so $a \rightarrow^* d \leftarrow^* b$. ■

Theorem 4.3 (Newman’s lemma, 1942). *A terminating and locally confluent system is confluent.*

Proof. Since the system is terminating, $\rightarrow^+$ is well-founded and we may argue by well-founded induction. Let $P(a)$ say that $b \leftarrow^* a \rightarrow^* c$ implies $b \downarrow c$. Assume $P(a')$ for every a' with $a \rightarrow^+ a'$, and let $b \leftarrow^* a \rightarrow^* c$. If $a = b$ then $b = a \rightarrow^* c$ so $b \downarrow c$, and symmetrically if $a = c$. Otherwise $a \rightarrow a_1 \rightarrow^* b$ and $a \rightarrow a_2 \rightarrow^* c$, and by local confluence $a_1 \rightarrow^* d \leftarrow^* a_2$.

The induction hypothesis holds at a_1 and at a_2 , since $a \rightarrow a_1$ and $a \rightarrow a_2$. Applying it at a_1 to $b \leftarrow^* a_1 \rightarrow^* d$ gives $b \rightarrow^* e \leftarrow^* d$. Applying it at a_2 to $d \leftarrow^* a_2 \rightarrow^* c$ gives $d \rightarrow^* f \leftarrow^* c$. Now $a \rightarrow a_1 \rightarrow^* d$ gives $a \rightarrow^+ d$, so the induction hypothesis holds at d , and applying it to $e \leftarrow^* d \rightarrow^* f$ gives $e \rightarrow^* g \leftarrow^* f$. Then $b \rightarrow^* e \rightarrow^* g$ and $c \rightarrow^* f \rightarrow^* g$, so $b \downarrow c$. ■

Corollary 4.4 (Unique normal forms). *Let $(A, \rightarrow)$ be convergent. Every $a \in A$ has exactly one normal form $\text{NF}(a)$, and $a \leftrightarrow^* b$ if and only if $\text{NF}(a) = \text{NF}(b)$. If in addition the successors of any element are computable, then $\leftrightarrow^*$ is decidable.*

Proof. A normal form exists by termination, since otherwise one could step forever. If b, c are normal forms of a then $b \leftarrow^* a \rightarrow^* c$, so $b \rightarrow^* d \leftarrow^* c$ by confluence; as b, c admit no steps, $b = d = c$.

If $a \leftrightarrow^* b$ then Lemma 4.2 gives $a \rightarrow^* e \leftarrow^* b$, hence $\text{NF}(a) = \text{NF}(e) = \text{NF}(b)$ by uniqueness. Conversely, $a \rightarrow^* \text{NF}(a) = \text{NF}(b) \leftarrow^* b$ gives $a \leftrightarrow^* b$. For the last claim, compute $\text{NF}(a)$ and $\text{NF}(b)$ by following any sequence of steps, which halts by termination, and compare. ■

Corollary 4.4 is the abstract content of every canonizer. Note what it does *not* say: all maximal reduction sequences from a end at the same element, but nothing is claimed about their lengths or their contents.

Proposition 4.5 (Strategy independence of the output). *In a convergent system, every maximal reduction sequence from a is finite and ends at $\text{NF}(a)$. In particular every strategy for choosing the next step, adaptive or not, has the same output, and strategies differ only in the traces they produce.*

Proof. A maximal sequence is finite by termination and ends in a normal form of a , which is $\text{NF}(a)$ by Corollary 4.4. ■

Example 4.6 (Same output, different traces). Take rules $a \rightarrow b$ and $h(x) \rightarrow d(x, x)$ over a signature with constants a, b , a unary h and a binary d . The rules have no overlap, and the system is terminating. Reducing $h(a)$ innermost gives $h(a) \rightarrow h(b) \rightarrow d(b, b)$ in two steps; reducing outermost gives $h(a) \rightarrow d(a, a) \rightarrow d(b, a) \rightarrow d(b, b)$ in three. The outputs coincide, in agreement with the proposition, while both the length and the rules used differ.

Proposition 4.5 places rewriting at the opposite pole from the search of Chapter 2. There, strategy affects which derivation is found and whether one is found at all. Here, in a convergent system, strategy affects only cost. This is the formal reason that rewriting-based canonization is a safe component of a larger prover: it is a function, not a search.

4.2 Term rewriting and termination by interpretation

Fix a signature as in Section 3.3 of Chapter 3. A *rewrite rule* is a pair $l \rightarrow r$ of terms with l not a variable and $\text{Var}(r) \subseteq \text{Var}(l)$. A *term rewriting system* (TRS) $\mathcal{R}$ is a finite set of rules. The rewrite relation is

$$s \rightarrow_{\mathcal{R}} t \iff s|_p = \sigma(l) \text{ and } t = s[\sigma(r)]_p \text{ for some rule } l \rightarrow r, \text{ position } p, \text{ substitution } \sigma,$$

where $s|_p$ is the subterm at position p and $s[u]_p$ is s with that subterm replaced by u .

The most direct method for proving termination assigns each symbol a function into a well-founded set.

Proposition 4.7 (Termination by monotone interpretation). *Let each n -ary symbol f be interpreted by a function $[f] : \mathbb{N}^n \rightarrow \mathbb{N}$ that is strictly increasing in each argument. For an assignment $\alpha : \mathcal{V} \rightarrow \mathbb{N}$ define $[t]_{\alpha}$ by $[x]_{\alpha} = \alpha(x)$ and $[f(t_1, \dots, t_n)]_{\alpha} = [f]([t_1]_{\alpha}, \dots, [t_n]_{\alpha})$. If $[l]_{\alpha} > [r]_{\alpha}$ for every rule $l \rightarrow r$ and every assignment α , then $\rightarrow_{\mathcal{R}}$ is terminating.*

Proof. We show that $s \rightarrow_{\mathcal{R}} t$ implies $[s]_{\alpha} > [t]_{\alpha}$ for every α . For a substitution σ and assignment α put $\alpha_{\sigma}(x) = [\sigma(x)]_{\alpha}$; by induction on terms, $[\sigma(u)]_{\alpha} = [u]_{\alpha_{\sigma}}$. Hence for a step at the root, $s = \sigma(l)$ and $t = \sigma(r)$, we get $[s]_{\alpha} = [l]_{\alpha_{\sigma}} > [r]_{\alpha_{\sigma}} = [t]_{\alpha}$. For a step at a deeper position, induct on the depth of p : if $s = f(s_1, \dots, s_i, \dots, s_n)$ and the step takes place inside s_i , giving s'_i with $[s_i]_{\alpha} > [s'_i]_{\alpha}$, then strict monotonicity of $[f]$ in its i -th argument gives $[s]_{\alpha} > [t]_{\alpha}$. An infinite reduction $s_0 \rightarrow s_1 \rightarrow \dots$ would give, for any fixed α , an infinite strictly decreasing sequence $[s_0]_{\alpha} > [s_1]_{\alpha} > \dots$ in $\mathbb{N}$, which is impossible. ■

Example 4.8 (Addition). Let $\mathcal{R}_+$ consist of $\text{add}(0, y) \rightarrow y$ and $\text{add}(s(x), y) \rightarrow s(\text{add}(x, y))$. Put $[0] = 1$, $[s](x) = x + 1$, $[\text{add}](x, y) = 2x + y$. All three are strictly increasing in each argument. For the first rule, $[\text{add}(0, y)]_{\alpha} = 2 + \alpha(y) > \alpha(y)$. For the second, $[\text{add}(s(x), y)]_{\alpha} = 2(\alpha(x) + 1) + \alpha(y) = 2\alpha(x) + \alpha(y) + 2$, which exceeds $[s(\text{add}(x, y))]_{\alpha} = 2\alpha(x) + \alpha(y) + 1$. By Proposition 4.7, $\mathcal{R}_+$ terminates. The two left-hand sides have no overlap, so there are no critical pairs in the sense of the next section, and $\mathcal{R}_+$ is convergent.

4.3 Critical pairs and local confluence

Because termination is available by Proposition 4.7 and Theorem 4.3 reduces confluence to a local condition, what remains is to check local confluence of a TRS. The check is finite, and it is carried out by unification.

Definition 4.9 (Critical pair). Let $l_1 \rightarrow r_1$ and $l_2 \rightarrow r_2$ be rules renamed to share no variables, and let p be a position of l_1 such that $l_1|_p$ is not a variable and unifies with l_2 with most general unifier σ . If p is the root then the two rules must not be variants of each other. The pair

$$\langle \sigma(r_1), \sigma(l_1[r_2]_p) \rangle$$

is a *critical pair*. It is *joinable* if its two components are joinable under $\rightarrow_{\mathcal{R}}$.

Lemma 4.10 (Critical pair lemma). [CITED: KNUTH–BENDIX 1970, HUET 1980] *A TRS is locally confluent if and only if all its critical pairs are joinable.*

Because a finite TRS has finitely many critical pairs, each computed by the unification procedure of Theorem 3.8, the lemma yields a decision procedure for confluence of terminating systems.

Proposition 4.11 (Confluence of terminating TRS is decidable). *There is an algorithm that, given a finite terminating TRS, decides whether it is confluent.*

Proof. Compute the finite set of critical pairs. For each pair, enumerate the reducts of each component. In a terminating, finitely branching system the set of reducts of a term is finite: otherwise by König's lemma the finitely branching tree of reduction sequences would contain an infinite branch, contradicting termination. So joinability is decidable by checking whether the two finite reduct sets intersect. By Lemma 4.10, the system is locally confluent exactly when all pairs are joinable, and by Theorem 4.3 local confluence is confluence for terminating systems. ■

Example 4.12 (Groups). Over the signature $\{\cdot, ^{-1}, e\}$ consider the ten rules

$$\begin{array}{lll} e \cdot x \rightarrow x & x^{-1} \cdot x \rightarrow e & (x \cdot y) \cdot z \rightarrow x \cdot (y \cdot z) \\ x^{-1} \cdot (x \cdot y) \rightarrow y & x \cdot e \rightarrow x & e^{-1} \rightarrow e \\ (x^{-1})^{-1} \rightarrow x & x \cdot x^{-1} \rightarrow e & x \cdot (x^{-1} \cdot y) \rightarrow y \\ (x \cdot y)^{-1} \rightarrow y^{-1} \cdot x^{-1} & & \end{array}$$

This is the classical completion of the three group axioms, due to Knuth and Bendix. Termination follows from the lexicographic path order with precedence $^{-1} > \cdot > e$, a standard result that we cite rather than reprove. Local confluence was checked in preparation of this book by computing every critical pair over all ordered pairs of rules and all non-variable positions, 55 in total, and normalizing each side: all were joinable. This is a computation by a short independent program and has not been mechanized in a proof assistant. Granting termination, Proposition 4.11 then makes the system convergent, so by Corollary 4.4 two group terms are equal in every group if and only if they have the same normal form (using the equational completeness theorem recalled below).

4.4 Equational theories and completion

Let E be a set of equations $s \approx t$ between terms. The relation $\leftrightarrow_E$ is the symmetric closure of single rewrite steps using equations of E in both directions, at any position and under any substitution, and $\leftrightarrow_E^*$ is its reflexive-transitive closure. Write $E \models s \approx t$ if every algebra satisfying all equations of E satisfies $s \approx t$ under every assignment.

Theorem 4.13 (Birkhoff 1935). *[CITED: BIRKHOFF] $E \models s \approx t$ if and only if $s \leftrightarrow_E^* t$.*

Birkhoff's theorem reduces equational validity to reachability in an abstract reduction system, so the machinery of Section 4.1 applies. If E has an equivalent presentation $\mathcal{R}$ in which every equation is oriented as a rule, and $\mathcal{R}$ is convergent, then $E \models s \approx t$ is decided by comparing normal forms (Corollary 4.4). *Completion* searches for such an $\mathcal{R}$. Given a reduction order $\succ$ (a well-founded order on terms closed under contexts and substitutions), the Knuth–Bendix procedure maintains a pair $(E_i, \mathcal{R}_i)$ of unoriented equations and oriented rules, starting from $(E, \emptyset)$, using the inferences

- Orient: move $s \approx t$ from E_i to $\mathcal{R}_i$ as $s \rightarrow t$ if $s \succ t$ (or as $t \rightarrow s$ if $t \succ s$);

- Deduce: add $s \approx t$ to E_i when $\langle s, t \rangle$ is a critical pair of $\mathcal{R}_i$;
- Simplify: rewrite a side of an equation, or the right-hand side of a rule, by $\mathcal{R}_i$;
- Delete: remove an equation $s \approx s$.

Theorem 4.14 (Correctness of completion). [CITED: BACHMAIR, DERSHOWITZ, HSIANG 1986]
*If a run of the procedure is fair, in the sense that every critical pair between rules that persist from some stage onward is eventually deduced, and the run never meets an equation that is unorientable by $\succ$ and ends with no unoriented equations, then the limit system $\mathcal{R}_\infty$ is convergent and $\leftrightarrow^*_{\mathcal{R}_\infty}$ coincides with $\leftrightarrow^*_E$. The procedure can fail on an unorientable equation, and can run forever.*

Completion is a fairness-governed search in the spirit of Chapter 2 (it deletes as well as adds, so Definition 2.2 does not apply literally), and the discipline of non-certification applies to it verbatim. If completion fails or does not terminate, nothing follows about the decidability of the word problem for E , which may be decidable by other means. If it succeeds, the result is a canonizer whose assurance rests on two ingredients: termination of $\mathcal{R}_\infty$ under $\succ$, and the joinability of its critical pairs, both of which a checker can verify afterwards without trusting the run.

4.5 Normal forms as a canonical residue

Let $\mathcal{R}$ be convergent. The map $\text{NF} : T \rightarrow T$ from terms to normal forms is idempotent, sends every term to an equal term, and identifies exactly the $\leftrightarrow^*$ -equivalent terms (Corollary 4.4). It is the extreme case of the compression studied in this book: a whole equivalence class, together with every path inside it, is replaced by one representative.

For a task, understood as in Proposition 1.12 of Chapter 1 as a function on the full *reduction sequence* rather than on its endpoint, Proposition 4.5 says that all sequences from s lie in a single fibre of the output map.

Proposition 4.15 (What normalization serves and what it cannot). *Let $\mathcal{R}$ be convergent and let ϱ map a maximal reduction sequence from s to its last term.*

- (i) *The task “decide whether $s \leftrightarrow^* t$ ” factors through ϱ .*
- (ii) *In Example 4.6, the task “report the number of rewrite steps used” does not factor through ϱ , and neither does “report whether the rule $a \rightarrow b$ was applied before $h(x) \rightarrow d(x, x)$ ”.*

Proof. (i) $s \leftrightarrow^* t$ if and only if $\text{NF}(s) = \text{NF}(t)$ by Corollary 4.4, and $\text{NF}(s) = \varrho(r)$ for every maximal sequence r from s . So the answer is a function of the endpoints. (ii) The two sequences from $h(a)$ in Example 4.6 have the same endpoint $d(b, b)$ and different lengths, and in the second the rule $a \rightarrow b$ is applied after $h(x) \rightarrow d(x, x)$ while in the first it is applied before. By Proposition 1.12 neither task factors through ϱ . ■

The proposition is trivial to state and it carries the central message of Part I. A representation is not sufficient or insufficient in the abstract; it is sufficient *for a declared task*. Normalization is sufficient for equality and for nothing that depends on how equality was reached. Resolution proofs (Chapter 3) are sufficient for checking and for nothing that depends on what was tried. Part II shows that SAT solvers, SMT solvers, and kernels exhibit the same pattern at industrial scale, and Part III turns the pattern into a theory.

Exercise 4.1. Show that the converse of Newman’s lemma fails: exhibit a confluent system that is not terminating, and a locally confluent one that is neither terminating nor confluent.

Exercise 4.2. Verify by hand the termination argument of Example 4.8 for $\text{add}(s(0), s(0))$ and compute its normal form by both innermost and outermost reduction.

Exercise 4.3. For the group system of Example 4.12, compute the normal forms of $(a \cdot b)^{-1} \cdot a$ and of b^{-1} , and decide whether they are equal in every group.

Part II

Decision Procedures and Kernels

Chapter 5

SAT and Clause Learning as Resolution with Memory

Modern propositional solvers decide satisfiability of formulas with millions of clauses, and they do so by a procedure whose logic is almost entirely resolution. This chapter makes the claim exact. A conflict-driven solver is a transition system whose learned clauses are resolution consequences, so every refutation it finds is a resolution refutation, and the trace of the search can be compressed into a certificate that an independent checker accepts. The same material supplies the first fully worked case of a recurring theme: an optimization that is perfectly sound for deciding satisfiability can discard exactly the information needed to produce a model, and the discarded information can be named.

5.1 Clauses, trails, and unit propagation

Notation follows Chapter 3. A *partial assignment* is represented as a *trail* M , a finite sequence of literals containing no complementary pair. Each element of the trail is either a *decision* literal, written l^d , or a *propagated* literal. We write $M \models \neg C$ if for every literal $l \in C$ the complement $\bar{l}$ occurs in M , and say M *falsifies* C . A literal is *undefined* in M if neither it nor its complement occurs.

A clause $C \cup \{l\}$ is *unit under* M with unit literal l if $M \models \neg C$ and l is undefined in M . *Unit propagation* extends M by unit literals until no clause is unit; it *refutes* M against a clause set F if some clause of F ends up falsified. A run of unit propagation is computable in time linear in the size of F with the standard watching schemes, and the only fact we need is that the result of propagation is well-defined as a refutation or non-refutation (it is confluent in this sense; see Exercise 5.1).

5.2 Conflict-driven clause learning as a transition system

We follow the abstract presentation of Nieuwenhuis, Oliveras and Tinelli. A state is either Fail or a pair $\langle M \parallel F \rangle$ of a trail and a finite set F of clauses. The initial state for a formula F_0 is $\langle \varepsilon \parallel F_0 \rangle$, with the empty trail.

Definition 5.1 (Abstract CDCL). The transition rules are as follows.

UnitPropagate $\langle M \parallel F \rangle \Rightarrow \langle M l \parallel F \rangle$ if some $C \cup \{l\} \in F$ is unit under M with unit literal l .

Decide $\langle M \parallel F \rangle \Rightarrow \langle M l^d \parallel F \rangle$ if l or $\bar{l}$ occurs in F and l is undefined in M .

Fail $\langle M \parallel F \rangle \Rightarrow \text{Fail}$ if $M \models \neg C$ for some $C \in F$ and M contains no decision literal.

Backjump $\langle M' l^d M'' \parallel F \rangle \Rightarrow \langle M' l' \parallel F \cup \{C'\} \rangle$ if some $C \in F$ is falsified by $M' l^d M''$, and there is a clause $C' = l' \cup D'$ such that $F \models C'$, $M' \models \neg D'$, and l' is undefined in M' .

The clause C' is called *asserting*: it is unit under M' . Backjump abstracts conflict analysis, learning, and backjumping together. It leaves the choice of C' open except for the requirement $F \models C'$; Section 5.3 shows what choices practical solvers make.

Lemma 5.2 (Invariants). *Let $\langle M \parallel F \rangle$ be reachable from $\langle \varepsilon \parallel F_0 \rangle$. Then*

- (i) F and F_0 have the same models;
- (ii) every literal l occurring in M before the first decision literal satisfies $F_0 \models l$.

Proof. By induction on the length of the run. Both hold initially.

(i) UnitPropagate, Decide, Fail leave F unchanged. Backjump adds C' with $F \models C'$, so $F \cup \{C'\}$ and F have the same models, and these are the models of F_0 by the induction hypothesis.

(ii) Call the literals before the first decision the *base*. Decide adds a decision, so the base is unchanged. UnitPropagate: if M contains a decision then the new literal lies after it and the base is unchanged; otherwise the whole of M is base, each member entailed by F_0 by hypothesis, and the new literal l comes from $C \cup \{l\} \in F$ with $M \models \neg C$. Then $F_0 \models \bar{c}$ for each $c \in C$, and $F_0 \models C \cup \{l\}$ by (i), so $F_0 \models l$. Backjump: the new trail $M' l'$ has M' as a prefix of the old trail. If M' contains a decision, its base is the old base, and l' lies after that decision. Otherwise M' is part of the old base, so each member is entailed by F_0 . Then $M' \models \neg D'$ gives $F_0 \models \bar{d}$ for each $d \in D'$, and $F \models l' \cup D'$ with (i) gives $F_0 \models l'$. ■

Theorem 5.3 (Soundness of failure). *If a run from $\langle \varepsilon \parallel F_0 \rangle$ reaches Fail, then F_0 is unsatisfiable. If a run reaches a state $\langle M \parallel F \rangle$ in which every atom of F is defined in M and no clause of F is falsified by M , then M satisfies F_0 .*

Proof. Suppose the last step is Fail from $\langle M \parallel F \rangle$ with $M \models \neg C$ and $C \in F$. M has no decision, so all of M is base and by Lemma 5.2(ii) $F_0 \models \bar{c}$ for every $c \in C$. Also $F_0 \models C$ by (i). A model of F_0 would falsify every member of C and satisfy one, which is impossible.

For the second claim, if M is total on the atoms of F and falsifies no clause then it satisfies every clause of F , hence by (i) it satisfies F_0 . ■

The converse, that a maximal run on an unsatisfiable input ends in Fail and that runs terminate, requires a termination argument that depends on how C' is chosen.

Theorem 5.4 (Termination, cited). [CITED: NIEUWENHUIS, OLIVERAS, TINELLI 2006, QUOTED IN THE FORM NEEDED HERE] *Suppose that in every Backjump step the clause C' is not already in F , and that Backjump is applied whenever the trail falsifies a clause and contains a decision (an asserting clause exists in that case, by conflict analysis to the first unique implication point). Then every run from $\langle \varepsilon \parallel F_0 \rangle$ is finite. A maximal run on an unsatisfiable F_0 ends in Fail, and on a satisfiable F_0 ends in a state $\langle M \parallel F \rangle$ in which M is total and satisfies F_0 .*

Together, Theorems 5.3 and 5.4 make the system a decision procedure for propositional satisfiability. Notice the structure it shares with Chapter 2: soundness of the verdict does not depend on the heuristics that drive Decide and the choice of C' , whereas termination does depend on a non-repetition condition. This is Theorem 1.5 again.

5.3 Conflict analysis is resolution

The rule Backjump requires only $F \models C'$. Practical solvers choose C' by a specific procedure, and this procedure is a sequence of resolution steps. For a propagated literal l in M , a *reason* for l is a clause $C \cup \{l\} \in F$ with $M_{<l} \models \neg C$, where $M_{<l}$ is the prefix of M before l .

Definition 5.5 (Conflict analysis). Let $C_0 \in F$ be falsified by M . For $i \geq 0$, if C_i contains a literal $\bar{l}$ with l a propagated literal of M , let l be the one occurring *latest* in M among those, take its reason $R = D_i \cup \{l\}$, and put

$$C_{i+1} = (C_i \setminus \{\bar{l}\}) \cup D_i,$$

the resolvent of C_i and R on l . Stop when no such literal remains.

Proposition 5.6 (Analysis steps are resolution steps). *Each C_{i+1} is a resolvent of C_i and a clause of F , so each C_i has a resolution derivation from F with at most i steps. The process terminates after at most as many steps as M has propagated literals. Every C_i is falsified by M .*

Proof. The first claim is the definition of resolution (Definition 3.1). Falsification: $M \models \neg C_0$; if $M \models \neg C_i$ and l is as in the definition, then $M \models \neg D_i$ is witnessed by the prefix $M_{<l}$ of M , and $\bar{l} \in C_i$ is removed, so $M \models \neg C_{i+1}$. For termination, every literal added to C_{i+1} has its complement in $M_{<l}$, so strictly earlier in M than l . Hence the latest propagated literal whose complement lies in C_{i+1} is strictly earlier than l , and the sequence of literals resolved on is strictly decreasing in trail position; each propagated literal is used at most once. ■

The standard stopping criterion (first unique implication point) halts the process at a clause that is falsified by M and contains exactly one literal assigned at the current decision level; this is asserting after backjumping to the second-highest level occurring in it. We do not need the criterion to state the following consequence.

Theorem 5.7 (CDCL refutations are resolution refutations). *Suppose a run from $\langle \varepsilon \parallel F_0 \rangle$ reaches Fail and every clause added by Backjump is the last or an intermediate clause C_i of a conflict analysis as in Definition 5.5, started at a falsified clause of the current F . Then F_0 has a resolution refutation in which every added clause is a line, and the total number of resolution steps is at most $\sum_j |M_j|$, where M_j ranges over the trails at the conflicts that occurred.*

Proof. By induction along the run, every clause in F has a resolution derivation from F_0 : initial clauses trivially, and each clause added by Backjump by Proposition 5.6, whose premises are clauses of the current F . At the final Fail, M has no decision, so every literal of M is propagated and has a reason. Let C_0 be the falsified clause. Running the analysis of Definition 5.5 until no propagated literal remains, with no decision literal to block it, leaves a clause C_n with no literals at all: every literal $c \in C_n$ would have $\bar{c} \in M$ propagated, and so would be resolved away. Hence $C_n = \square$. The step count follows from the proposition. ■

The theorem identifies the *proof system* that a conflict-driven solver searches in: general resolution, as a dag in which learned clauses are reusable lines. It does not claim that the solver finds short resolution proofs. The converse question, which proofs a CDCL search can find, has a definite answer that we quote.

Theorem 5.8 (Simulation, cited). [CITED: PIPATSRISAWAT AND DARWICHE 2011; ATSERIAS, FICHTE, THURLEY 2011] *CDCL with non-deterministic decisions and restarts polynomially simulates general resolution: for every unsatisfiable F_0 with a resolution refutation of size s , there is a run reaching Fail with a number of conflicts polynomial in s and in the number of variables. The exact side conditions on the learning scheme are in the sources.*

Theorem 5.9 (Haken 1985, cited). [CITED: HAKEN] *Every resolution refutation of the pigeonhole formula PHP_n^{n+1} has size $2^{\Omega(n)}$.*

Theorems 5.7 to 5.9 show that the compression from a search trace to a resolution proof is neither lossless nor bounded. CDCL on pigeonhole formulas is exponential because the proof system is, whatever the heuristics. This is a different barrier from the uncomputable bound of Proposition 2.10: it is a concrete, quantitative gap, established for a specific family.

5.4 Certificates: RUP, RAT, and DRAT

A solver that answers “satisfiable” hands back an assignment, which any one can check against the formula in linear time. A solver that answers “unsatisfiable” owes a proof, and the proofs that matter in practice are logs of learned clauses.

Definition 5.10 (RUP). Let F be a clause set. A clause C has the *reverse unit propagation* property with respect to F , written $F \vdash_1 C$, if unit propagation starting from the trail consisting of the complements $\bar{l}$ for $l \in C$, as propagated literals, refutes the clause set F .

Proposition 5.11 (RUP is sound). *If $F \vdash_1 C$ then $F \models C$.*

Proof. Suppose $v \models F$ and $v \not\models C$, so v makes every $\bar{l}$ with $l \in C$ true, and v satisfies every initial trail literal. Unit propagation adds a literal l only from a clause $C' \cup \{l\} \in F$ whose other literals are false in the current trail. By induction on the length of propagation, v satisfies every literal of the trail: if v satisfies all literals in the trail, then $v \models \bar{c}$ for each $c \in C'$, and since $v \models C' \cup \{l\}$ we get $v \models l$. So v satisfies the trail. A refutation produces a clause of F all of whose literals are false under the trail, hence under v , contradicting $v \models F$. ■

Definition 5.12 (RUP refutation, checker). A *RUP refutation* of F is a sequence of clauses $C_1, \dots, C_n$ with $C_n = \square$ such that $F \cup \{C_1, \dots, C_{i-1}\} \vdash_1 C_i$ for each i . The relation $\text{Chk}_{\text{RUP}}(d, F)$ holds when d is a RUP refutation of F ; it is decidable in polynomial time, by running unit propagation for each line.

Theorem 5.13 (RUP is a complete proof system). *The system with Φ the finite clause sets, D the finite sequences of clauses, Chk_{RUP} as above, and Val the unsatisfiable clause sets is sound and complete.*

Proof. Soundness follows from Proposition 5.11 by induction on i : every C_i is entailed by F , so $\square$ is entailed, so F is unsatisfiable.

For completeness let F be unsatisfiable over atoms $p_1, \dots, p_n$. For $0 \leq k \leq n$ call a clause k -full if it contains, for each $1 \leq j \leq k$, exactly one of $p_j, \neg p_j$, and no other literals. The n -full clauses each exclude exactly one total assignment. Each n -full clause C is RUP with respect to F : the trail $\bar{C}$ is a total assignment, and since F is unsatisfiable it falsifies a clause of F , so propagation refutes immediately. List all 2^n n -full clauses. Having listed all $(k+1)$ -full clauses, each k -full clause C is RUP with respect to them: from the trail $\bar{C}$, the clause $C \cup \{p_{k+1}\}$ is unit and propagates p_{k+1} , after which the clause $C \cup \{\neg p_{k+1}\}$ is falsified. List the k -full clauses next. Continuing down to $k = 0$ lists $\square$. This is a RUP refutation. ■

The proof is, of course, exponentially long in n . Its role is to show that RUP is not weaker than resolution in provability, while the actual strength in terms of proof size is that of the clause sequences produced by solvers, which are polynomial in the number of learned clauses (each clause learned by the analysis of Definition 5.5 is RUP; see Exercise 5.2).

Certificates in practice permit one further kind of line, whose soundness is subtler and instructive.

Definition 5.14 (RAT). Let F be a clause set, C a clause and $l \in C$ a literal. C has the *resolution asymmetric tautology* property with respect to F on l if for every $D \in F$ with $\bar{l} \in D$, the resolvent $(C \setminus \{l\}) \cup (D \setminus \{\bar{l}\})$ satisfies $F \vdash_1 (\cdot)$ (or is a tautology).

Proposition 5.15 (RAT preserves satisfiability, not entailment). *If C is RAT with respect to F on l , and F is satisfiable, then $F \cup \{C\}$ is satisfiable. In general $F \not\models C$.*

Proof. Let $v \models F$. If $v \models C$ we are done. Otherwise let v' agree with v except $v'(l) = 1$, so $v' \models C$. Let $D \in F$. If $\bar{l} \notin D$, then $v' \models D$, since $v \models D$ and only the value of l changed in the direction that cannot falsify a clause not containing $\bar{l}$. If $\bar{l} \in D$, the resolvent $R = (C \setminus \{l\}) \cup (D \setminus \{\bar{l}\})$ is entailed by F (it is RUP, hence entailed by Proposition 5.11, or it is a tautology), so $v \models R$. Since $v \not\models C$, we have $v \not\models C \setminus \{l\}$, hence $v \models D \setminus \{\bar{l}\}$, and the same literals are true under v' because v' differs from v only on the atom of l , which does not occur in $D \setminus \{\bar{l}\}$ unless the clause is tautological. So $v' \models D$. Thus $v' \models F \cup \{C\}$.

For the failure of entailment take $F = \{\{q\}\}$ and $C = \{p\}$ with $l = p$: no clause of F contains $\neg p$, so C is RAT on p , and $F \not\models \{p\}$. ■

A *DRAT proof* is a sequence of steps each of which adds a clause that is RUP or RAT with respect to the current database, or deletes a clause from the current database, and ends with an addition of $\square$.

Theorem 5.16 (DRAT checking is sound). *If d is a DRAT proof for F_0 , then F_0 is unsatisfiable.*

Proof. Let $DB_0 = F_0$ and DB_{i+1} the database after step $i + 1$. We claim DB_i is satisfiable whenever F_0 is. By induction: an RUP addition C satisfies $DB_i \models C$ by Proposition 5.11, so $DB_i \cup \{C\}$ has the same models as DB_i . An RAT addition preserves satisfiability by Proposition 5.15. A deletion gives a subset, which is satisfiable if DB_i is. If F_0 is satisfiable then the final database is satisfiable; but it contains $\square$, which no assignment satisfies. Hence F_0 is unsatisfiable. ■

Theorem 5.16 deserves emphasis because it shows how little a certificate need certify. The intermediate lines of a DRAT proof need not be consequences of F_0 , and a deletion step may weaken what later steps are checked against. What the checker certifies is a single

fact, the unsatisfiability of F_0 , and the proof's internal story about why is not a part of the certified content.

5.5 Elimination, and what a reduced formula forgets

Chapter 3 introduced elimination of an atom p by Davis and Putnam, which replaces N by N_p . Practical solvers use it as bounded variable elimination during preprocessing, and the transformation is sound for satisfiability by Lemma 3.3. It is not an equivalence-preserving transformation: $N \models N_p$, but $N_p \not\models N$ in general, and models of N_p are not models of N until a value for p is supplied.

Proposition 5.17 (Model reconstruction). *Let $N = E \cup P \cup Q$ be as in Lemma 3.3, with $P^- = \{C : C \cup \{p\} \in P\}$. If $v \models N_p$, define v' to agree with v off p and put $v'(p) = 1$ if v falsifies some member of P^- , and $v'(p) = 0$ otherwise. Then $v' \models N$.*

Proof. This is the pair of cases in the proof of Lemma 3.3. ■

The reconstruction rule reads the *eliminated clauses* P (and Q is not needed). It is a function of v and of P , so a task that asks for a model of the original formula N is not determined by the reduced formula alone. A single example makes this precise.

Example 5.18 (Same reduced formula, opposite extensions). Let $N_1 = \{\{p, a\}, \{\neg p, b\}\}$ and $N_2 = \{\{p, b\}, \{\neg p, a\}\}$. Eliminating p gives, in both cases, $(N_i)_p = \{\{a, b\}\}$. The valuation v with $v(a) = 1$, $v(b) = 0$ satisfies $\{a, b\}$. To extend v to a model of N_1 we need $\neg p \vee b$ true with b false, so $p = 0$. To extend it to a model of N_2 we need $p \vee b$ true with b false, so $p = 1$. The same reduced formula and the same model of it demand opposite values of p .

Example 5.18 instantiates Proposition 1.12 exactly. Consider the task “produce a model of the original formula”, and let the *fibre* over a reduced formula consist of all originals that reduce to it. The task is not constant on the fibre, so it does not factor through the reduction. Practical solvers respond by retaining a *reconstruction stack*, the eliminated clauses in order, which is precisely the residue that Proposition 5.17 requires. Decision (satisfiable or not) factors through the reduction; model production does not; and the stack is the minimal supplement that restores it.

5.6 The residue of a solver run

The pieces assemble into a description of what a SAT result consists of, and of what it discards.

- A *satisfiable* verdict is certified by a total assignment, which is checkable in linear time and carries no information about the search.
- An *unsatisfiable* verdict is certified by a DRAT proof, whose lines are not necessarily consequences of the input (Theorem 5.16). The proof certifies the verdict and nothing about the decisions, heuristic scores, restarts, or clause deletions of the run that produced it, nor about preprocessing steps that were later undone by reconstruction.
- Different runs, with different decision orders, yield different proofs of the same verdict. The verdict is the common output of a large fibre of runs (Definition 1.11).

This is not a flaw. The checker's job is served exactly. The next chapters extend the pattern to theories (Chapter 6) and to kernels (Chapter 7), where the discarded information starts to include things that a person may need.

Exercise 5.1. Show that unit propagation is confluent in the sense that if two maximal sequences of unit propagation steps from the same trail and the same clause set both end without a falsified clause then they end in the same set of trail literals, and that if one reaches a falsified clause then every maximal sequence does.

Exercise 5.2. Prove that every clause C_i produced by Definition 5.5 is RUP with respect to the clause set F at the conflict. (Hint: starting from the complement of C_i , propagate the literals that were resolved away, in the reverse of the order in which they were resolved.)

Exercise 5.3. Give a DRAT proof, with at least one RAT step that is not RUP, of the unsatisfiability of $\{\{p\}, \{\neg p\}\}$. Explain why the RAT step is not entailed by the formula, and why this does not endanger Theorem 5.16.

Chapter 6

Theories, Congruence, and Combination

Propositional search proves facts about Boolean structure. Most of the questions asked of automated provers in verification, planning, and scientific computing involve arithmetic, equality, arrays, and data types, and the solvers that answer them are propositional solvers wrapped around decision procedures for theories. The wrapping is logically simple, since it amounts to adding *theory lemmas* to a resolution proof, but its consequences for what a certificate means are substantial. A certificate now has two parts, a Boolean skeleton and a collection of theory facts, each checked by a different mechanism, and the correctness of the whole depends on the agreement of each mechanism with the intended meaning of the theory. This chapter proves correctness for the three components that matter for certificates: the lazy architecture, congruence closure for equality, and Farkas certificates for linear arithmetic; it states the combination theorem of Nelson and Oppen and proves the half of it that explains its hypothesis.

6.1 Theories and the lazy architecture

Definition 6.1 (Theory, T-validity). A *theory* T is a class of structures over a signature Σ (its *models*), closed under isomorphism. A formula is *T-valid*, written $\models_T \varphi$, if every model of T satisfies it under every assignment, and *T-satisfiable* if some model of T and some assignment satisfy it. A *T-lemma* is a T -valid clause of ground literals.

A quantifier-free formula in conjunctive normal form over Σ -atoms is a set of clauses whose atoms are Σ -literals (equalities, inequalities, applications of predicate symbols). The *Boolean abstraction* replaces each distinct atom by a propositional variable. The lazy architecture couples the CDCL system of Chapter 5 to a procedure that decides T -satisfiability of conjunctions of literals.

Definition 6.2 (DPLL(T)). The rules are those of Definition 5.1 with one addition, in which entailment is replaced by T -entailment, and with the abstraction understood:

TheoryLearn $\langle M \parallel F \rangle \Rightarrow \langle M \parallel F \cup \{C\} \rangle$ if C is a T -lemma over atoms occurring in F or M .

Backjump as before, with the side condition $F \models_T C'$ in place of $F \models C'$.

The theory solver communicates with the Boolean engine only through TheoryLearn: when the literals on the trail are jointly T -unsatisfiable, it returns a clause $\neg \ell_1 \vee \dots \vee \neg \ell_k$ built from an unsatisfiable subset $\{\ell_1, \dots, \ell_k\}$ of the trail, which is a T -lemma, and the next Conflict analysis proceeds as in the propositional case.

Theorem 6.3 (Soundness of DPLL(T)). *If a run from $\langle \varepsilon \parallel F_0 \rangle$ reaches Fail then F_0 is T -unsatisfiable. Moreover $F_0 \cup L$ has a resolution refutation, where L is the set of clauses added by TheoryLearn, and each member of L is T -valid.*

Proof. The proof of Lemma 5.2 goes through with “ F and F_0 have the same models” replaced by “ F and F_0 have the same T -models” and with $\models_T$ in place of $\models$ throughout. TheoryLearn adds a T -lemma, which every T -model satisfies, so it preserves T -models. The remaining cases are verbatim. At Fail the argument of Theorem 5.3 shows that $F_0 \models_T \bar{c}$ for every $c \in C$ and $F_0 \models_T C$, which no T -model can satisfy. The second claim follows from Theorem 5.7, regarding each member of L as an additional input clause: the analysis uses only clauses of the current F , which consists of F_0 , members of L , and clauses learned by resolution. ■

Theorem 6.3 gives the shape of a certificate for a theory-unsatisfiable result: a resolution (or RUP) refutation from $F_0 \cup L$, together with a proof for each member of L that it is T -valid. The first part is checked as in Chapter 5. The second part is where theory-specific certificate formats enter.

6.2 Equality: congruence closure

The theory EUF of equality with uninterpreted functions has as models all structures for a signature of function symbols (constants being 0-ary symbols). A *ground term* is a term without variables, where free variables are treated as constants. Let S be a finite set of ground terms closed under subterms and $E \subseteq S \times S$ a set of equations.

Definition 6.4 (Congruence closure). The *congruence closure* of E on S is the smallest equivalence relation $\equiv$ on S such that $E \subseteq \equiv$ and, whenever $f(s_1, \dots, s_n), f(t_1, \dots, t_n) \in S$ with $s_i \equiv t_i$ for all i , we have $f(s_1, \dots, s_n) \equiv f(t_1, \dots, t_n)$.

Theorem 6.5 (Correctness of congruence closure). *For $s, t \in S$: $E \models_{\text{EUF}} s \approx t$ if and only if $s \equiv t$. Consequently a conjunction $E \wedge \bigwedge_j s_j \not\approx t_j$ with all terms in S is EUF-unsatisfiable if and only if $s_j \equiv t_j$ for some j .*

Proof. ($\Leftarrow$) We show every pair of $\equiv$ is entailed by E by induction on the generation of $\equiv$ as a closure: the pairs of E are entailed; reflexivity, symmetry and transitivity of $=$ are sound; and congruence is sound, since in any structure equal arguments give equal values of f .

($\Rightarrow$) Suppose $s \not\equiv t$. Build a structure $\mathcal{A}$ with domain $S/\equiv$. For an n -ary symbol f and classes $c_1, \dots, c_n$: if there are representatives $t_i \in c_i$ with $f(t_1, \dots, t_n) \in S$, put $f^{\mathcal{A}}(c_1, \dots, c_n) = [f(t_1, \dots, t_n)]$; otherwise put $f^{\mathcal{A}}(c_1, \dots, c_n) = c_0$ for a fixed class c_0 . The first clause is well defined: if $t_i, t'_i \in c_i$ and both $f(\vec{t}), f(\vec{t}') \in S$ then $t_i \equiv t'_i$, so $f(\vec{t}) \equiv f(\vec{t}')$ by the congruence clause. By induction on terms in S , which is subterm-closed, $\llbracket u \rrbracket^{\mathcal{A}} = [u]$ for every $u \in S$. Hence $\mathcal{A}$ satisfies every equation of E (as its sides are $\equiv$) and falsifies $s \approx t$ because $[s] \neq [t]$. So $E \not\models s \approx t$.

For the consequence: if some $s_j \equiv t_j$ then $E \models s_j \approx t_j$ contradicts the disequality, so the conjunction is unsatisfiable. If no $s_j \equiv t_j$ then $\mathcal{A}$ satisfies E and every disequality. ■

Proposition 6.6 (The merging algorithm). *Start from the identity relation on S . Repeat: merge the classes of s, t for some $(s \approx t) \in E$ with $s \not\equiv t$, or for some $f(\vec{s}), f(\vec{t}) \in S$ with $s_i \equiv t_i$ for all i and $f(\vec{s}) \not\equiv f(\vec{t})$. This halts after at most $|S| - 1$ merges, and the final relation is the congruence closure.*

Proof. Each merge reduces the number of classes by one, so at most $|S| - 1$ merges occur. At the end no merge is applicable, so the relation contains E and is closed under congruence; hence it contains the congruence closure. Conversely, by induction on the number of merges performed, each merged pair belongs to the congruence closure, since it follows from E and congruence from pairs already in the closure. So the final relation equals it. ■

With union-find and a signature table the algorithm runs in time $O(n \log n)$ (cited: Downey, Sethi, Tarjan 1980). For certificates the important feature is that the algorithm can record *why* each merge occurred, either because of an input equation or because of congruence on earlier merges. The record is a derivation in the equational calculus with rules for hypotheses, reflexivity, symmetry, transitivity, and congruence, and the checking relation for such derivations is a simple tree check.

Proposition 6.7 (Equality lemmas have checkable explanations). *If a congruence closure computation derives $s_j \equiv t_j$ then it yields a subset $E' \subseteq E$ and a derivation of $s_j \approx t_j$ from E' in the equational calculus, and the clause $\bigvee_{e \in E'} \neg e \vee s_j \approx t_j$ is an EUF-lemma.*

Proof. Associate with each merge the set of input equations used in its justification, which is finite and defined by recursion on the order of merges. The derivation is assembled from the recorded justifications, and soundness of the calculus rules gives $E' \models s_j \approx t_j$, so the clause is valid. ■

Example 6.8 (Explanations are not unique). Let $E = \{a \approx b, b \approx c, a \approx c\}$. Both $E' = \{a \approx c\}$ and $E'' = \{a \approx b, b \approx c\}$ explain $a \approx c$. A run that processes the third equation first records E' , and one that processes the first two records E'' . Both are valid lemmas. They yield different clauses in the Boolean engine and so different subsequent behavior, with the same verdict.

6.3 Linear arithmetic: Farkas certificates

For linear arithmetic over the rationals the natural lemma certificate is a nonnegative combination of the inequalities that contradicts itself. Let $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$.

Proposition 6.9 (Farkas certificates are sound). *If $y \in \mathbb{Q}^m$ satisfies $y \geq 0$, $y^\top A = 0$ and $y^\top b < 0$, then the system $Ax \leq b$ has no solution $x \in \mathbb{Q}^n$.*

Proof. If $Ax \leq b$ and $y \geq 0$ then $y^\top Ax \leq y^\top b$. But $y^\top Ax = 0$ and $y^\top b < 0$, giving $0 < 0$. ■

Theorem 6.10 (Farkas' lemma). [CITED: FARKAS 1902] *The system $Ax \leq b$ over $\mathbb{Q}$ is unsatisfiable if and only if there is $y \geq 0$ with $y^\top A = 0$ and $y^\top b < 0$.*

Together these make the Farkas vector y a complete certificate for the linear-rational theory LRA (non-strict case), checkable in exact rational arithmetic by one matrix-vector multiplication. The checker is tiny and the certificate says nothing about *how* it was found: simplex pivots, bound propagation, and branching on disequalities are all absent from it. For linear integer arithmetic no certificate with comparable simplicity is complete in the same way, because deciding integer feasibility is NP-hard and the certificates involve cuts and case splits, whose format we do not develop here.

Remark 6.11 (Exactness). A certificate y is a rational vector and the check is an equation in $\mathbb{Q}$. A solver that finds y in floating point is free to do so; the check, however, must be exact. A claim certified with floating-point arithmetic is a different claim, about a different structure. This is the first appearance of the translation of Section 1.3 of Chapter 1 in the context of theories, where the choice of the structure (rationals, integers with unbounded range, machine integers with overflow, floating-point numbers) is the content of τ .

6.4 Combining theories

Real problems mix theories. The classical method of Nelson and Oppen combines decision procedures for theories T_1, T_2 over disjoint signatures, first purifying a formula so that each literal belongs to one signature (introducing fresh variables for foreign subterms), and then exchanging equalities among the shared variables.

Definition 6.12 (Stably infinite, arrangement). A theory T is *stably infinite* if every T -satisfiable quantifier-free formula is satisfiable in an infinite model of T . Let V be a finite set of variables. An *arrangement* of V is an equivalence relation on V ; write $\delta_\sim$ for the conjunction of the equalities $x = y$ for $x \sim y$ and the disequalities $x \neq y$ for $x \not\sim y$.

Theorem 6.13 (Nelson–Oppen). [CITED: NELSON AND OPPEN 1979; TINELLI AND HARANDI 1996] Let T_1, T_2 be stably infinite theories over disjoint signatures, φ_1, φ_2 conjunctions of literals over Σ_1 and Σ_2 respectively, and V the set of variables occurring in both. Then $\varphi_1 \wedge \varphi_2$ is satisfiable in $T_1 \cup T_2$ if and only if there is an arrangement $\sim$ of V such that $\varphi_i \wedge \delta_\sim$ is T_i -satisfiable for $i = 1, 2$.

The direction $(\Rightarrow)$ holds without any hypothesis and has a one-line proof: a model of $T_1 \cup T_2$ satisfying $\varphi_1 \wedge \varphi_2$ determines the arrangement $x \sim y$ iff x, y have equal values, and each reduct is a model of T_i satisfying $\varphi_i \wedge \delta_\sim$. The direction $(\Leftarrow)$ is where stable infinity is used: given models $\mathcal{A}_1, \mathcal{A}_2$ satisfying the two parts, one uses the Löwenheim–Skolem theorem and stable infinity to make both of the same infinite cardinality, builds a bijection between their domains that respects the arrangement on V , and transports one structure along the bijection to form a model of the union. The next example shows the hypothesis cannot be dropped.

Example 6.14 (Stable infinity is necessary). Let T_1 over the empty signature (equality only) have as its models exactly the structures with exactly two elements. T_1 is not stably infinite. Let $T_2 = \text{EUF}$ over a signature $\{f\}$. Take $\varphi_2 = f(x) \neq f(y) \wedge f(y) \neq f(z) \wedge f(x) \neq f(z)$ and φ_1 the empty conjunction. Then $V = \emptyset$ since φ_1 mentions no variables, so the unique arrangement is the empty relation. The formula φ_1 is T_1 -satisfiable and φ_2 is T_2 -satisfiable. Nevertheless $\varphi_1 \wedge \varphi_2$ is unsatisfiable in $T_1 \cup T_2$: in every model there are exactly two elements, while φ_2 requires three pairwise distinct values $f(x), f(y), f(z)$. The combination procedure would answer *satisfiable*, which is wrong, and the error is silent.

The example is a concrete instance of the discipline that runs through this book. A procedure's verdict is correct relative to hypotheses about its inputs (here, stable infinity), and a certificate that does not carry those hypotheses can be valid in the checker's sense and misleading in the user's.

6.5 Certificates for theory results and the growth of the trusted base

Combining the pieces: an SMT unsatisfiability result is certified by a resolution proof over the Boolean abstraction together with a certificate for each theory lemma (Theorem 6.3). The checker for the whole is the union of checkers: a clause-level proof checker, a derivation checker for EUF lemmas (Proposition 6.7), an exact-arithmetic checker for LRA (Proposition 6.9), and so on for each theory.

Theorem 6.15 (Soundness of the combined checker). *Let Chk_{res} be a sound checker for resolution refutations, and for each theory T_i let Chk_{T_i} be a sound checker for T_i -lemmas, in the sense that $\text{Chk}_{T_i}(c, C)$ implies $\models_{T_i} C$. Define $\text{Chk}_{\text{SMT}}((\vec{c}, d), F)$ to hold if d is a resolution refutation of $F \cup \{C_1, \dots, C_k\}$ and $\text{Chk}_{T_{i_j}}(c_j, C_j)$ for each j . Suppose in addition that every clause valid in some T_i is valid in the combined theory T (the models of T are structures whose reducts to each component signature are models of that component). Then $\text{Chk}_{\text{SMT}}((\vec{c}, d), F)$ implies that F is unsatisfiable in T .*

Proof. A model of T and an assignment satisfying F would satisfy each C_j , which is T -valid by the additional hypothesis and the soundness of $\text{Chk}_{T_{i_j}}$. It would therefore satisfy all of $F \cup \{C_1, \dots, C_k\}$, contradicting the soundness of the resolution refutation. ■

The additional hypothesis is not automatic. It is exactly what fails if a component is stated for a different structure than the one the combined problem intends. The trusted base therefore grows with each theory, and in two ways: the checker for each theory must be trusted, and the *intended structure* of each theory must be correctly described. The second is the translation of Proposition 1.7 again.

Remark 6.16 (Quantifiers and the third outcome). Solvers also accept quantified formulas, and treat quantifiers by instantiation. A refutation found this way is certified by the instances used, which are ground and so fall under the preceding theorem. A failure to find a refutation is not a model: it supports only the answer *unknown*. The three-valued outcome of Definition 2.14 appears here for the same reason as in first-order logic, and the discipline of non-certification forbids reading unknown as satisfiable.

6.6 What an SMT result discards

The structure of the residue is now visible in miniature. A result is a Boolean skeleton plus lemmas, and discarded are: the order of theory conflicts; which of several valid explanations was chosen (Example 6.8); how a Farkas vector was found; the arrangement guesses made during combination; and, for quantified input, which instantiations were tried and abandoned. None of this affects the soundness of the verdict. A task that depends on any of it, for instance diagnosing why a verification condition failed to prove, or porting a proof

to a slightly different theory, cannot be served by the certificate alone. Chapter 7 turns to the case where the certificate is checked by a small kernel and the same questions arise at the level of tactics and proof terms.

Exercise 6.1. Compute the congruence closure for $E = \{f(f(f(a))) \approx a, f(f(f(f(f(a)))) \approx a\}$ and show that $f(a) \equiv a$. Give the derivation of $f(a) \approx a$ in the equational calculus and the set E' of Proposition 6.7.

Exercise 6.2. Show that Theorem 6.5 fails if S is not subterm-closed, by giving $E, S, s, t \in S$ with $E \models_{\text{EUF}} s \approx t$ but $s \not\equiv t$ for the congruence closure computed on S .

Exercise 6.3. Verify Proposition 6.9 on the system $x \leq 0, -x \leq -1$ by exhibiting y .

Chapter 7

Kernels, Tactics, and the Trusted Base

The previous chapters concerned procedures whose outputs are certificates for an independent checker. This chapter concerns the architecture in which the checker is built into the interface: a small *kernel* guards the only way to construct a theorem, and arbitrary programs, called *tactics*, may drive the kernel as they please. The architecture, due to Milner and his collaborators in the LCF system, is the reason the soundness of a large body of machine-checked mathematics reduces to the correctness of a few thousand lines of code. This chapter states that reduction precisely, extends it to dependent type theory with proof irrelevance, and examines three things it leaves open: whether the theorem proved is the one intended, which axioms the proof actually uses, and what the kernel's identification of proofs forgets.

7.1 The LCF discipline

Definition 7.1 (Kernel). A *kernel* for a set Φ of statements with semantic validity $\text{Val} \subseteq \Phi$ consists of an abstract type Thm , a function $\text{concl} : \text{Thm} \rightarrow \Phi$, and a finite set of *rules*. Each rule is a partial function $r : \text{Thm}^k \times \text{Args} \rightarrow \text{Thm}$. The only way to obtain a value of type Thm is to apply a rule to arguments (the *abstraction barrier*). The kernel is *locally sound* if for every rule r and arguments $\theta_1, \dots, \theta_k, a$ with $\text{concl}(\theta_i) \in \text{Val}$ for all i and $r(\vec{\theta}, a) = \theta$ defined, we have $\text{concl}(\theta) \in \text{Val}$.

The abstraction barrier is not a mathematical notion; it is enforced by the type system of the implementation language, by module opacity in ML or by the checker of the proof assistant in more modern systems. Granting it, the following holds for arbitrary client programs.

Theorem 7.2 (Soundness for arbitrary clients). *Let K be a locally sound kernel and P any program, with any control flow, any use of heuristics or randomness, and possibly nonterminating, that interacts with K only through its interface. If P evaluates to a value $\theta : \text{Thm}$ in some execution, then $\text{concl}(\theta) \in \text{Val}$.*

Proof. Every value of type Thm that exists in an execution was created by some rule application, since the abstraction barrier forbids any other construction. By induction on the number of rule applications that precede the creation of a given value, each existing value has a valid conclusion: the arguments to a rule application are earlier values, hence valid

by the induction hypothesis (or are non-Thm data, which are unconstrained), and local soundness gives validity of the result. ■

Theorem 7.2 is Theorem 1.5 of Chapter 1 in a different dress. The producer is a program and the checker is distributed into the constructors of Thm, but the structure is the same: assurance depends on the kernel and on nothing else. A program that fails to produce a Thm value says nothing.

Definition 7.3 (Tactic). A *tactic* is a function t from goals $g \in \Phi$ to pairs $t(g) = (\vec{g}, v)$ where $\vec{g} = (g_1, \dots, g_k)$ is a list of *subgoals* and $v : \text{Thm}^k \rightarrow \text{Thm}$ is a *validation*. The tactic is *valid* if for every goal g , with $t(g) = (\vec{g}, v)$, and every $\vec{\theta}$ with $\text{concl}(\theta_i) = g_i$, the value $v(\vec{\theta})$ is defined and $\text{concl}(v(\vec{\theta})) = g$.

A *tactic proof* of g with a family of tactics resolves the subgoals recursively: $\text{solve}(g) = v(\text{solve}(g_1), \dots, \text{solve}(g_k))$ for $t(g) = (\vec{g}, v)$ and a tactic t chosen for each goal; a goal for which the chosen tactic returns the empty list of subgoals is closed by $v()$.

Proposition 7.4 (Valid tactics prove what they say). *If every tactic used is valid and $\text{solve}(g)$ terminates, then $\text{concl}(\text{solve}(g)) = g$. If some tactic is not valid, then $\text{solve}(g)$, if it terminates, still has a valid conclusion by Theorem 7.2, but $\text{concl}(\text{solve}(g))$ may differ from g .*

Proof. The first claim is by induction on the recursion depth of solve : each $\text{solve}(g_i)$ has conclusion g_i by hypothesis, so validity of the tactic at g gives $\text{concl}(v(\dots)) = g$. The second is Theorem 7.2. For a tactic that is not valid, take $t(g) = ((), v)$ with $v()$ a theorem whose conclusion is some valid $h \neq g$. ■

The proposition separates two guarantees that are often conflated. A bug in a tactic cannot produce an *invalid theorem*, but can produce a *valid theorem about the wrong statement*. The defense in practice is that the client states g and compares it with $\text{concl}(\theta)$ at the end, or in a dependently typed system that the kernel checks a proof term against a stated type; either way the agreement of the statement is checked, and that check is a separate component of the trusted base (Section 7.5).

7.2 Dependent type theory as kernel

Modern proof assistants of the Lean family replace the abstract type Thm by the typing judgement of a dependent type theory, with the Curry–Howard reading in which a proposition φ is a type and a proof is a term of that type. The proof system of Definition 1.1 is then:

- Φ is the set of types in a given environment of declarations,
- D is the set of terms,
- $\text{Chk}(d, \varphi)$ holds when the kernel’s type checking derives $d : \varphi$ in the environment, and
- Val is truth in an intended interpretation of the type theory.

Soundness is the consistency and correctness of the type theory with respect to a model; completeness fails by Gödel’s incompleteness theorem for the theories at issue and is not expected. The decidability of Chk is a hypothesis of Definition 1.1 that holds for strongly normalizing theories. The metatheory of the type theories implemented by real kernels,

which add features such as proof irrelevance, quotients, and primitive literals, is subtler than the textbook case, and the question of how closely the implemented checker matches the idealized rules has been studied in detail; we treat decidability and soundness of the kernel as hypotheses and do not rederive them (see Carneiro 2024 for a careful analysis of the Lean kernel).

Three features of the Lean kernel matter for this book.

- (i) *Proof irrelevance.* Any two proofs of the same proposition (a term of a type living in the universe `Prop`) are definitionally equal.
- (ii) *Axioms are declarations.* Besides definitions and theorems, an environment may contain axioms: constants with a type and no value. Lean’s standard library uses three, `propext`, `Classical.choice` and `Quot.sound`, and the *elaborator*, a trusted-for-convenience but untrusted-for-soundness component, can add a distinguished axiom `sorryAx` as a placeholder for incomplete work.
- (iii) *Elaboration is separate from checking.* Tactics and the elaborator build terms; the kernel checks the final terms. By the argument of Theorem 7.2, bugs in the elaborator cannot produce an invalid theorem. They can produce a term whose type is not the one the author intended.

7.3 Proof irrelevance as a collapse

Let $\varphi : \text{Prop}$ and let $\text{Proof}(\varphi)$ be the set of closed terms of type φ in some environment.

Proposition 7.5 (No term distinguishes proofs). *Let $\pi_1, \pi_2 \in \text{Proof}(\varphi)$. Then $\pi_1 \equiv \pi_2$ by definitional equality. Consequently, for every term f of function type $(h : \varphi) \rightarrow B$ in the kernel, $f \pi_1 \equiv f \pi_2$, and in particular no definable function from proofs of φ to a data type separates π_1 from π_2 .*

Proof. The first claim is the proof irrelevance rule. The second follows because definitional equality is a congruence: the application $f \pi_i$ is built from π_i , and $\equiv$ is closed under application. Two definitionally equal terms of a data type are indistinguishable by any term, since any observation is again a term and congruence applies. ■

The proposition states, as a theorem, the way in which the kernel is a quotient. All proofs of a proposition are mapped to a single point, and the structure internal to a proof, namely which lemmas it used, which case split it chose, and how long it is, is invisible *from inside the system*. This is a deliberate and valuable design choice. It makes checking cheaper, since equality of proofs need never be tested, and it keeps data from depending on proofs. In the vocabulary of Chapter 1, the kernel realizes the factorization criterion (Proposition 1.12) as an architectural principle: the only tasks permitted on proofs are those that factor through the single class.

The proof term itself, however, is an ordinary data structure outside the kernel, available to meta-level programs, and these can read what the kernel refuses to distinguish. This is the first appearance of a fact that organizes Part III: the residue is not destroyed by the collapse, it is relocated, and whether it is retained depends on whether anyone keeps the term.

7.4 Axiom footprints

A proof term mentions constants, each constant has a type and possibly a value mentioning further constants. The *dependency closure* $\text{cl}(c)$ of a constant c is the smallest set containing c and every constant occurring in the type or value of a member of $\text{cl}(c)$. The *axiom footprint* $\text{Ax}(c)$ is the set of axiom constants in $\text{cl}(c)$.

Proposition 7.6 (Locality of checking). *Let Γ be a well-formed environment and $c : \varphi$ a declaration in it. The sub-environment Γ_c consisting of the members of $\text{cl}(c)$, in their original order, is well-formed and $c : \varphi$ is derivable in it. Consequently φ is derivable in the pure type theory extended by exactly the axioms of $\text{Ax}(c)$ and the definitions in $\text{cl}(c)$.*

Proof. Inspection of the typing and reduction rules shows that checking a declaration consults only the constants occurring in it and those whose definitions are unfolded during conversion; unfolding a constant reveals its value, whose constants lie in the closure by definition. Hence the check of each declaration in $\text{cl}(c)$ succeeds in Γ_c exactly as in Γ , by induction along the declaration order. This is a property of the kernel’s design and is assumed here. ■

The proposition says that the footprint is the *logical cost* of a theorem: it identifies the additional principles that were assumed in order to establish c . Two warnings follow from earlier chapters.

Proposition 7.7 (Footprint is not a function of the statement or the definitional class). *There are two proofs π_1, π_2 of the same proposition φ , definitionally equal by proof irrelevance, with $\text{Ax}(\pi_1) \neq \text{Ax}(\pi_2)$.*

Proof. Let φ be the proposition $2 + 2 = 4$ over the natural numbers. The term $\pi_1 = \text{rfl}$ depends on no axioms. Let π_2 be the term obtained by proving φ via excluded middle, case analysis on $\varphi \vee \neg\varphi$ using `Classical.em`; its closure contains `Classical.choice`. Both have type $\varphi : \text{Prop}$, so $\pi_1 \equiv \pi_2$ by Proposition 7.5. Their footprints differ. ■

The proposition has a consequence that is easy to miss. Since the footprint is a property of the *term* and not of the definitional class, a statement of the form “the theorem is choice-free” is a claim about a particular term and must be established by inspecting that term, for instance by a footprint audit of the final environment. It cannot be read off from the statement of the theorem, and it is not preserved if the kernel replaces a proof by a definitionally equal one.

Proposition 7.8 (The placeholder axiom is inconsistent). *The constant `sorryAx` has a type of the form $\forall(\alpha : \text{Sort } u), \alpha$ (with an additional Boolean flag distinguishing synthetic placeholders). Hence every proposition φ has a term `sorryAx  $\varphi$` of type φ whose footprint contains `sorryAx`. A theorem whose footprint contains `sorryAx` therefore certifies nothing beyond the Boolean fact that the file compiled with warnings.*

Proof. Instantiate the type at $\alpha = \varphi$. ■

The elaborator inserts `sorryAx` not only for an explicit placeholder but also as error recovery when elaboration of a declaration fails, so that later declarations can be elaborated against it. The practical consequence, which was found in the experiments underlying

the verification-inheritance papers of the author’s corpus, is that the presence of a stored value for a declaration is not evidence of successful verification: the check that matters is the footprint, computed on the final environment, which must exclude `sorryAx`.

Example 7.9 (An axiom that encodes the compiler). The decision procedures `decide` and `native_decide` both establish decidable propositions by evaluation. The first has the kernel itself reduce a decision procedure to true, so its footprint is the footprint of the procedure. The second evaluates the procedure with the compiled evaluator and records the result through an axiom, `Lean.ofReduceBool`, whose soundness is the correctness of the compiler and its runtime. Its footprint therefore contains an axiom with no model-theoretic content, whose justification is the correctness of a code generator. Both procedures produce a term of the same type; they differ in what they ask the reader to trust.

7.5 The trusted base of a verified statement

Gathering the components identified in this and the preceding chapters, the assurance attached to a declaration $c : \varphi$ depends on the following list, and on nothing else.

Definition 7.10 (Trusted base). The *trusted base* of a verified result $c : \varphi$ consists of

- (T1) the kernel implementation and the soundness of its type theory with respect to the intended interpretation;
- (T2) the axioms in $\text{Ax}(c)$, each read at its intended meaning (and including any axiom, such as `Lean.ofReduceBool`, that encodes a trusted program);
- (T3) the definitions in $\text{cl}(c)$ that appear in φ , as faithful formalizations of the notions they name; and
- (T4) the translation τ from the intended claim ψ to φ (Definition 1.6).

Items (T1) and (T2) are the traditional focus of discussions of the trusted base and are the part addressed by re-checkers and footprint audits. Items (T3) and (T4) are specifications, and no kernel checks them. A theorem about a quantity named `jerk` in a formalized mechanics is a theorem about the formal definition. Whether the definition captures the physical or mathematical notion is an extra-kernel obligation, which is the subject of Chapter 9.

Proposition 7.11 (Assurance decomposition). *Suppose (T1) holds, the axioms in $\text{Ax}(c)$ are true in the intended interpretation, the definitions in $\text{cl}(c)$ interpret as intended, and τ is adequate. Then the intended claim ψ is true. If any of these hypotheses fails, no conclusion about ψ follows from $c : \varphi$.*

Proof. Soundness of the kernel gives $\varphi \in \text{Val}$ relative to the interpretation, which by hypothesis validates the axioms and definitions. Adequacy of τ gives ψ by Proposition 1.7. The last sentence records that nothing in the premises substitutes for a failed hypothesis. Example 6.14 shows a verdict going wrong when a hypothesis of a procedure fails silently, and Example 7.9 shows the trusted base growing by an axiom that encodes a program. ■

7.6 Scripts and terms

A tactic script is a history, a sequence of commands that, run in the elaborator, produces a term. The elaboration map from scripts to terms is a many-to-one function, so scripts and terms are related exactly as search traces and derivations were in Chapter 1.

Proposition 7.12 (Scripts are not recoverable from terms). *Let Elab map scripts to the terms they produce. In general Elab is not injective: there are distinct scripts producing the same term, and a script's structure (the decomposition into lemmas, the order of independent subgoals, comments) is not determined by the term.*

Proof. For a goal $p \wedge q$ with hypotheses p, q , the scripts that close the two conjuncts in either order both produce the same term $\wedge I h_p h_q$, as in Example 1.10. Comments and layout do not affect the term at all. ■

Consequently the three layers of a machine-checked result, the script, the term, and the proposition proved, are linked by two many-to-one maps, and each map is a collapse in the sense that will be made precise in Part III. The term is the right object for the kernel, the script is the right object for a human who wants to understand, adapt, or repair the proof, and the proposition is the right object for the reader of the result. A representation fixed at one layer is sufficient for the tasks of that layer and, in general, for no others.

Exercise 7.1. Prove that the composition of two valid tactics, run in sequence on the subgoals of the first, is a valid tactic.

Exercise 7.2. Show that the conclusion of Proposition 7.5 fails in a type theory that has no proof irrelevance and in which disjunction may eliminate into data: for a proposition p with a proof h , define a function $p \vee p \rightarrow \text{Bool}$ that separates $\text{inl } h$ from $\text{inr } h$.

Exercise 7.3. Give an example of a proposition φ for which every proof has footprint containing `Classical.choice`, and explain why the footprint of such a proof is still not an invariant of φ in the sense of Proposition 7.7.

Chapter 8

Certificates and Checkers

Chapters 5 to 7 met certificates in three forms: logs of learned clauses, theory lemmas with their explanations, and terms checked by a kernel. This chapter abstracts what they share. A *certifying computation* returns an answer together with a witness that a simple, independent procedure can check; the witness is bound to a particular input; and the checker's correctness, not the producer's, carries the assurance. After the abstract definition, the chapter treats the case that is not a single call but a *pipeline* of transformations, where a certificate for the last stage must be connected to the original input; proves the asymmetry between certifying unsatisfiability and certifying satisfiability through a pipeline; and describes the third architecture for a checker, proof by reflection, in which the certificate is the input itself and all of the content lies in a verified computation.

8.1 Certifying computation

Definition 8.1 (Certificate scheme). Let $R \subseteq X \times Y$ be a relation between inputs and answers, to be read as “ y is a correct answer for x ”. A *certificate scheme* for R consists of a set W of *witnesses* and a decidable relation $\text{Wit} \subseteq X \times Y \times W$ such that

- (i) (*soundness*) $\text{Wit}(x, y, w)$ implies $R(x, y)$, and
- (ii) (*completeness*) $R(x, y)$ implies $\text{Wit}(x, y, w)$ for some w .

A *certifying algorithm* for R computes, from x , a pair (y, w) ; its output is *accepted* if $\text{Wit}(x, y, w)$.

Theorem 8.2 (Assurance rests on the checker). *If (x, y, w) is accepted, then $R(x, y)$, whatever algorithm computed (y, w) . If $R(x, y)$ then there exists some witness accepted, but no particular algorithm is guaranteed to find it.*

Proof. Soundness and completeness of the scheme, respectively. ■

This is Theorem 1.5 for a general relation. The examples below fix the scale of the checkers involved. In each, the witness is much simpler to check than the answer is to find.

Problem	Answer	Witness	Checker's work
Satisfiability	satisfiable unsatisfiable	total assignment DRAT proof (Ch. 5)	evaluate each clause unit propagation per line
Linear programming	optimal value v	primal and dual feasible points with equal objective	feasibility and equality of objective values (strong duality, cited)
Bipartiteness	bipartite not bipartite	2-colouring odd cycle	test each edge follow the cycle
Primality	prime	Pratt certificate (recursively certified factorization of $p - 1$ and a generator)	modular exponentiations
Maximum flow	flow of value f	flow and cut of equal value (max-flow min-cut, cited)	feasibility of both

In every row the witness certifies one answer and says nothing about the process that found it. For problems with both an affirmative and a negative certificate (the bipartiteness rows), the relation Wit is indexed by the answer.

8.2 Pipelines and the direction of preservation

Real solvers do not apply a checker to the input they are given. They transform the input (encode it, simplify it, eliminate variables, split it into cases), solve the transformed problem, and read the answer back. A certificate for the final problem must be connected to the original, and the connection is a property of each transformation.

Let X be a set of instances (clause sets, say) and $\text{Mod}(x)$ the set of models of an instance x .

Definition 8.3 (Reflecting and reconstructing transformations). A transformation $T : X \rightarrow X'$ is *unsat-reflecting* if $x' = T(x)$ unsatisfiable implies x unsatisfiable. It is *model-reconstructing* if there is a set $\text{Data}_T(x)$ of side data produced alongside $T(x)$ and a function ρ_T such that for every model m' of $T(x)$ and $s \in \text{Data}_T(x)$, $\rho_T(m', s)$ is a model of x .

Theorem 8.4 (Composition along a pipeline). Let $x_0 \xrightarrow{T_1} x_1 \rightarrow \dots \xrightarrow{T_n} x_n$ with $x_i = T_i(x_{i-1})$.

- (i) If every T_i is *unsat-reflecting* and x_n is unsatisfiable, then x_0 is unsatisfiable. A certificate for the unsatisfiability of x_n is thus a certificate for x_0 provided each reflection property holds.
- (ii) If every T_i is *model-reconstructing* and m_n is a model of x_n , then, given side data $s_i \in \text{Data}_{T_i}(x_{i-1})$ for $i = n, \dots, 1$, the model $m_0 = \rho_{T_1}(\dots \rho_{T_n}(m_n, s_n) \dots, s_1)$ is a model of x_0 .

Proof. (i) By downward induction: x_n unsatisfiable gives x_{n-1} unsatisfiable, and so on. (ii) By downward induction on i , each intermediate value is a model of x_{i-1} . ■

The two parts are asymmetric in what they require.

Proposition 8.5 (Unsatisfiability needs no data; satisfiability does). *In (i) no additional information about the pipeline is required beyond the reflection property of each stage, which is a statement about the stage and not about the run. In (ii) the side data s_i must be retained from the run. In general, a satisfying assignment for x_0 is not determined by a satisfying assignment for x_n .*

Proof. The first two sentences restate the theorem. For the last, apply Example 5.18 of Chapter 5: two inputs N_1, N_2 with the same reduced formula and the same model of it need opposite extensions to the eliminated variable, so no function of the reduced model alone yields a model of the original. ■

Example 8.6 (Where the reflection property comes from). Variable elimination (Lemma 3.3) is unsat-reflecting, because satisfiability of the original implies satisfiability of N_p (the resolvents are entailed). It is model-reconstructing with $\text{Data} = P$, the eliminated clauses (Proposition 5.17). Tseitin’s encoding of a Boolean circuit as a clause set is both unsat-reflecting and, with the identity on the original variables as ρ and no side data, model-reconstructing.

A transformation whose reflection property is merely *believed* enlarges the trusted base by the transformation’s implementation. The DRAT format removes preprocessors from the trusted base in the unsatisfiability direction by logging each preprocessing step as an RAT addition or a deletion (Theorem 5.16): the check of the log then re-establishes the reflecting property for those steps, rather than assuming it. The *encoding* from the user’s problem to clauses is typically not logged this way, and remains an item (T4) of the trusted base in the sense of Definition 7.10.

8.3 Proof by reflection

The certificates described so far are objects that a checker examines. A different architecture runs the decision procedure *inside* the checker. Suppose a kernel can evaluate functions and has a theorem relating a Boolean-valued function $A : X \rightarrow \text{Bool}$ to a property P :

$$\text{sound} : \forall x, A(x) = \text{true} \rightarrow P(x).$$

Definition 8.7 (Reflective proof). The *reflective proof* of $P(x_0)$ is the term $\pi(x_0) = \text{sound } x_0 \text{ rfl}$.

The kernel accepts $\pi(x_0) : P(x_0)$ exactly when it can verify that the argument $\text{rfl} : A(x_0) = \text{true}$ typechecks, that is, when it reduces $A(x_0)$ to true by computation.

Theorem 8.8 (Size and cost of reflective proofs). *Assume sound is a theorem accepted by the kernel. Then $\pi(x_0)$ has size $|x_0| + O(1)$ plus the size of a reference to sound, independent of the running time of A . The kernel’s checking cost is the cost of reducing $A(x_0)$. If $\pi(x_0)$ is accepted, then $P(x_0)$ holds in every interpretation validating the kernel.*

Proof. The term consists of one constant, one argument x_0 , and the constant rfl . Acceptance requires the type of rfl , which is $A(x_0) = A(x_0)$, to be convertible with $A(x_0) = \text{true}$, which holds if and only if $A(x_0)$ and true are definitionally equal, that is, if $A(x_0)$ reduces to true. Soundness of the kernel gives the last claim. ■

Reflective proofs invert the usual trade-off: the certificate is tiny, the check is expensive, and the correctness argument has moved into the proof of sound, which is done once. They are also the extreme case of a trace-free certificate. The proof term contains nothing that depends on how A reached its answer, and the reduction performed by the kernel is not recorded. A reader who asks *why* $P(x_0)$ holds is told to rerun the computation. This will become the example of an object that is sufficient for checking and *minimally* informative for diagnosis, in the sense made precise in Chapter 10.

A variant evaluates $A(x_0)$ with a compiled evaluator outside the kernel and records the outcome through an axiom (Example 7.9). The proof term is the same and the cost drops, at the price of replacing the kernel's reduction by trust in the compiler. The footprint (Definition of Section 7.4) is where the difference appears.

8.4 How large must certificates be?

Certificate size is a resource in its own right. For propositional logic the question has a famous formulation.

Theorem 8.9 (Cook and Reckhow 1979). [CITED: COOK AND RECKHOW] *There is a propositional proof system, in the sense of Definition 1.1 restricted to propositional tautologies and polynomial-time Chk, in which every tautology has a proof of polynomial size in the size of the tautology, if and only if $NP = co-NP$.*

This sits beside Proposition 2.10, which showed that for first-order logic no computable bound exists. For propositional logic, bounds exist for each fixed system (Haken's theorem, Theorem 5.9, shows exponential lower bounds for resolution), and the existence of a system with polynomial bounds for all tautologies is equivalent to an open question of complexity theory. Whatever the answer, the shape of the situation is that of the first chapter: a certificate scheme is a compression of the evidence, and the compressed object can be large.

Verified checkers for the standard clausal certificate formats have been built inside proof assistants (Cruz-Filipe, Heule, Hunt, Kaufmann, Schneider-Kamp 2017; Tan, Heule, Myreen 2021), so that for propositional unsatisfiability the chain from certificate to meaning can be mechanized end to end, leaving the encoding of the user's problem as the remaining item of the trusted base.

8.5 Certificates are bound to inputs

A witness certifies $R(x, y)$ for the particular x it was checked against. Two features follow, and Chapter 12 builds on both.

Proposition 8.10 (Fragility of binding). *In general $Wit(x, y, w)$ does not imply $Wit(x', y, w)$ for an instance x' equivalent to x in meaning. Even an alphabetic renaming of variables may invalidate a literal-clause proof.*

Proof. A DRAT proof refers to variables by number. Let $F = \{\{1, 2\}, \{-1\}, \{-2\}\}$ with the proof consisting of the lines $\{2\}$ and $\square$: the line $\{2\}$ is RUP, since assuming $\neg 2$ makes $\{1, 2\}$ unit, propagating 1 and falsifying $\{-1\}$. Rename variable 2 to 3 to obtain $F' =$

$\{\{1, 3\}, \{-1\}, \{-3\}\}$, equivalent to F up to renaming. The same proof fails on F' : the line $\{2\}$ is not RUP, because assuming $\neg 2$ propagates nothing. ■

The certificate's validity is a fact about a syntactic pair, and its *meaning* (the semantic claim it supports) is a fact about the pair's interpretation. The two can come apart when an artifact is transformed. An analogous point holds for proof terms against statements: a term checked against a type is checked against that exact type up to definitional equality, not up to logical equivalence.

8.6 Certificates are not explanations

By Theorem 8.2, any two witnesses for the same (x, y) serve the acceptance task equally. This is both the strength of certificates and the reason they cannot serve every purpose. Consider tasks that depend on the witness itself: finding the *shortest* witness, finding the witness that uses a given lemma, locating which input clauses a witness never touches, adapting a witness to a perturbed input. These are functions of w , not of (x, y) , and by Proposition 1.12 none of them factors through the acceptance relation. The consequence for later chapters is stated here in its simplest form.

Proposition 8.11 (Acceptance is invariant on witnesses). *Let w_1, w_2 be two witnesses with $\text{Wit}(x, y, w_1)$ and $\text{Wit}(x, y, w_2)$. Any task that depends on the acceptance of (x, y, w) alone gives the same result for w_1 and w_2 . A task with $T(w_1) \neq T(w_2)$ is not determined by acceptance.*

Proof. The first sentence is the definition of a task depending on acceptance only; the second is its contrapositive. ■

In Part III this will be developed as the claim that a verified result serves a *family* of tasks, of which checking is one, and that the right question about any representation is which members of the family it is sufficient for.

Exercise 8.1. Complete the table of Section 8.1 with a row for graph non-isomorphism, and explain why a witness scheme with a deterministic polynomial-time checker is not known for it.

Exercise 8.2. Show that the composition of an unsat-reflecting transformation with a model-reconstructing one is unsat-reflecting but need not reconstruct models. What additional data must be retained for a pipeline whose first stage is only unsat-reflecting?

Exercise 8.3. For a reflective proof $\pi(x_0)$ as in Definition 8.7, describe a modified decision function A' that returns, besides true, a trace of its computation, and a corresponding soundness theorem sound' such that the trace is part of the proof term. State how this changes the size bound of Theorem 8.8.

Part III

Sufficiency of Verified Results

Chapter 9

Verification Obligations

Part II ended with a checked derivation and a list of what the check leaves out: the translation from the intended claim, the meaning of the definitions, the structure of the search. This chapter begins the construction of a framework for the second half of that list, where what is at stake is not whether a proof is correct but whether a *correct proof* supports the use that is made of it. In the sciences the use is a claim about a physical or empirical system, and three questions can then be asked of a machine-checked result separately: whether the mathematics is right, whether the mathematical model respects the principles the science takes for granted, and whether the model corresponds to what is measured. The questions are independent, in a sense that can be made exact, and the three answers need different kinds of evidence. The chapter defines a verified result as a structured object, proves the independence, gives one of the obligations (dimensional consistency) a complete formal treatment, and introduces the ledger in which an honest report records which obligations are discharged by what.

9.1 A verified result is a tuple

Definition 9.1 (Bridged result). A *bridged result* consists of

- (i) a proof system S (Definition 1.1) and a *formal theorem* $\varphi \in \Phi$ with a checked derivation $d, \text{Chk}(d, \varphi)$;
- (ii) a class $\mathcal{M}$ of *models* (mathematical objects, such as dynamical systems or structures) of which φ is a statement;
- (iii) a class $\mathcal{P}$ of *target systems* (physical, computational, or empirical) together with a *bridge* $\beta : \mathcal{P} \rightarrow \mathcal{M}$ assigning to each target system within its *scope* $S = \text{dom}(\beta)$ an idealizing model, under stated *bridge hypotheses*;
- (iv) an *observation protocol* ν specifying, for each target system $p \in S$, which operations produce data and how data are compared with the values of model quantities, with a stated error model; and
- (v) an *empirical claim* ψ about target systems in S , asserted to follow from φ together with the bridge hypotheses.

The tuple makes explicit what Proposition 1.7 left implicit. The kernel certifies (i) alone. Everything else is an obligation of a different kind. Their separation has a precise form.

Definition 9.2 (Three obligation layers). For a bridged result,

- (O1) **mathematical correctness:** $\text{Chk}(d, \varphi)$ holds, and the definitions in the closure of d formalize the models of $\mathcal{M}$ faithfully (items (T1) to (T3) of Definition 7.10);
- (O2) **compatibility with principles:** every model in $\mathcal{M}$ satisfies the principles declared for the science in question (dimensional homogeneity, conservation laws, symmetries, causality), each stated as a predicate Π_j on models;
- (O3) **correspondence:** for every $p \in S$ the model $\beta(p)$ predicts the values given by the protocol ν within the declared tolerance, and the conditions under which this prediction is counted as success or failure were fixed before the data were collected.

The layers are logically independent: no two imply the third.

Proposition 9.3 (Independence of the layers). *For each choice of a layer there is a bridged result satisfying the other two and failing that one.*

Proof. We give a one-line witness for each; the second and third are developed in the examples below.

(O1) *fails*, (O2) *and* (O3) *hold*. A dimensionally homogeneous, empirically excellent conjecture with no proof: the relation $E = mc^2$ before any derivation from the premises of a theory, stated as a theorem of a formal system with a missing proof. Here $\text{Chk}(d, \varphi)$ fails for every d available, while the model satisfies the principles and agrees with measurement.

(O2) *fails*, (O1) *and* (O3) *hold in a trivial regime*. Example 9.7 below: a formally proved statement about a sum of a length and a time whose truth changes with units, which by choice of units can be made to agree with any single measurement.

(O3) *fails*, (O1) *and* (O2) *hold*. Example 9.9: the free-fall law proved for the ideal model and applied where drag cannot be neglected. ■

The proposition is deliberately weak in what it asserts, namely independence of satisfiability, and strong in what it forbids: no verification of one layer, however thorough, discharges another. In particular, a machine-checked proof is about O1 only.

9.2 Dimensional homogeneity as a theorem

Layer (O2) is easy to leave abstract, so we give one principle a complete treatment. It is chosen because it is checkable, because it is routinely violated in informal and sometimes in formal work, and because the kernel does not check it.

Fix k base dimensions (length, time, mass, and so on). A *dimension* is a vector $a \in \mathbb{Z}^k$. A *dimensional context* Γ assigns a dimension to each variable. *Terms* are generated by variables, real constants (dimensionless), sums $t_1 + t_2$, products $t_1 t_2$, integer powers t^n , and applications $g(t)$ of functions g from a fixed set G of transcendental functions (such as $\exp, \sin, \log$).

Definition 9.4 (Dimensional typing). The judgment $\Gamma \vdash t : a$ is generated by:

- $\Gamma \vdash x : \Gamma(x)$ and $\Gamma \vdash c : 0$ for a real constant c ;
- if $\Gamma \vdash t_1 : a$ and $\Gamma \vdash t_2 : a$ then $\Gamma \vdash t_1 + t_2 : a$;
- if $\Gamma \vdash t_1 : a$ and $\Gamma \vdash t_2 : b$ then $\Gamma \vdash t_1 t_2 : a + b$;
- if $\Gamma \vdash t : a$ then $\Gamma \vdash t^n : na$;
- if $\Gamma \vdash t : 0$ and $g \in G$ then $\Gamma \vdash g(t) : 0$.

A *change of units* is an element $\lambda \in (\mathbb{R}_{>0})^k$. It acts on a dimension a by the scalar $\lambda^a = \prod_i \lambda_i^{a_i}$, and on an assignment x of real values to variables by $(\lambda \cdot x)(v) = \lambda^{\Gamma(v)} x(v)$.

Theorem 9.5 (Dimensional homogeneity). *If $\Gamma \vdash t : a$ then $t(\lambda \cdot x) = \lambda^a t(x)$ for every change of units λ and assignment x .*

Proof. By induction on the derivation. For a variable v : $t(\lambda \cdot x) = \lambda^{\Gamma(v)} x(v)$. For a constant, $\lambda^0 c = c$. For a sum with both summands of dimension a : $t_1(\lambda \cdot x) + t_2(\lambda \cdot x) = \lambda^a (t_1(x) + t_2(x))$. For a product: the factors scale by λ^a and λ^b , so the product scales by λ^{a+b} . For a power: $(\lambda^a t(x))^n = \lambda^{na} t(x)^n$. For $g(t)$ with $t : 0$: $t(\lambda \cdot x) = t(x)$, so $g(t)(\lambda \cdot x) = g(t(x)) = \lambda^0 g(t)(x)$. ■

Corollary 9.6 (Unit invariance of well-typed equations). *If $\Gamma \vdash t_1 : a$ and $\Gamma \vdash t_2 : a$ then for every assignment x and change of units λ , $t_1(x) = t_2(x)$ if and only if $t_1(\lambda \cdot x) = t_2(\lambda \cdot x)$. The solution set of a well-typed equation is therefore invariant under the action of units.*

Proof. Both sides scale by the same positive factor λ^a . ■

The converse direction, that every unit-invariant relation among quantities of given dimensions is expressible through dimensionless combinations, is the Buckingham π theorem ([CITED: BUCKINGHAM 1914]); it is not needed here.

Example 9.7 (An ill-typed statement that is formally provable). Take $\Gamma(x) = (1, 0)$ (a length) and $\Gamma(y) = (0, 1)$ (a time), and the equation $x + y = 1$. The term $x + y$ has no dimensional type, since the summands have different dimensions. The equation is satisfied by $x = y = 1/2$. Under the change of units $\lambda = (2, 1)$ (halving the length unit) the same physical situation is described by $\lambda \cdot x = (1, 1/2)$, and $1 + 1/2 \neq 1$. The truth of the equation depends on the choice of units, so it cannot express a fact about the system.

The kernel, however, sees only real numbers. The statement $\forall x y : \mathbb{R}, x = y \rightarrow x + y = 2x$, for instance, or any specific instance of the equation above, is a perfectly well-typed formal statement of an ordinary real-number theory, and can be proved. The dimensional well-typedness of a formalization is not checked by the kernel unless the formalization encodes dimensions in its types, which the standard libraries of real analysis do not.

Remark 9.8 (What the example shows about the trusted base). Example 9.7 is the observation of Proposition 7.11 for item (T3): a theorem about a formal definition of a physical quantity certifies only the formal object. Dimensional consistency is a checkable property of the formalization, as Theorem 9.5 shows, and checking it is an obligation of layer (O2) that can be discharged by a decision procedure for $\Gamma \vdash t : a$, or by a typed encoding of dimensions inside the proof assistant. It must be discharged by something.

9.3 Correspondence and scope: the free-fall example

Layer (O3) needs the bridge to be explicit, and the bridge carries the scope.

Example 9.9 (Free fall). The ideal model $\mathcal{M}_0$ is the equation $\dot{v} = g$ with $v(0) = 0$. Its solution is $v(t) = gt$, and the statement $\forall t, v(t) = gt$ is provable in any standard formalization of real analysis (O1). The statement is dimensionally homogeneous: $[v] = LT^{-1}$, $[g] = LT^{-2}$, $[t] = T$ (O2). The empirical claim ψ is “an object released at rest from a

high place has speed gt after t seconds". The bridge sends a falling body to $\mathcal{M}_0$, and its hypothesis is that drag is negligible.

Take the model with linear drag, $m\dot{v} = mg - kv$. Its solution is $v_{\text{drag}}(t) = (mg/k)(1 - e^{-kt/m})$. With $\epsilon = kt/m$ the relative error of the ideal law is

$$\frac{gt - v_{\text{drag}}(t)}{gt} = 1 - \frac{1 - e^{-\epsilon}}{\epsilon} \leq \frac{\epsilon}{2},$$

using $e^{-\epsilon} \leq 1 - \epsilon + \epsilon^2/2$. So the ideal law is accurate to relative error $\epsilon/2$ whenever $kt/m \leq \epsilon$, and the empirical claim has a scope: $S_\epsilon = \{(t, k, m) : kt/m \leq \epsilon\}$. Outside it the theorem remains true of $\mathcal{M}_0$ and the claim ψ is false of the target, to a degree that grows with ϵ .

The example separates cleanly what the kernel can and cannot do. The inequality $\epsilon/2$ bounding the error is a second theorem, about the pair of models $\mathcal{M}_0$ and the drag model, and it too can be machine-checked. What the kernel cannot do is decide that a linear drag model is the right comparison class for a real falling body, or that its parameter k has the value a particular experiment requires. Those are bridge hypotheses, and their support is empirical.

Proposition 9.10 (Post hoc scope immunizes a claim). *Let a claim be a pair (ψ, S) of a prediction map $P : S \rightarrow \text{Out}$ (conditions to sets of acceptable outcomes) and a scope S . An observation (c, o) refutes the claim if $c \in S$ and $o \notin P(c)$. If the scope may be revised after the observation is made, then no observation refutes the claim: for any (c, o) with $o \notin P(c)$, replace S by $S \setminus \{c\}$.*

Proof. After revision, $c \notin S$, so the observation does not satisfy the refutation condition. ■

The proposition is a trivial fact with a pointed consequence. A claim is falsifiable only to the extent that the scope was declared before data were seen; hence the requirement in (O3) that the success and failure conditions be fixed in advance. Scope declarations are therefore part of the claim, and a *revision* of scope is a new claim, with a new set of obligations.

9.4 The ledger

An honest report of a verified result lists its obligations and, for each, the evidence that discharges it and the kind of that evidence.

Definition 9.11 (Obligation ledger). An *obligation* o has a *kind* (formal, dimensional, principle, bridge, scope, measurement), a *statement*, a set $\text{deps}(o)$ of obligations it depends on, and a *status* in

- *proved*(d): discharged by a checked derivation or a decision procedure whose verdict is checked;
- *tested*(plan, r): discharged by a preregistered test with plan plan and result r that met the preregistered success condition;
- *assumed*: used without discharge, explicitly declared;
- *open*: not yet addressed;
- *refuted*: a proof or a preregistered test has contradicted it.

A *ledger* is a finite acyclic graph of obligations. A claim ψ has a set $D(\psi)$ of obligations on which it depends.

Let Sup , the *supported* set, be the least set of obligations containing every o whose status is proved or tested and whose dependencies lie in Sup .

Proposition 9.12 (Supported claims). (i) If $D(\psi) \subseteq \text{Sup}$ then ψ is unconditionally supported. Otherwise ψ is supported at most conditionally on the obligations in $D(\psi) \setminus \text{Sup}$ of status assumed or open.
(ii) If some obligation in the dependency closure of ψ is refuted, then ψ is not supported.
(iii) The retraction of a non-formal obligation does not retract any proved(d) obligation of kind formal that does not depend on it.

Proof. (i) and (ii) are the definition of Sup together with the observation that a refuted obligation has status outside $\{\text{proved}, \text{tested}\}$ and so lies outside Sup , as do all obligations whose dependencies include it. For (iii), Sup is defined by dependencies, and a formal obligation not depending on the refuted one is unaffected by its status. ■

Part (iii) states the independence of the layers (Proposition 9.3) in the language of the ledger. When a physical claim is retracted because the bridge fails, the *theorem* on which it was based stands. This is a feature, not an embarrassment, of formal verification: the failure is localized to the layer where it occurred. A report that does not separate the layers invites the opposite inference, that the failure of the claim undermines the proof or that the proof secures the claim.

9.5 What discharges what

The assignment of evidence to obligation types is summarized below. The right column states the *kind* of evidence admissible; the difference between a proof and a test is not a difference of degree.

Obligation	Discharged by	Checker
Theorem φ holds	derivation d	kernel (Ch. 7)
Definitions faithful	review against the intended notion; sometimes a second formalization and a proved equivalence	humans; kernel for the equivalence
Dimensional homogeneity	typing derivation (Def. 9.4)	decision procedure or typed encoding
Conservation or symmetry principle	a theorem about the model class	kernel
Bridge hypotheses	derivation of the bound relating models, as in Example 9.9; otherwise an empirical test	kernel for the derivation; experiment for the rest
Scope	preregistered declaration (Prop. 9.10)	procedure; audit
Correspondence	preregistered measurement with a stated success condition	experiment

Two rows deserve emphasis. The fourth shows that parts of the bridge *can* be proved, namely the mathematical relation between an idealized model and a more detailed one,

and that doing so converts an assumed bridge hypothesis into a theorem about a family of models. The last shows that correspondence is never discharged by proof.

The remaining chapters of Part III address what a verified result must retain in order for these obligations to stay dischargeable under later work. Chapter 10 formalizes the notion of sufficiency of a representation for a declared family of tasks, which is the right general framework. Chapter 11 shows what is lost when a result is retained only as a certificate. Chapter 12 treats what carries over when the artifact is transformed.

Exercise 9.1. Extend Definition 9.4 to the case where integer powers are replaced by rational powers $t^{p/q}$, noting where the proof of Theorem 9.5 needs positivity of λ .

Exercise 9.2. Using the series for $e^{-\epsilon}$, prove the stronger bound $1 - (1 - e^{-\epsilon})/\epsilon \leq \epsilon/2$ for all $\epsilon > 0$, and show that the constant $1/2$ cannot be improved.

Exercise 9.3. Give a ledger with five obligations in which the retraction of a single obligation of kind scope leaves exactly two claims unsupported and no formal obligation retracted.

Chapter 10

Sufficiency of Certificates

Chapter 1 showed that a task is served by a representation exactly when it is constant on the fibres of the extraction map (Proposition 1.12). That statement is true of any function, and its content in the present setting depends entirely on which tasks are in play. This chapter makes the dependence precise. A *task family* is declared, sufficiency is defined relative to it, and the resulting structure is shown to be a Galois connection on the partition lattice of the set of histories. Three consequences follow: a minimal sufficient representation exists and is unique up to isomorphism; sufficiency has a quantitative lower bound in cardinality and in entropy; and no representation that identifies two distinct histories is sufficient for every task. The last statement is the formal core of the claim that a verified result cannot be both maximally compressed and maximally reusable.

10.1 Histories, representations and task families

Definition 10.1 (Setting). A *history space* is a nonempty set H . A *representation* is a function $\sigma : H \rightarrow R$. A *task* with codomain Y is a function $f : H \rightarrow Y$. A *task family* is a set $\mathcal{F}$ of tasks, with codomains allowed to vary. The *kernel* of a function g on H is the equivalence relation $\ker g = \{(h, h') \in H^2 : g(h) = g(h')\}$.

In the application, H is the set of complete runs of a search procedure on a goal (the set $\text{Run}(\varphi)$ of Chapter 1), and the canonical representation is the extraction map $\varrho : \text{Run}(\varphi) \rightarrow D$ to the checked derivation. The abstraction is nevertheless useful because the same definitions apply to the obligation ledgers of Chapter 9, to proof terms modulo definitional equality (Chapter 7), and to normal forms (Chapter 4).

Definition 10.2 (Sufficiency). A representation $\sigma : H \rightarrow R$ is *f-sufficient* if there is $f' : R \rightarrow Y$ with $f = f' \circ \sigma$, and *$\mathcal{F}$ -sufficient* if it is *f-sufficient* for every $f \in \mathcal{F}$.

By Proposition 1.12, σ is *f-sufficient* if and only if $\ker \sigma \subseteq \ker f$. The family enters through a single relation.

Definition 10.3 (Family kernel). For a nonempty family $\mathcal{F}$ put $\ker \mathcal{F} = \bigcap_{f \in \mathcal{F}} \ker f$, that is, $h \sim_{\mathcal{F}} h'$ if and only if $f(h) = f(h')$ for all $f \in \mathcal{F}$. For the empty family $\ker \emptyset = H^2$.

Equivalently, $\ker \mathcal{F}$ is the kernel of the joint task $\langle \mathcal{F} \rangle : H \rightarrow \prod_{f \in \mathcal{F}} Y_f$, $h \mapsto (f(h))_f$.

Theorem 10.4 (Sufficiency is refinement). *A representation σ is $\mathcal{F}$ -sufficient if and only if $\ker \sigma \subseteq \ker \mathcal{F}$. Consequently:*

- (i) *the quotient map $q_{\mathcal{F}} : H \rightarrow H / \ker \mathcal{F}$ is $\mathcal{F}$ -sufficient;*
- (ii) *every $\mathcal{F}$ -sufficient σ factors $q_{\mathcal{F}}$, that is, $q_{\mathcal{F}} = u \circ \sigma$ for a unique $u : \sigma(H) \rightarrow H / \ker \mathcal{F}$; and*
- (iii) *if $\mathcal{F} \subseteq \mathcal{F}'$ then every $\mathcal{F}'$ -sufficient representation is $\mathcal{F}$ -sufficient.*

Proof. If σ is $\mathcal{F}$ -sufficient then $\ker \sigma \subseteq \ker f$ for every $f \in \mathcal{F}$, hence $\ker \sigma \subseteq \bigcap_f \ker f$. Conversely, if $\ker \sigma \subseteq \ker \mathcal{F} \subseteq \ker f$, Proposition 1.12 gives sufficiency for f . For (i), $\ker q_{\mathcal{F}} = \ker \mathcal{F}$. For (ii), define $u(\sigma(h)) = [h]_{\mathcal{F}}$; this is well defined because $\sigma(h) = \sigma(h')$ implies $(h, h') \in \ker \sigma \subseteq \ker \mathcal{F}$, and it is unique because σ is onto $\sigma(H)$. For (iii), $\ker \mathcal{F}' \subseteq \ker \mathcal{F}$. ■

Theorem 10.4(ii) says that $q_{\mathcal{F}}$ is the *minimal sufficient representation*: it retains exactly the distinctions the family can detect and no others, and every sufficient representation retains at least these. The set of $\mathcal{F}$ -sufficient kernels is the down-set of the partition lattice $\Pi(H)$ generated by $\ker \mathcal{F}$, ordered by refinement. In particular it is closed under common refinement (meet in the refinement order) and under passage to finer partitions, and it is *not* closed under coarsening beyond $\ker \mathcal{F}$.

Example 10.5 (A derivation as a representation). Let H consist of runs $h = (d, A)$ where d is a checked derivation of a fixed goal and $A \subseteq \{1, \dots, k\}$ is the set of indices of alternative lemmas that were tried and refuted before d was found. Take $\sigma(d, A) = d$. For the family $\mathcal{F}_{\text{chk}} = \{h \mapsto \text{Chk}(d, \varphi)\}$ the kernel $\ker \mathcal{F}_{\text{chk}}$ identifies all runs with the same d , so σ is sufficient and indeed minimal. For the family $\mathcal{F}_{\text{ref}} = \{h \mapsto [i \in A] : 1 \leq i \leq k\}$ the kernel is $\{((d, A), (d', A')) : A = A'\}$, which σ does not refine whenever there are two runs with equal d and different A . Each of the 2^k subsets A is a separate class of $\ker \mathcal{F}_{\text{ref}}$, so a sufficient representation for $\mathcal{F}_{\text{ref}}$ has at least 2^k values (Theorem 10.10).

10.2 The Galois connection and the scope of a representation

Declaring a family is one half of a duality; the other half asks what a given representation serves.

Definition 10.6 (Served tasks and sufficient representations). For a representation σ let $\text{Serv}(\sigma)$ be the set of all tasks $f : H \rightarrow Y$ (any codomain) with $\ker \sigma \subseteq \ker f$. For a family $\mathcal{F}$ let $\text{Rep}(\mathcal{F})$ be the set of $\mathcal{F}$ -sufficient representations.

Theorem 10.7 (Closure of a task family). *The pair $(\text{Rep}, \text{Serv})$ is a Galois connection in the sense that $\mathcal{F} \subseteq \text{Serv}(\sigma)$ if and only if $\sigma \in \text{Rep}(\mathcal{F})$. Moreover*

$$\text{Serv}(\text{Rep}(\mathcal{F})) := \bigcap_{\sigma \in \text{Rep}(\mathcal{F})} \text{Serv}(\sigma) = \{f : \ker \mathcal{F} \subseteq \ker f\},$$

which is the set of tasks of the form $g \circ \langle \mathcal{F} \rangle$ for some function g . In words: a representation sufficient for $\mathcal{F}$ is automatically sufficient for every task computable from the joint values of $\mathcal{F}$, and for no other task in general.

Proof. The first statement is the definition of both sets. For the second, if $\ker \mathcal{F} \subseteq \ker f$ and $\sigma \in \text{Rep}(\mathcal{F})$ then $\ker \sigma \subseteq \ker \mathcal{F} \subseteq \ker f$ by Theorem 10.4, so $f \in \text{Serv}(\sigma)$. Conversely,

the quotient $q_{\mathcal{F}}$ lies in $\text{Rep}(\mathcal{F})$, so any f in the intersection satisfies $\ker \mathcal{F} = \ker q_{\mathcal{F}} \subseteq \ker f$. Finally $\ker \mathcal{F} \subseteq \ker f$ holds if and only if $f = g \circ \langle \mathcal{F} \rangle$ for some g , by Proposition 1.12 applied to $\langle \mathcal{F} \rangle$. ■

The closure operator $\mathcal{F} \mapsto \text{Serv}(\text{Rep}(\mathcal{F}))$ is idempotent, monotone and extensive, as for any Galois connection. Its practical content is a precise statement of what is gained and not gained by declaring a family. Declaring the checking task serves also every function of the checking verdict; declaring the checking task and the lemma-refutation tasks together serves all functions of the pair; but nothing declared serves a task whose answer depends on a distinction that none of the declared tasks detects. That a task fails to be in the closure is a decidable property when H is finite and the tasks are given, and in general it is as hard as deciding the kernel inclusion.

Remark 10.8 (Verified by enumeration). For all history spaces with at most six elements, and for randomly drawn families of up to three tasks, the equivalence in Theorem 10.4 and the characterization of the closure in Theorem 10.7 were confirmed by exhaustive enumeration of every representation up to relabelling (about three thousand families, no failures). This is a sanity check of the statements, not a proof; the proofs above are complete.

Extending a representation to a larger family

A result is certified once, against a family declared at the time. Later uses enlarge the family, and the question is what the cheapest extension is.

Proposition 10.9 (Cheapest extension). *Let σ be $\mathcal{F}$ -sufficient and let $\mathcal{F}'$ be a further family. Among representations τ that are $\mathcal{F}'$ -sufficient and that serve every task served by σ (that is, $\text{Serv}(\sigma) \subseteq \text{Serv}(\tau)$), the coarsest has kernel $\ker \sigma \cap \ker \mathcal{F}'$, and its number of values is at most $|\sigma(H)| \cdot |\langle \mathcal{F}' \rangle(H)|$.*

Proof. The condition $\text{Serv}(\sigma) \subseteq \text{Serv}(\tau)$ holds if and only if $\ker \tau \subseteq \ker \sigma$: the implication from right to left is immediate, and for the converse take the task σ itself, which lies in $\text{Serv}(\sigma)$ and so in $\text{Serv}(\tau)$, giving $\ker \tau \subseteq \ker \sigma$. Together with $\ker \tau \subseteq \ker \mathcal{F}'$ this gives $\ker \tau \subseteq \ker \sigma \cap \ker \mathcal{F}'$, and the quotient by this intersection satisfies both conditions. The classes of $\ker \sigma \cap \ker \mathcal{F}'$ are the nonempty intersections of a class of $\ker \sigma$ with a class of $\ker \mathcal{F}'$, whence the bound. ■

The extension is the pairing $\langle \sigma, \langle \mathcal{F}' \rangle \rangle$ restricted to its image. Nothing cheaper is available, and the bound is generally not attained: whenever the new tasks are partly determined by σ the number of classes is smaller. When the new family is a function of old data, $\ker \sigma \subseteq \ker \mathcal{F}'$, the extension has no cost.

10.3 Lower bounds

Theorem 10.10 (Cardinality bound). *If σ is $\mathcal{F}$ -sufficient then $|\sigma(H)| \geq |H/\ker \mathcal{F}| = |\langle \mathcal{F} \rangle(H)|$. In particular, any binary encoding of an $\mathcal{F}$ -sufficient representation of a finite H has length at least $\lceil \log_2 |\langle \mathcal{F} \rangle(H)| \rceil$ for some history, and this is attained by $q_{\mathcal{F}}$.*

Proof. By Theorem 10.4(ii) there is a surjection $u : \sigma(H) \rightarrow H/\ker \mathcal{F}$, so the domain has at least as many elements. The quotient $H/\ker \mathcal{F}$ is in bijection with the image of $\langle \mathcal{F} \rangle$. A

binary code assigning distinct strings to N values has a string of length at least $\lceil \log_2 N \rceil$ for some value, by the pigeonhole principle; the quotient attains $\lceil \log_2 N \rceil$ for fixed-length codes. ■

For Example 10.5 the bound gives 2^k classes and k bits, whatever else is stored. The derivation, which contributes zero bits to this family, is entirely irrelevant to it.

The same bound holds in expectation under a probability distribution on H , which is the natural setting if the family is to be served for an unknown, randomly arising run. Distributions are not given in this book and none is invented; the statement below holds for every distribution and is used only qualitatively.

Theorem 10.11 (Entropic characterization). *Let P be a probability distribution on a finite set H , let $S = \sigma(h)$ and $T = f(h)$ for $h \sim P$. Then f is σ -computable P -almost surely, that is, there is f' with $f'(S) = T$ almost surely, if and only if $H(T | S) = 0$. In that case $H(S) \geq H(T)$, with equality if and only if S is also a function of T almost surely.*

Proof. If $T = f'(S)$ almost surely then T is determined by S and $H(T | S) = 0$. If $H(T | S) = 0$ then for each s with positive probability the conditional distribution of T is a point mass, and $f'(s)$ may be taken as its support; elsewhere f' is arbitrary. By the chain rule

$$H(S) = H(S, T) - H(T | S) = H(S, T) = H(T) + H(S | T) \geq H(T),$$

with equality if and only if $H(S | T) = 0$. ■

The information-theoretic version of the lossy case, which does not appear in the remainder of the book except as a remark, is the data-processing inequality $I(T; g(S)) \leq I(T; S)$ for any function g [CITED: COVER AND THOMAS, *ELEMENTS OF INFORMATION THEORY*, THEOREM 2.8.1]. It shows that sufficiency, in the sense $I(T; S) = H(T)$, cannot be restored by further processing once lost.

10.4 No free sufficiency

The preceding results are meaningful only if some tasks are not served by compressed representations. The next theorem makes that unconditional.

Theorem 10.12 (No free sufficiency). *Let $\sigma : H \rightarrow R$. The following are equivalent.*

- (i) σ is injective.
- (ii) σ is $\mathcal{F}$ -sufficient for the family $\mathcal{F}_{\text{all}}$ of all functions $H \rightarrow \{0, 1\}$.
- (iii) σ is $\mathcal{F}$ -sufficient for the family $\mathcal{F}_{\text{pt}} = \{\mathbf{1}_{\{h\}} : h \in H\}$ of point indicators.

In particular, if σ identifies two distinct histories then there is a task, indeed a $\{0, 1\}$ -valued one, that σ does not serve.

Proof. (i) $\Rightarrow$ (ii): if σ is injective, any f factors as $f \circ \sigma^{-1}$ on the image and arbitrarily elsewhere. (ii) $\Rightarrow$ (iii) is trivial since $\mathcal{F}_{\text{pt}} \subseteq \mathcal{F}_{\text{all}}$. (iii) $\Rightarrow$ (i): suppose $\sigma(h) = \sigma(h')$ with $h \neq h'$. Then $\mathbf{1}_{\{h\}}(h) = 1 \neq 0 = \mathbf{1}_{\{h\}}(h')$, so $\ker \sigma \not\subseteq \ker \mathbf{1}_{\{h\}}$. ■

Corollary 10.13 (Compression is a bet on a family). *Every non-injective representation of the runs of a search procedure is sufficient for some declared families and insufficient for others. The set of families it is sufficient for is exactly $\{\mathcal{F} : \ker \sigma \subseteq \ker \mathcal{F}\}$, an up-closed set in the lattice of families under the order by kernel inclusion.*

The corollary is the formal justification for requiring a declaration. An artifact that carries only a derivation is a bet that no later task will distinguish runs with the same derivation. The bet is often correct for checking and for citation; it is not correct for tasks whose answer depends on the discarded search record, such as asking which strengthening of a hypothesis would have been cheaper to prove, or why a related goal did not succeed under the same strategy.

10.5 Proof irrelevance as a sufficiency statement

Chapter 7 noted that proof irrelevance in a type theory with a proof-irrelevant sort of propositions collapses all proofs of the same proposition. The language of this chapter turns that observation into a theorem about which tasks survive.

Definition 10.14 (Proof-irrelevant representation). Let H be a set of pairs (P, p) of a proposition P and a proof term p of P (up to definitional equality). The *proof-irrelevant representation* is $\pi(P, p) = P$.

Proposition 10.15 (Tasks served by proof irrelevance). *A task f is π -sufficient if and only if $f(P, p) = f(P, p')$ for all $(P, p), (P, p') \in H$, that is, if and only if its answer depends on the proposition alone. In particular:*

- (i) *type checking of any term that mentions the proof only through its type is π -sufficient;*
- (ii) *the task “return the algorithm encoded by p ” is not π -sufficient whenever two proofs of some P encode different algorithms;*
- (iii) *the task “does p use the axiom a ” is not π -sufficient in general, so axiom footprints are not an invariant of the proposition.*

Proof. The first statement is Proposition 1.12 for π . For (i), a term of the form $c P p$ is typed by the type of p alone, by the definition of the typing rule for a proof-irrelevant argument. For (ii) and (iii), let P have proofs p, p' with different footprints; Chapter 7 gives such pairs, for instance a proof that uses classical choice and one that does not. The task is non-constant on the fibre over P . ■

This is the precise sense in which proof irrelevance is an *admissible collapse* for dependents that consume a theorem by its statement and an *inadmissible collapse* for any procedure that must recover what was done. It is not that one is correct and the other wrong; they are tasks with different kernels, and the declared family decides.

10.6 A geometric instance: exterior data and hidden material

The abstract statements of this chapter have a concrete instance in which the extraction map, its fibres, and a decidable certificate for the nonemptiness of a fibre can all be written down. A flat sheet is mapped into space; the extraction keeps only what the map does on a marked part of the sheet; the question is what that restriction fixes about the rest.

Definition 10.16 (Marking problem). Let $\Sigma \subset \mathbb{R}^2$ be a compact convex set with nonempty interior, the *sheet*. A *marking* is a compact set $A = A_1 \cup \dots \cup A_k \subset \Sigma$ with the A_i pairwise disjoint, together with a *target* $\tau : A \rightarrow \mathbb{R}^3$ whose restriction to each A_i preserves distances.

A *path isometry* is a continuous map $f : \Sigma \rightarrow \mathbb{R}^3$ such that $f \circ \gamma$ has the same length as γ for every rectifiable path γ in Σ ; every C^1 isometric immersion is one. For $\delta \geq 0$, a δ -*realization* of τ is a path isometry f with $|f(p) - \tau(p)| \leq \delta$ for $p \in A$; a 0-realization is a *realization*. Write $\mathcal{R}_\delta(\tau)$ for the set of δ -realizations. The *exterior map* is $E(f) = f|_A$, so that $\mathcal{R}_0(\tau) = E^{-1}(\tau)$ is a fibre of E on the set of path isometries.

Path isometries may self-intersect. Nothing below concerns embeddedness, which is a stricter question.

Definition 10.17 (Nonexpansion condition). The target τ satisfies (L) if $|\tau(p) - \tau(q)| \leq |p - q|$ for all $p, q \in A$, and (L[<]) if the inequality is strict whenever p, q lie in different A_i . Since τ preserves distances on each A_i , equality holds within each square, so (L) is a condition on cross pairs.

Lemma 10.18 (Necessity). *If f is a δ -realization of τ , then $|\tau(p) - \tau(q)| \leq |p - q| + 2\delta$ for $p, q \in A$. In particular $\mathcal{R}_0(\tau) \neq \emptyset$ implies (L).*

Proof. The segment from p to q lies in Σ by convexity and has length $|p - q|$. Its image under f has the same length and joins $f(p)$ to $f(q)$, so $|f(p) - f(q)| \leq |p - q|$. Then $|\tau(p) - \tau(q)| \leq |f(p) - f(q)| + 2\delta$. ■

Theorem 10.19 (Realizability criterion). *The following are equivalent: (a) $\mathcal{R}_\delta(\tau) \neq \emptyset$ for every $\delta > 0$; (b) (L) holds. Moreover, when they hold, the δ -realizations may be chosen with image in the δ -neighbourhood of the convex hull $\text{conv } \tau(A)$.*

Proof. (a) $\Rightarrow$ (b) is Lemma 10.18 with $\delta \rightarrow 0$. For (b) $\Rightarrow$ (a), the hypothesis makes τ a 1-Lipschitz map from $A \subset \mathbb{R}^2$ to $\mathbb{R}^3$: within a square it preserves distances, and across squares it is nonexpanding by (L). By Kirszbraun's theorem (Appendix A, A.23) τ extends to a 1-Lipschitz map $F : \mathbb{R}^2 \rightarrow \mathbb{R}^3$. Let $K = \text{conv } \tau(A)$ and π_K the nearest-point projection to K , which is 1-Lipschitz and fixes $\tau(A)$; replace F by $\pi_K \circ F$, so $F(\mathbb{R}^2) \subset K$. Fix $c \in K$ and $s \in (0, 1)$ and put $G = (1 - s)F + sc$, a $(1 - s)$ -Lipschitz map into K with $|G - F| \leq s \text{ diam } K$. Mollify G by a kernel of radius η : the result G_η is smooth, still $(1 - s)$ -Lipschitz, takes values in K (mollification averages points of K), and $|G_\eta - G| \leq (1 - s)\eta$ on Σ . Its differential satisfies $\|dG_\eta\| \leq 1 - s < 1$, so G_η is a strictly short smooth map on a smooth compact convex neighbourhood $\tilde{\Sigma} \supset \Sigma$. By the Nash–Kuiper theorem in Gromov's form (A.24), for every $\rho > 0$ there is a C^1 isometric immersion $f : \tilde{\Sigma} \rightarrow \mathbb{R}^3$ with $|f - G_\eta| < \rho$. Restricted to Σ , f is a path isometry, and on A

$$|f - \tau| \leq \rho + \eta + s \text{ diam } K \leq \delta$$

for ρ, η, s chosen small in terms of δ . The image of f lies within ρ of K . ■

Exact attainment on A is a different matter, since it asks for a path isometry that agrees with τ on A and is strictly short nowhere that matters near ∂A ; it needs a relative form of the isometric immersion theorem and is not claimed here.

The criterion is a *certificate of satisfiability* in the sense of Chapter 8: the target τ together with the verification of (L) certifies that a hidden extension exists, and does so without exhibiting it.

Proposition 10.20 (Decidability). *If the A_i are polygons and $\tau|_{A_i}$ are affine, then (L) is a universal sentence of real closed fields and is decidable; the three-valued verdict **REALIZABLE** (L holds, Theorem 10.19), **NOT REALIZABLE** (L fails, Lemma 10.18) has no third value.*

Proof. Each A_i is a finite union of sets defined by linear inequalities, τ is given by rational affine maps, and (L) is $\forall p \forall q (p \in A \wedge q \in A \rightarrow |\tau p - \tau q|^2 \leq |p - q|^2)$, a first-order sentence of real arithmetic. Tarski's theorem (A.25) decides it. If (L) fails, then $|\tau(p) - \tau(q)| > |p - q|$ for some pair, and 2δ below the excess gives a contradiction with Lemma 10.18, so $\mathcal{R}_\delta(\tau) = \emptyset$ for small δ . ■

In the vocabulary of Chapter 12 this is a certified fragment in which preservation is decidable and the verdict **OUTSIDE THE CERTIFIED FRAGMENT** is never needed.

Example 10.21 (Three faces at a corner). Let $g > 0$ and take three unit squares in the plane,

$$A_1 = \{(-z - g, y)\}, \quad A_2 = \{(x, -z - g)\}, \quad A_3 = [0, 1]^2, \quad x, y, z \in [0, 1],$$

mapped to three faces of the unit cube meeting at the origin by $\tau(-z - g, y) = (0, y, z)$, $\tau(x, -z - g) = (x, 0, z)$, $\tau(x, y) = (x, y, 0)$. Each restriction preserves distances. For $p = (-z - g, y_1) \in A_1$ and $q = (x', -z' - g) \in A_2$ expanding the squares gives

$$|p - q|^2 - |\tau p - \tau q|^2 = 2x'(z + g) + 2y_1(z' + g) + 2zz' + 2g(z + z') + 2g^2 \geq 2g^2,$$

and for $p \in A_1, q = (x, y) \in A_3$,

$$|p - q|^2 - |\tau p - \tau q|^2 = 2xz + 2g(x + z) + g^2 \geq g^2,$$

with the same bound for A_2 and A_3 by symmetry. Hence (L[<]) holds with slack at least g^2 in squared distance, and by Theorem 10.19 the three faces can be exposed, up to any tolerance, by a path isometry of any sheet containing them, with the material between and around them accommodated in the convex hull of the three faces. The squares need not be adjacent in the sheet; the gap g is the price of their separation.

The remaining question is what the exterior data leave undetermined.

Theorem 10.22 (The exterior does not determine the interior). *Assume (L) and let $p_0 \in \Sigma \setminus A$. Put*

$$\Phi(p_0) = \{y \in \mathbb{R}^3 : |y - \tau(q)| < |p_0 - q| \text{ for all } q \in A\}.$$

Then $\Phi(p_0)$ is open and convex; for every $y \in \Phi(p_0)$ and $\delta > 0$ there is a δ -realization f with $|f(p_0) - y| \leq \delta$; and every δ -realization satisfies $|f(p_0) - \tau(q)| \leq |p_0 - q| + 2\delta$ for all $q \in A$, so that an exact realization has $f(p_0)$ in the closed set $\bigcap_q \overline{B}(\tau(q), |p_0 - q|) \supseteq \Phi(p_0)$. In particular, if $\Phi(p_0) \neq \emptyset$ then the value $f(p_0)$ is not a function of the exterior data: there are δ -realizations with the same exterior map up to δ and values at p_0 at arbitrary distance up to $\text{diam } \Phi(p_0)$.

Proof. Convexity holds because each set $\{y : |y - \tau(q)| < r\}$ is a convex ball. Openness: $y \mapsto \max_{q \in A} (|y - \tau(q)| - |p_0 - q|)$ is a continuous function on $\mathbb{R}^3$ (the maximum of a continuous function over the compact A), and $\Phi(p_0)$ is where it is negative. For the second statement, set $A' = A \cup \{p_0\}$ with $A_{k+1} = \{p_0\}$ and $\tau'(p_0) = y$. The restriction to a single point preserves distances, and (L) holds for τ' : pairs in A are covered by the hypothesis and pairs (p_0, q) by $y \in \Phi(p_0)$. Theorem 10.19 gives δ -realizations of τ' , which are δ -realizations of τ

with $|f(p_0) - y| \leq \delta$. The inequality $|f(p_0) - \tau(q)| \leq |p_0 - q| + 2\delta$ is Lemma 10.18 applied to the pair (p_0, q) for a δ -realization, using the path-isometry bound $|f(p_0) - f(q)| \leq |p_0 - q|$. For the non-function statement, pick $y_1 \neq y_2$ in $\Phi(p_0)$ and $\delta < |y_1 - y_2|/2$; the realizations f_1, f_2 so obtained have the same exterior up to δ and $f_1(p_0) \neq f_2(p_0)$. If $f(p_0)$ were a function of the exterior data it would be constant on the fibre, contradicting Proposition 1.12. ■

Example 10.23. In Example 10.21 take $p_0 = (-g, -g)$, which lies in the gap between the three squares, and $y = (0, 0, 0)$. For $q = (-z - g, y_1) \in A_1$ one has $|p_0 - q|^2 = z^2 + (g + y_1)^2 > z^2 + y_1^2 = |y - \tau(q)|^2$; for A_2 the same by symmetry; for $q = (x, y_1) \in A_3$, $|p_0 - q|^2 = (g + x)^2 + (g + y_1)^2 > x^2 + y_1^2$. So $y \in \Phi(p_0)$, and $\Phi(p_0)$ is a nonempty open set. The exterior data, the three exposed faces, leave the position of the sheet's gap point free in an open region of space.

By Theorem 10.22 the residue of the exterior map has positive dimension whenever there is slack, and the slack $\min_q (|p_0 - q| - |y - \tau(q)|)$ is its quantitative size. That is the same quantity as the margin of a Farkas certificate in Chapter 11: a certified region has room, and the room is what the certificate does not pin down. Conversely, the closed region $C(p_0) = \bigcap_q \overline{B}(\tau(q), |p_0 - q|)$ is exactly the set of positions attainable at p_0 , and by strict convexity of the balls it is either a single forced point or has nonempty interior; intermediate dimensions do not occur. (For two distinct points of $C(p_0)$, the midpoint satisfies $|m - \tau(q)| < |p_0 - q|$ for every q , since the Euclidean norm is strictly convex and two distinct points at equal distance from $\tau(q)$ cannot both lie on one ray from it; so the midpoint lies in the open set $\Phi(p_0)$.) Wherever $C(p_0)$ has more than one point, a task that depends on $f(p_0)$, such as locating a crease or deciding which layer lies above which, is therefore not served by the exterior map, by Proposition 1.12, and is served by any extraction that retains $f(p_0)$.

10.7 Declaring families in practice

A declaration should name tasks, not representations. Table 10.1 lists task families used in the remainder of the book, with the kernel each induces on a run of a proof search and the smallest standard artifact that is sufficient for it.

The table's entries are definitions of family names, not empirical claims, with one exception. The assertion that the conventional publication of a result, a statement together with a derivation, is not sufficient for the diagnosis and audit families is a consequence of Theorem 10.12 and a nonconstancy check on a single example, as in Example 10.5. Chapters 11 and 18 supply the examples for the remaining rows.

10.8 What this chapter fixes

Sufficiency is always sufficiency for something. (1) A representation serves a task if and only if the representation's kernel refines the task's kernel (Theorem 10.4). (2) Declaring a family fixes a minimal artifact and a closure of served tasks (Theorem 10.7). (3) The cost of any sufficient representation is bounded below by the number of classes

Family	Distinction it detects	Smallest standard sufficient artifact
Check	Derivation validity	Checked derivation
Cite	Statement and axiom footprint	Statement and footprint
Replay	Derivation under changed implementation	Derivation plus kernel version
Adapt to a nearby goal	Steps that depend on the goal's details	Derivation with dependency marking (Ch. 11)
Diagnose a failure	Which options were refuted, and by what	Run trace restricted to refusals (Ch. 13)
Audit scope	Which assumptions entered the correspondence claim	Obligation ledger (Ch. 9)

Table 10.1: Task families and the minimal artifacts that serve them. The last column states what is sufficient, not what is conventionally published.

of the family kernel (Theorem 10.10). (4) No non-injective representation serves all tasks (Theorem 10.12).

The chapters that follow ask how a representation fares under *change*, which requires more than a family of tasks over a fixed history space: it requires maps between history spaces, and a notion of which of them a representation survives. That is the subject of Chapter 11.

Exercise 10.1. Show that the set of $\mathcal{F}$ -sufficient kernels is closed under common refinement but, in general, not under the join in $\Pi(H)$. Give a four-element example.

Exercise 10.2. Prove that $\text{Serv}(\text{Rep}(\text{Serv}(\sigma))) = \text{Serv}(\sigma)$ and that $\text{Rep}(\text{Serv}(\text{Rep}(\mathcal{F}))) = \text{Rep}(\mathcal{F})$.

Exercise 10.3. In Example 10.5 let each A occur with exactly one d . Compute $\ker \sigma \cap \ker \mathcal{F}_{\text{ref}}$ and state the cost of the cheapest extension of σ to $\mathcal{F}_{\text{ref}}$ using Proposition 10.9.

Chapter 11

History, Stability and Repair

Chapter 10 fixed a history space and asked which tasks a representation serves. Reuse of a verified result is rarely a task on a fixed history space: a hypothesis is strengthened, a constant is changed, a lemma is renamed, a library is upgraded. Each is a map from one problem instance to another, and the question becomes what a certificate says about the instance it was not issued for. This chapter treats that question in three settings where the answer can be computed exactly: linear arithmetic, where a certificate has a geometric stability region; propositional clause sets, where a certificate has a combinatorial one; and local repair of proof terms, where the relevant structure is a metric. A relational version of the factorization criterion handles tasks with many correct outputs, and a locality theorem shows that no representation which identifies two distant proofs can support local repair.

11.1 Stability regions

Definition 11.1 (Instance space and region). Let $S = (\Phi, D, \text{Chk}, \text{Val})$ be a proof system and let $I \subseteq \Phi$ be a set of *instances*, regarded as the points of a space with some notion of neighbourhood. The *region* of a derivation d is $\text{Reg}(d) = \{\varphi \in I : \text{Chk}(d, \varphi)\}$. The *verdict region* is $\text{Reg}_{\text{Val}} = I \cap \text{Val}$. By soundness $\text{Reg}(d) \subseteq \text{Reg}_{\text{Val}}$, and by completeness $\text{Reg}_{\text{Val}} = \bigcup_d \text{Reg}(d)$.

A verdict is a fact about one instance. A certificate is a fact about a region containing that instance. The distinction matters because a change of the instance asks whether the new instance still lies in a region one already has, and it is only the certificate that supplies a region. The next sections compute these regions.

11.2 Linear arithmetic: certificates are half-spaces

Fix a matrix $A \in \mathbb{Q}^{m \times n}$ and regard the right-hand side $b \in \mathbb{R}^m$ as the instance. Write $Ax \leq b$ for the system, $Y = \{y \in \mathbb{R}^m : y \geq 0, y^\top A = 0\}$ for the cone of Farkas multipliers, and

$$U = \{b \in \mathbb{R}^m : Ax \leq b \text{ has no solution } x \in \mathbb{R}^n\}.$$

A *certificate for b* is a $y \in Y$ with $y^\top b < 0$. Farkas' lemma (Theorem 6.10, stated over $\mathbb{Q}$) holds over $\mathbb{R}$ as well, and says that $b \in U$ if and only if a certificate for b exists. For a

nonzero $y \in Y$ put

$$\text{Reg}(y) = \{b' \in \mathbb{R}^m : y^\top b' < 0\}.$$

This is the region of the derivation y in the sense of Definition 11.1, because checking y against b' consists of verifying $y \geq 0$, $y^\top A = 0$ (independent of b') and $y^\top b' < 0$.

Theorem 11.2 (Geometry of Farkas certificates).

- (a) For nonzero $y, y' \in Y$ one has $\text{Reg}(y) = \text{Reg}(y')$ if and only if $y' = cy$ for some $c > 0$.
- (b) $U = \bigcup_{y \in Y \setminus \{0\}} \text{Reg}(y)$. The set U is open, closed under multiplication by positive scalars, and downward closed: if $b \in U$ and $b' \leq b$ componentwise then $b' \in U$.
- (c) If $r_1, \dots, r_k$ generate the cone Y then $U = \bigcup_{i=1}^k \text{Reg}(r_i)$.

Proof. (a) Sufficiency of proportionality is clear. Conversely suppose y, y' nonzero in Y and not positively proportional. They are not negatively proportional either, since $y' = -cy$ with $c > 0$ would give $y' \leq 0$ and $y' \geq 0$, so $y' = 0$. Hence y, y' are linearly independent, the map $b \mapsto (y^\top b, y'^\top b)$ is onto $\mathbb{R}^2$, and some b has $y^\top b < 0 \leq y'^\top b$. Then $b \in \text{Reg}(y) \setminus \text{Reg}(y')$.

(b) The union formula is Farkas' lemma (the zero multiplier is excluded since $0^\top b < 0$ is false). Each $\text{Reg}(y)$ is open and closed under positive scaling, hence so is the union. For downward closure, if $y^\top b < 0$ and $b' \leq b$ then $y^\top b' \leq y^\top b < 0$ since $y \geq 0$.

(c) Every $y \in Y$ is a nonnegative combination $y = \sum_i \lambda_i r_i$. If $y^\top b < 0$ then $\sum_i \lambda_i (r_i^\top b) < 0$, so $r_i^\top b < 0$ for some i with $\lambda_i > 0$. By the Minkowski–Weyl theorem [CITED: SCHRIJVER, *THEORY OF LINEAR AND INTEGER PROGRAMMING*, SECTION 7.2] the polyhedral cone Y has a finite generating set. ■

Part (c) gives a finite description of the unsatisfiability region as a union of k open half-spaces, one per generator, while part (a) says that two certificates have the same region only if they differ by a positive scalar. In particular certificates that produce the same verdict at b typically have different regions.

Example 11.3 (One variable, three constraints). Take $n = 1, m = 3$ and the system $x \leq b_1, -x \leq b_2, x \leq b_3$, so $A = (1, -1, 1)^\top$. Then $Y = \{(a, a + c, c) : a, c \geq 0\}$ with generators $r_a = (1, 1, 0)$ and $r_b = (0, 1, 1)$, and

$$U = \{b_1 + b_2 < 0\} \cup \{b_2 + b_3 < 0\},$$

the instances for which the interval $[-b_2, \min(b_1, b_3)]$ is empty. At $b = (1, -3, 0)$ both generators are certificates: $r_a^\top b = -2$ and $r_b^\top b = -3$. Increase b_1 by 3 to $b' = (4, -3, 0)$. Then $r_a^\top b' = 1 \geq 0$, so r_a no longer certifies, while $r_b^\top b' = -3$ and the combination $(1, 2, 1)$ with $(1, 2, 1)^\top b' = -2$ still do. The system is still unsatisfiable; the certificate r_a has become useless and r_b has not. A verdict “unsatisfiable at b'' ” is identical in the two cases.

Margins

The perturbation tolerance of a certificate is exact. Measure perturbations in the supremum norm and write $\|y\|_1 = \sum_i |y_i|$ for the dual norm.

Proposition 11.4 (Margin of a certificate). Let $y \in Y \setminus \{0\}$ and $b \in \text{Reg}(y)$. For $\rho > 0$ the open ball $B_\rho(b) = \{b' : \|b' - b\|_\infty < \rho\}$ is contained in $\text{Reg}(y)$ if and only if $\rho \leq m(y, b)$, where

$$m(y, b) = \frac{-y^\top b}{\|y\|_1}.$$

Proof. For $b' \in B_\rho(b)$ write $b' = b + \delta$ with $\|\delta\|_\infty < \rho$. Then $y^\top \delta < \rho \|y\|_1$, with supremum $\rho \|y\|_1$ not attained. If $\rho \|y\|_1 \leq -y^\top b$ then $y^\top b' < y^\top b + \rho \|y\|_1 \leq 0$. If $\rho \|y\|_1 > -y^\top b$, take $\delta = \rho' \text{sign}(y)$ with $\rho' < \rho$ close enough to ρ that $\rho' \|y\|_1 > -y^\top b$; then $y^\top (b + \delta) > 0$ and $b + \delta \in B_\rho(b) \setminus \text{Reg}(y)$. ■

Theorem 11.5 (The best certificate attains the true radius). *For $b \in U$ let $R(b) = \sup\{\rho : B_\rho(b) \subseteq U\}$. Then $R(b) = \max\{m(y, b) : y \in Y \setminus \{0\}\}$, and the maximum is attained.*

Proof. Since $\text{Reg}(y) \subseteq U$, Proposition 11.4 gives $R(b) \geq m(y, b)$ for every y . The set $\{y \in Y : \|y\|_1 = 1\}$ is a compact polytope and $y \mapsto -y^\top b$ is continuous on it, so the maximum of $m(y, b)$ over $Y \setminus \{0\}$ (a scale-invariant function) is attained.

For the reverse inequality let $C = \mathbb{R}^m \setminus U = \{Ax + s : x \in \mathbb{R}^n, s \in \mathbb{R}_{\geq 0}^m\}$, the set of feasible right-hand sides. It is a finitely generated cone, hence closed and convex, and contains 0. Since $b \notin C$ and C is closed, $R = R(b)$ is finite or $+\infty$; if $R = +\infty$ then $U = \mathbb{R}^m$ and $0 \in C$ is a contradiction, so $R < \infty$. The open ball $B_R(b)$ is disjoint from C and convex. By the separation theorem for disjoint convex sets, one of which is open, there is $\lambda \neq 0$ with $\lambda^\top c \leq \lambda^\top b'$ for all $c \in C$ and $b' \in B_R(b)$ [CITED: ROCKAFELLAR, CONVEX ANALYSIS, THEOREM 11.3]. The left side is bounded above over the cone C only if $\lambda^\top A = 0$ (since $\pm Ax \in$ the generators) and $\lambda \leq 0$ (since $e_i \in C$); put $y = -\lambda \in Y \setminus \{0\}$. Then $\sup_C \lambda^\top c = 0$ (attained at $c = 0$) and

$$0 \leq \inf_{b' \in B_R(b)} \lambda^\top b' = \lambda^\top b - R \|\lambda\|_1 = -y^\top b - R \|y\|_1,$$

so $R \leq m(y, b)$. ■

The maximizing certificate is exactly the one that reports the largest uniform perturbation of the constants under which the unsatisfiability result remains valid. In Example 11.3, with $b = (1, -3, 0)$,

$$m(r_a, b) = \frac{2}{2} = 1, \quad m(r_b, b) = \frac{3}{2} = \frac{3}{2}, \quad m((1, 2, 1), b) = \frac{5}{4},$$

and writing a general multiplier as $(a, a + c, c)$ one gets $m = (2a + 3c)/(2a + 2c) \leq \frac{3}{2}$ with equality at $a = 0$. The distance from b to the feasible set, computed independently by enumeration over a fine grid of perturbations, is 1.5. The certificate r_a , which a solver might equally output, guarantees only a perturbation of size 1 in the supremum norm, and a perturbation of size 3 in b_1 destroys it (though not the verdict).

Theorem 11.6 (Stability family and minimal artifact). *Fix $b \in U$. Let H_b be the set of runs of any procedure that outputs a certificate for b , with $\varrho(h) \in Y \setminus \{0\}$ the certificate output. For $\delta \in \mathbb{R}^m$ let $S_\delta : H_b \rightarrow \{0, 1\}$ be $S_\delta(h) = 1$ iff $\varrho(h)$ certifies $b + \delta$. For the family $\mathcal{S} = \{S_\delta : \delta \in \mathbb{R}^m\}$, the minimal $\mathcal{S}$ -sufficient representation is the ray $y \mapsto \mathbb{R}_{>0} y$, equivalently the normalization $y/\|y\|_1$. The verdict “unsatisfiable” is not $\mathcal{S}$ -sufficient unless all certificates for b are positively proportional.*

Proof. $S_\delta(h) = 1$ iff $b + \delta \in \text{Reg}(\varrho(h))$. Hence $\ker \mathcal{S}$ identifies h, h' iff $\text{Reg}(\varrho(h)) = \text{Reg}(\varrho(h'))$ (the regions agree on every translate $b + \delta$, and δ ranges over all of $\mathbb{R}^m$), which by Theorem 11.2(a) holds iff the certificates are positively proportional. The conclusion follows from Theorem 10.4. The verdict is a constant function on H_b , whose kernel is all of H_b^2 . ■

The theorem identifies precisely what a certificate carries beyond a verdict: its direction, no more and no less, for the purpose of predicting survival under changes to the constants. If the family is enlarged to include perturbations of A the answer changes, since $y^\top A = 0$ is then no longer a fixed condition, and the region of a certificate becomes a more complicated algebraic set; nothing in the theorem extends to that case.

11.3 Clause sets: regions are up-sets

For propositional clause sets, the natural change is addition or removal of clauses. Fix a finite universe $\mathcal{U}$ of clauses over a fixed set of atoms. An instance is a subset $G \subseteq \mathcal{U}$, ordered by inclusion. The verdict region $\mathcal{U}$ of unsatisfiable subsets is an up-set. Write $\uparrow M = \{G : M \subseteq G\}$.

Lemma 11.7 (Unit propagation is monotone). *Let $G \subseteq G'$ be clause sets and T a set of literals. If some maximal unit propagation sequence from T on G ends in a conflict, then every maximal unit propagation sequence from T on G' ends in a conflict.*

Proof. Let $T = T_0 \subset T_1 \subset \dots \subset T_k$ be a propagation sequence on G ending in a conflict, where step $i+1$ adds l_{i+1} using $C_{i+1} \cup \{l_{i+1}\} \in G$ with $\overline{C_{i+1}} \subseteq T_i$ and the final trail falsifies a clause of G . Let T' be the final trail of a maximal propagation sequence on G' from T and suppose it contains no conflict. We show by induction that $T_i \subseteq T'$. The case $i = 0$ is trivial. If $T_i \subseteq T'$, the clause $C_{i+1} \cup \{l_{i+1}\} \in G'$ has all literals but l_{i+1} false under T' . If $\overline{l_{i+1}} \in T'$ the clause is falsified, a conflict. If l_{i+1} is unassigned the clause is unit and propagation could continue, contradicting maximality. So $l_{i+1} \in T'$. Finally the clause of G falsified by $T_k \subseteq T'$ is a clause of G' falsified by T' , a conflict. This contradicts the assumption. ■

Proposition 11.8 (RUP refutations are monotone). *If d is a RUP refutation of G and $G \subseteq G'$ then d is a RUP refutation of G' . Consequently $\text{Reg}(d) = \{G' : \text{Chk}_{\text{RUP}}(d, G')\}$ is an up-set.*

Proof. For each line C_i , the refutation condition is that unit propagation from $\overline{C_i}$ on $G \cup \{C_1, \dots, C_{i-1}\}$ ends in a conflict. This clause set is contained in $G' \cup \{C_1, \dots, C_{i-1}\}$, so Lemma 11.7 applies. ■

Definition 11.9 (Used clauses). For a RUP refutation d of G and for each line, fix one successful propagation trace; let $\text{use}(d) \subseteq G$ be the set of input clauses (not lemma lines) that occur as reasons in these chosen traces together with the clauses falsified at their ends.

By construction d is a RUP refutation of $\text{use}(d)$, so $\uparrow \text{use}(d) \subseteq \text{Reg}(d)$ by Proposition 11.8. The set $\text{use}(d)$ is a certified unsatisfiable core.

Theorem 11.10 (Verdict region is a union of certificate regions). *Let $\mathcal{M}$ be the set of minimal unsatisfiable subsets of $\mathcal{U}$. Then*

$$\mathcal{U} = \bigcup_{M \in \mathcal{M}} \uparrow M = \bigcup_d \text{Reg}(d),$$

the second union over all RUP refutations d , and for each $M \in \mathcal{M}$ there is a refutation d_M with $\text{use}(d_M) = M$.

Proof. Every unsatisfiable G contains a minimal unsatisfiable subset because $\mathcal{U}$ is finite, and each $\uparrow M \subseteq \mathcal{U}$. The second equality is Theorem 5.13 together with Proposition 11.8. For $M \in \mathcal{M}$ take a refutation d_M of M , which exists by completeness. Then $\text{use}(d_M) \subseteq M$ is unsatisfiable by soundness of d_M as a refutation of $\text{use}(d_M)$, so equals M by minimality. ■

Theorem 11.10 is the combinatorial counterpart of Theorem 11.2(c): the verdict region is covered by finitely many certificate regions, each a principal up-set or a half-space, and an individual certificate covers a proper part of it.

Example 11.11 (Two refutations, two regions). Let a, b, e be atoms and $F_4 = \{a \vee b, a \vee \neg b, \neg a \vee b, \neg a \vee \neg b\}$. Put $G = F_4 \cup \{e, \neg e\}$ and $G' = F_4 \cup \{\neg e\}$. The one-line derivation $d_1 = (\square)$ is a RUP refutation of G : propagation from the empty trail derives e and then falsifies $\neg e$. It is not a refutation of G' , which has no unit clause that propagates to a conflict, though G' is unsatisfiable. The derivation $d_2 = (a, \square)$ is a RUP refutation of F_4 . From the trail $\{\neg a\}$ the clause $a \vee b$ propagates b , and $a \vee \neg b$ is then falsified, so a is RUP. With a added, propagation from the empty trail uses a to derive b through $\neg a \vee b$ and falsifies $\neg a \vee \neg b$, so $\square$ is RUP. Therefore $\text{Reg}(d_1) \supseteq \uparrow \{e, \neg e\}$ and $\text{Reg}(d_2) \supseteq \uparrow F_4$; also $G \in \text{Reg}(d_1) \cap \text{Reg}(d_2)$ and $G' \in \text{Reg}(d_2) \setminus \text{Reg}(d_1)$. Both derivations certify the same fact about G .

The check of d_1 and d_2 against G and G' in this example was also run mechanically with a short unit-propagation script.

A solver that produces d_1 has done nothing wrong, and the solver's choice of d_1 over d_2 is invisible in the verdict. If a clause e is removed later because the corresponding hypothesis is weakened, the proof d_1 fails while the verdict, for the weaker problem, persists. This is the clause-set analogue of Example 11.3: *which* certificate was produced is a fact about the history, and the stability of the result under change depends on it.

11.4 Repair as a relation

The tasks of the preceding chapters were functions: a history determines the answer. Repair is different. After a change there are usually many correct outputs, and the requirement is that a representation suffices to produce *one*. The factorization criterion must be extended from functions to relations.

Definition 11.12 (Repair relation, repair function). Let H be a history space, Δ a set of changes, O a set of outputs, and $\text{Good} \subseteq H \times \Delta \times O$ a relation, read as “ o is an acceptable repair of the history h under the change δ ”. For a representation $\sigma : H \rightarrow R$, a σ -repair function is $r : R \times \Delta \rightarrow O$ with $(h, \delta, r(\sigma(h), \delta)) \in \text{Good}$ for all $h \in H, \delta \in \Delta$.

Theorem 11.13 (Factorization for relations). A σ -repair function exists if and only if for every fibre $F = \sigma^{-1}(s)$ with $s \in \sigma(H)$ and every $\delta \in \Delta$,

$$\bigcap_{h \in F} \text{Good}(h, \delta) \neq \emptyset, \quad \text{where } \text{Good}(h, \delta) = \{o : (h, \delta, o) \in \text{Good}\}.$$

Proof. If r exists then $r(s, \delta)$ lies in every $\text{Good}(h, \delta)$, $h \in F$. Conversely, choose for each pair (s, δ) with $s \in \sigma(H)$ an element of the nonempty intersection, and an arbitrary element of O if $s \notin \sigma(H)$ (assuming $O \neq \emptyset$). This defines r . ■

When each $\text{Good}(h, \delta)$ is a singleton $\{f_\delta(h)\}$ the condition says that f_δ is constant on fibres, which is Proposition 1.12. The relational form is needed because it captures a requirement not expressible by functions: that a repair be *local*.

Definition 11.14 (Locality). Let (O, dist) be a metric space, for instance proof terms under a tree edit distance, and let $\pi : H \rightarrow O$ be the artifact that each history produced. Let $\text{Cor}(\delta) \subseteq O$ be the set of correct artifacts after the change δ . For $k \geq 0$ define

$$\text{Good}_k(h, \delta) = \{o \in \text{Cor}(\delta) : \text{dist}(o, \pi(h)) \leq k\},$$

the k -local repairs of h under δ .

Theorem 11.15 (Locality needs history). Let $\sigma : H \rightarrow R$ and let $h, h' \in H$ satisfy $\sigma(h) = \sigma(h')$ and $\text{dist}(\pi(h), \pi(h')) > 2k$. Then for every $\delta \in \Delta$ the intersection $\text{Good}_k(h, \delta) \cap \text{Good}_k(h', \delta)$ is empty. Consequently no σ -repair function exists for Good_k on a history space containing both, even when each of h and h' individually has a k -local repair under every $\delta \in \Delta$.

Proof. If o lay in both sets then $\text{dist}(\pi(h), \pi(h')) \leq \text{dist}(\pi(h), o) + \text{dist}(o, \pi(h')) \leq 2k$, contradicting the hypothesis. The consequence follows from Theorem 11.13 with $F \ni h, h'$. ■

Applied to the proof-irrelevant representation of Definition 10.14, with $\sigma(P, p) = P$ and $\pi(P, p) = p$, the theorem says: if a proposition has two proofs more than $2k$ edits apart, there is no procedure which, given only the proposition and a change, returns a repair within k edits of whichever proof was originally given. This is the formal sense in which one cannot patch a result from its statement, and it holds however the change δ is chosen. It remains consistent with the fact that a procedure can always *reprove* the changed statement from scratch, which is Good_∞ .

Remark 11.16 (The same theorem for certificates). In Example 11.3 let σ be the verdict, let $\pi(h)$ be the output certificate normalized to $\|y\|_1 = 1$, so $\pi(h_a) = (\frac{1}{2}, \frac{1}{2}, 0)$ and $\pi(h_b) = (0, \frac{1}{2}, \frac{1}{2})$, and take the Euclidean distance, which is $1/\sqrt{2}$. For $k < 1/(2\sqrt{2})$ Theorem 11.15 applies: no procedure that sees only the verdict can return a certificate within distance k of whichever certificate was originally output. The numerical case is thus an instance of the general statement.

11.5 Scripts are functions, terms are points

Proof assistants distinguish a *script*, a sequence of tactic invocations, from the *term* it elaborates to (Chapter 7). The distinction is, in the terminology of this chapter, the distinction between a family of certificates indexed by goals and one member of the family.

Definition 11.17 (Script). Let $\mathcal{G}$ be a set of goals. A *script* is a partial computable function $s : \mathcal{G} \rightarrow D$. It is *correct at* g if $s(g)$ is defined and $\text{Chk}(s(g), g)$. Its *region* is $\text{Reg}(s) = \{g \in \mathcal{G} : s \text{ is correct at } g\}$.

Proposition 11.18 (Terms are points). Suppose the checking relation has unique types: $\text{Chk}(t, g)$ and $\text{Chk}(t, g')$ imply $g \equiv g'$, where $\equiv$ is the definitional equality of the calculus. Then for every term t , $\text{Reg}(t) \subseteq [g]_\equiv$ for any $g \in \text{Reg}(t)$, that is, the region of a term is contained in a single definitional class of goals, while the region of a script can be infinite and can contain goals pairwise not definitionally equal.

Proof. The first claim is the hypothesis. For the second, let $\mathcal{G} = \{n + 0 = n : n \in \mathbb{N}\}$ with addition defined by recursion on the second argument, so each $n + 0 = n$ holds by reflexivity, and let $s(n + 0 = n) = \text{rfl}_n$ where rfl_n is the reflexivity proof at n . The script is correct at every goal, so $\text{Reg}(s) = \mathcal{G}$ is infinite, while the goals $n + 0 = n$ for distinct numerals n are not definitionally equal. ■

A script therefore carries a *program* for producing certificates and its region is the domain on which that program is correct. The kernel stores one value of the program. The two have different stability properties: a term is stable under changes to the surrounding context that preserve its type, and a script is stable under changes that preserve its elaboration, which depend on the version of the tactic library and not on the goal alone. These are different kernels on different history spaces, and neither contains the other. A statement that a proof is “robust” is therefore incomplete until it says on which of these spaces the perturbations are taken.

11.6 What this chapter fixes

A certificate is a region, a verdict is a point. (1) In linear arithmetic the region of a Farkas certificate is an open half-space, determined by the certificate’s direction, and the verdict region is a finite union of such half-spaces (Theorems 11.2 and 11.6). (2) A certificate’s perturbation margin is explicit and the best certificate attains the true radius, but a solver need not output it (Proposition 11.4, Theorem 11.5). (3) In clause sets, RUP refutations are monotone under adding clauses, the verdict region is a union of principal up-sets, and verdict-equal refutations can have incomparable regions (Proposition 11.8, Theorem 11.10, Example 11.11). (4) Repair is a relation, and a σ -repair function exists iff the acceptable repairs of the histories in each fibre have a common element (Theorem 11.13); local repair is impossible from any representation that identifies two distant artifacts (Theorem 11.15).

The results above are about what a result *survives*. The next chapter turns to a result’s dependents, the other direction of change: what is inherited by a theorem that uses a lemma, and what is not. There the relevant structure is a typed dependency graph, and the interface lemma will say that proof irrelevance makes the body of a lemma invisible to dependents.

Exercise 11.1. In Example 11.3 compute $R(b)$ for $b = (1, -3, 5)$ and identify which generator attains it.

Exercise 11.2. Show that the verdict region U of Theorem 11.2 is not in general convex. Give an example with $m = 3$.

Exercise 11.3. Show that the maximum in Theorem 11.5 is attained at a normalized generator (an extreme ray) of Y . (Hint: on the polytope $\{y \in Y : \|y\|_1 = 1\}$ the function $y \mapsto -y^\top b$ is linear.)

Exercise 11.4. In Example 11.11, compute $\text{use}(d_1)$ and $\text{use}(d_2)$ and show $\uparrow \text{use}(d_1) \cap \uparrow \text{use}(d_2) = \uparrow G$.

Chapter 12

Inheritance along Transformations

Results are not used where they were proved. A theorem is invoked inside a larger development; a formula is simplified before it is handed to a solver; a formal statement is read as a statement about a physical system. Each of these is a transformation, and each raises the same question: what does the transformed object inherit from the verified original? This chapter separates three things that are routinely conflated under the word “inherit”. *Validity* is whether a checker accepts the transformed object. *Trust* is which principles the acceptance rests on. *Correspondence* is whether the transformed object still says what the original was meant to say. The three have different logics. Validity is decidable and governed by the structure of the checker (Sections 12.3 and 12.4); correspondence is undecidable in general and can be certified only on declared fragments (Sections 12.1 and 12.2); trust propagates along a different graph from validity (Proposition 12.17).

12.1 Correspondence under rewriting is undecidable

Let Φ_{FO} be the set of sentences of first-order logic over a fixed countable signature containing at least one binary relation symbol, and let $\varphi \equiv \psi$ mean that φ and ψ have the same models.

Definition 12.1 (Preservation predicate). A *transformation* is a computable function $R : \Phi_{\text{FO}} \rightarrow \Phi_{\text{FO}}$ (a rewrite pass, given as a program). The *preservation predicate* is

$$\text{Pres}(R, \varphi) \iff \varphi \equiv R(\varphi).$$

A transformation that fails Pres at φ changes the meaning of φ ; one that passes preserves it. The predicate is the simplest formal shadow of the correspondence predicate of Chapter 9: if a formal result about φ is to speak for the transformed $R(\varphi)$, then $\text{Pres}(R, \varphi)$ is required.

Theorem 12.2 (Undecidability of preservation). *There is a fixed computable transformation $R_{\top}$ for which $\{\varphi : \text{Pres}(R_{\top}, \varphi)\}$ is undecidable. Hence there is no algorithm that, given a transformation (as a program) and a sentence, decides Pres .*

Proof. Let $R_{\top}(\varphi) = \top$, the constantly true sentence $\forall x (x = x)$. Then $\text{Pres}(R_{\top}, \varphi)$ holds iff $\varphi \equiv \top$, that is, iff φ is valid. By Church’s theorem (Chapter 2, Proposition 2.10), validity of first-order sentences over a signature with a binary relation symbol is undecidable. ■

Remark 12.3. The statement is about transformations arbitrary enough to include constant maps; it is deliberately weak. The usual reading in practice is stronger: for transformations that are *intended* to be equivalences, such as a simplifier, whether a given application is faithful is again undecidable, as one sees by embedding the above reduction in a rule set that contains a rule with a hard-to-check side condition. By Rice's theorem [CITED: ROGERS, *THEORY OF RECURSIVE FUNCTIONS AND EFFECTIVE COMPUTABILITY*, SECTION 14.2] the property “ R preserves $\equiv$ at every φ ” of a program R is also undecidable.

Corollary 12.4 (No total sound verdict). *There is no computable function $Vd(R, \varphi) \in \{PRESERVED, NOT_PRESERVED\}$ that is correct (that is, returns $PRESERVED$ only if $Pres(R, \varphi)$ and $NOT_PRESERVED$ only if $\neg Pres(R, \varphi)$) on all arguments.*

Proof. Such a function would decide $Pres$, contradicting Theorem 12.2. ■

The corollary is the reason that a third value is not a convenience but a necessity. Any honest, computable report of preservation must be allowed to say that it does not know.

12.2 Certified fragments and a three-valued verdict

Definition 12.5 (Certified fragment). A *certified fragment* is a triple $(\mathcal{F}, \mathcal{R}, Ev)$ where $\mathcal{F} \subseteq \Phi_{FO}$ is a decidable set of sentences, $\mathcal{R}$ is a decidable set of transformations (given as programs), and Ev is a computable function that, for $R \in \mathcal{R}$ and $\varphi \in \mathcal{F}$ with $R(\varphi) \in \mathcal{F}$, returns either a *preservation certificate* or a *separation certificate*, such that a decidable relation $ChkP$ accepts a preservation certificate for (R, φ) only if $\varphi \equiv R(\varphi)$, and a decidable relation $ChkN$ accepts a separation certificate only if $\varphi \not\equiv R(\varphi)$. The fragment is *complete* if Ev is total on such arguments.

The verdict function $Vd(R, \varphi)$ attached to the fragment has three values:

- $PRESERVED$, if $R \in \mathcal{R}$, $\varphi, R(\varphi) \in \mathcal{F}$ and $Ev(R, \varphi)$ is a preservation certificate accepted by $ChkP$;
- $NOT_PRESERVED_UNDER_R$, if the same hypotheses hold with a separation certificate accepted by $ChkN$;
- $OUTSIDE_CERTIFIED_FRAGMENT$ otherwise.

Theorem 12.6 (Soundness and non-collapse of the verdict). *For every certified fragment:*

- (i) $Vd(R, \varphi) = PRESERVED$ implies $Pres(R, \varphi)$.
- (ii) $Vd(R, \varphi) = NOT_PRESERVED_UNDER_R$ implies $\neg Pres(R, \varphi)$.
- (iii) $Vd(R, \varphi) = OUTSIDE_CERTIFIED_FRAGMENT$ implies nothing about $Pres(R, \varphi)$: there are fragments and arguments with this value for which $Pres$ holds and others for which it fails.

If the fragment is complete then Vd is two-valued on $\mathcal{R} \times \mathcal{F}$ restricted to $R(\varphi) \in \mathcal{F}$.

Proof. (i) and (ii) are the soundness of $ChkP$ and $ChkN$. For (iii), take the fragment with $\mathcal{F} = \emptyset$: it is a certified fragment vacuously, Vd is everywhere $OUTSIDE_CERTIFIED_FRAGMENT$, and the argument $(R_{\top}, \top)$ has $Pres$ true while $(R_{\top}, \perp)$ has it false. The last statement is the definition of completeness. ■

Theorem 12.7 (Completeness fails for full first-order logic). *Let $(\mathcal{F}, \mathcal{R}, \text{Ev})$ be a certified fragment with $R_\top \in \mathcal{R}$ and $\mathcal{F} = \Phi_{\text{FO}}$. Then $\text{Ev}(R_\top, \cdot)$ fails to return a certificate on infinitely many sentences. In particular the fragment is not complete, and the verdict takes the third value infinitely often.*

Proof. Suppose $\text{Ev}(R_\top, \varphi)$ returned a certificate for all but finitely many φ . Validity of φ could then be decided by consulting a finite table for the exceptions and otherwise running Ev and checking the certificate with ChkP (valid) or ChkN (not valid). This contradicts Theorem 12.2. ■

Complete fragments exist when the underlying equivalence problem is decidable. Two examples follow, one deliberately minimal and one of practical interest.

Example 12.8 (Conjunctions modulo associativity, commutativity and idempotence). Let $\mathcal{F}$ be the set of finite nonempty conjunctions $p_{i_1} \wedge \cdots \wedge p_{i_r}$ of propositional atoms, with repetitions and order allowed, regarded as formulas over the atoms. Each is equivalent to the conjunction over its *atom set* $\text{at}(\varphi)$, by associativity, commutativity and idempotence of $\wedge$. Let $\mathcal{R}$ be the computable maps R with $R(\mathcal{F}) \subseteq \mathcal{F}$.

Claim. $\varphi \equiv \psi$ iff $\text{at}(\varphi) = \text{at}(\psi)$.

If the sets are equal the formulas have the same models. Conversely if $p \in \text{at}(\varphi) \setminus \text{at}(\psi)$ then the assignment that makes the atoms of ψ true and all others false satisfies ψ but not φ ; symmetrically for $\text{at}(\psi) \setminus \text{at}(\varphi)$.

Take as preservation certificate the equality of the two atom sets, and as separation certificate an atom lying in exactly one of them together with the assignment above. Both are checked by a terminating computation, and Ev is total, so the fragment is complete. For $R(\varphi) = \varphi \wedge \varphi$ the verdict is PRESERVED; for $R(\varphi) = \varphi \wedge q$ with $q \notin \text{at}(\varphi)$ it is NOT_PRESERVED_UNDER_R; for a φ containing a disjunction it is OUTSIDE_CERTIFIED_FRAGMENT, whatever R does.

Example 12.9 (Propositional formulas with refutation certificates). Let $\mathcal{F}$ be all propositional formulas and $\mathcal{R}$ all computable maps preserving $\mathcal{F}$. The equivalence $\varphi \equiv R(\varphi)$ holds iff the formula $\neg(\varphi \leftrightarrow R(\varphi))$ is unsatisfiable, a SAT instance via the Tseitin transformation (equisatisfiable [CITED: TSEITIN 1968]). A preservation certificate is a DRAT refutation of the Tseitin clause set (Chapter 5), a separation certificate is a satisfying assignment, which is checked by evaluating both formulas. The fragment is complete, since SAT is decidable, although the certificates may be exponentially large (Theorem 5.9).

The fragment is declared before the transformation is applied, and the verdict is per pair (R, φ) , not per transformation: the same pass R is PRESERVED at some formulas and NOT_PRESERVED_UNDER_R at others. Consistent with the discipline of Chapter 1, an OUTSIDE_CERTIFIED_FRAGMENT verdict is never reported as a failure and never as a success.

12.3 Proof terms inherit along definitional equality

Validity is inherited by a transformation of the *statement* exactly along the equivalence the checker already identifies. Let $\mathcal{R}$ be a convergent presentation of the conversion relation $\equiv$ on the types of a calculus, as in Chapter 4, so that by Corollary 4.4 $A \equiv B$ iff $\text{NF}(A) =$

$\text{NF}(B)$. Suppose, as in Proposition 11.18, that each term t has a type $\text{ty}(t)$ unique up to $\equiv$ and that $\text{Chk}(t, P)$ holds iff $\text{ty}(t) \equiv P$.

Proposition 12.10 (Region of a term). $\text{Reg}(t) = \{P : \text{NF}(P) = \text{NF}(\text{ty}(t))\}$. In particular a proof term of P is a proof term of P' if and only if $\text{NF}(P) = \text{NF}(P')$, and not merely if $P \leftrightarrow P'$ is provable.

Proof. Immediate from the standing hypotheses and Corollary 4.4. If $P \leftrightarrow P'$ is provable but $\text{NF}(P) \neq \text{NF}(P')$ then $\text{Chk}(t, P)$ and $\text{Chk}(t, P')$ cannot both hold. ■

Example 12.11 (Addition by recursion on the second argument). Let $\mathcal{R}'_+$ consist of $\text{plus}(x, 0) \rightarrow x$ and $\text{plus}(x, s(y)) \rightarrow s(\text{plus}(x, y))$. With the interpretation $[0] = 1$, $[s](x) = x + 1$, $[\text{plus}](x, y) = x + 2y$ both rules strictly decrease: $[\text{plus}(x, 0)] = x + 2 > x$, and $[\text{plus}(x, s(y))] = x + 2y + 2 > x + 2y + 1 = [s(\text{plus}(x, y))]$. The left-hand sides do not overlap, so there are no critical pairs and $\mathcal{R}'_+$ is convergent. For a variable n :

- $\text{plus}(n, 0) \rightarrow n$, so $\text{NF}(\text{plus}(n, 0)) = n = \text{NF}(n)$ and $\text{plus}(n, 0) \equiv n$;
- $\text{plus}(0, n)$ matches neither rule since the second argument is a variable, so it is a normal form distinct from n , and $\text{plus}(0, n) \not\equiv n$.

Consequently the reflexivity proof rfl_n has type $n = n$, hence also type $\text{plus}(n, 0) = n$ by conversion, and by Proposition 12.10 is *not* a proof of $\text{plus}(0, n) = n$, for which an inductive argument and hence a different term is required. For each numeral instance $n = s^k(0)$ the term rfl does check, since $\text{plus}(0, s^k(0)) \rightarrow^* s^k(0)$. The two statements agree on every closed instance and differ on the open one, and what the checker certifies depends on that difference.

Example 12.11 is the type-theoretic statement of what Chapter 9 called *scope*: a transformation of a statement inherits a verified proof only along a relation that the checker already treats as equality, and any other propositionally valid transformation costs a new proof and a new history.

12.4 Typed dependency edges and the interface lemma

A development is an environment Γ , a finite list of constants, each an *axiom* (a type only), a *definition* (a type and a value, transparent to conversion) or a *theorem* (a type and a value, opaque to conversion). This is the structure of Chapter 7.

Convention 12.12 (Kernel discipline). The checker is assumed to satisfy the following design properties, which hold for the kernels considered in Chapter 7 and are not proved here:

- (K1) Checking a constant c consults, for each constant e occurring in the derivation, the type of e , and the value of e only if e is a definition that conversion unfolds.
- (K2) Conversion never unfolds a theorem.

Definition 12.13 (Typed dependency edges). For constants c, e of Γ :

- $c \rightarrow_{\text{sem}} e$ if e occurs in the *type* of c (a *semantic edge*);
- $c \rightarrow_{\text{int}} e$ if e is a theorem occurring in the *value* of c (an *interface edge*);
- $c \rightarrow_{\text{prf}} e$ if e is a non-theorem constant occurring in the value of c (a *proof-term edge*).

The *flat graph* is the union of the three relations with their labels forgotten.

Definition 12.14 (Consulted set). The *consulted set* $\text{Cons}(c)$ is the smallest set of constants such that

- (a) if $c \rightarrow e$ for any of the three edge kinds then $e \in \text{Cons}(c)$;
- (b) if $e \in \text{Cons}(c)$ and $e \rightarrow_{\text{sem}} e'$ then $e' \in \text{Cons}(c)$;
- (c) if $e \in \text{Cons}(c)$ is a *definition* and $e \rightarrow_{\text{prf}} e'$ or $e \rightarrow_{\text{int}} e'$ then $e' \in \text{Cons}(c)$.

In words: from the declaration being checked, any dependency is followed; from a consulted constant the type is always read, and the value is read only for definitions. The value of a theorem, once reached, is not read.

Lemma 12.15 (Interface lemma). Assume Convention 12.12. Let T be a theorem of Γ with type φ and value v , and let Γ' be obtained by replacing v with a term v' such that $T : \varphi := v'$ checks in the prefix of Γ before T . Then every constant $c \neq T$ that checks in Γ also checks in Γ' .

Proof. Here “ c checks in Γ ” means that the typing derivation of the declaration c is valid relative to the constants preceding it. A constant before T is unchanged and cannot mention T . For a constant c after T , a derivation of c in Γ uses, for each occurrence of T , only the type φ by (K1) and (K2), since T is a theorem and is never unfolded; and for any other constant e it uses data identical in Γ and Γ' (types always, and values of definitions, which are unchanged). The same derivation is therefore a derivation in Γ' . ■

Theorem 12.16 (Typed blast radius). Assume Convention 12.12. Let Γ' be obtained from Γ by replacing the value of the constant D by one that checks at D 's type. Let $c \neq D$ be a constant after D that checks in Γ .

- (a) If D is a theorem then c checks in Γ' .
- (b) If D is a definition and $D \notin \text{Cons}(c)$ then c checks in Γ' .
- (c) If D is a definition and $D \in \text{Cons}(c)$, then c need not check in Γ' .

Proof. (a) is Lemma 12.15. (b) By Definition 12.14 and (K1), the derivation of c reads the value only of definitions in $\text{Cons}(c)$ and the type of constants in $\text{Cons}(c)$, and c 's own data; D is not among them, so the derivation transfers unchanged. (c) Let $D : \mathbb{N} := 0$ and $c : D = 0 := \text{rfl}$, checks since D unfolds to 0. Replace the value of D by 1. Then c no longer type-checks, since $1 \neq 0$. ■

Theorem 12.16 gives a *prediction rule* for mutations of a verified development: replacing the value of a theorem breaks nothing, replacing the value of a definition can break only constants for which the definition is consulted, and in the flat graph the second set is overestimated by paths through theorem bodies and the first is overestimated entirely. A prediction made with the flat graph that treats a path through a theorem body as a dependency will predict breakage that does not occur; a prediction that treats an edge to a definition in a *type* as opaque will miss breakage that does. Chapter 18 records the preregistered mutation matrix in which two mutants, M09 and M16, had outcomes that contradicted the prediction of the flat model; the typed edges above are the amendment, and the two mutants are retained there as falsified under the earlier model, not corrected after the fact.

Trust does not follow the same edges

Replacing a theorem's value does not alter validity of dependents (Lemma 12.15). It can alter what they rest on.

Proposition 12.17 (Footprint follows value edges through theorems). *Let $\text{cl}(c)$ be the dependency closure of Chapter 7, taken in the flat graph, and $\text{Ax}(c)$ its axiom footprint. If $c \rightarrow_{\text{int}} T$ then $\text{Ax}(T) \subseteq \text{Ax}(c)$. Under the replacement of Lemma 12.15, the check of c is unchanged and $\text{Ax}(c)$ may change, in particular so that it contains sorryAx when v' does.*

Proof. The inclusion holds since $\text{cl}(c) \supseteq \text{cl}(T)$ when $T \in \text{cl}(c)$, and cl includes values by definition. For the second statement let T be a theorem proved by an elementary argument, and let v' be a proof of the same statement containing sorryAx . Lemma 12.15 gives that c checks in Γ' whenever it did in Γ , and $\text{cl}(c)$ in Γ' includes sorryAx . ■

Hence a successful kernel check of c after replacing the body of T is not evidence that the replacement is harmless: it certifies validity only relative to the footprint, and the footprint is the object that must be audited. Proposition 7.8 is the corresponding observation for a single declaration; here it is stated for the dependents. In the language of Chapter 10: the family of tasks “does c check” factors through the statement of each theorem it uses, which is the proof-irrelevant representation (Proposition 10.15(i)), and the family of tasks “on what does c rest” does not (Proposition 10.15(iii)).

12.5 What this chapter fixes

Validity, trust and correspondence inherit along different relations. (1) Preservation of meaning under a transformation is undecidable in general, so any computable report must have a third value, and a failure to certify must not be reported as a certified failure (Theorem 12.2, Corollary 12.4, Theorem 12.6). (2) A proof term inherits to a transformed statement exactly along definitional equality (Proposition 12.10). (3) The consulted set of a constant is computed from typed dependency edges; the body of a theorem is invisible to dependents for validity but visible for trust (Theorem 12.16, Proposition 12.17).

This closes the treatment of results as static objects subjected to change. The next part turns to the process that produces them and asks which of its states the preceding structure applies to.

Exercise 12.1. Show that in Example 12.8 the preservation certificate for (R, φ) may be taken to be a pair of lists of atom positions witnessing mutual inclusion of the atom sets, and describe a linear-time check.

Exercise 12.2. Give a convergent presentation of the conversion relation in which $\text{plus}(0, n) \equiv n$ and $\text{plus}(n, 0) \equiv n$ both hold, and show that its normal forms of closed terms agree with those of $\mathcal{R}'_+$.

Exercise 12.3. Prove, assuming Convention 12.12, that $\text{Cons}(c) \subseteq \text{cl}(c)$ and that the inclusion can be strict even when c is well-formed.

Exercise 12.4. Replace the value of a definition D by one that is *propositionally* but not definitionally equal to the old. Show by an example of the kind in the proof of Theorem 12.16(c) that c with $D \in \text{Cons}(c)$ can fail, and state a sufficient condition on the new value under which no c fails.

Part IV

Admissibility of Search

Chapter 13

Search States as Worlds

Part III asked what a finished result retains. Part IV asks the corresponding question of the process that produces results, at the level of individual steps. A search procedure repeatedly selects an item, derives new items from it, discards items it judges unnecessary, and sometimes replaces its whole state by a smaller one. Each of these is an operation on a state, and the correctness of a prover lives in which of the operations preserve which of the state's distinctions. This chapter gives the operations names and the state a precise structure, and proves four things: the option space of a search is changed only by refusal (Proposition 13.5); a refusal is lossless exactly when the refused item is redundant (Proposition 13.5(c)); the extensional state is the minimal representation that supports every future of the search (Theorem 13.8); and proofs are not recoverable from that state (Proposition 13.11).

The vocabulary of four operations, *pop*, *refuse*, *bind* and *collapse*, follows the author's earlier work on binding and on disposition calculi [35, 36]. Nothing from those works is assumed. Every definition is restated for search, and every result below is proved from the definitions given here.

13.1 Closure systems and option spaces

Fix a set $\mathfrak{F}$ of items, for instance clauses, and an inference system I over $\mathfrak{F}$ in the sense of Definition 2.1.

Definition 13.1 (Option closure). A *closure operator* on $2^{\mathfrak{F}}$ is a map Cn that is extensive ($S \subseteq Cn(S)$), monotone ($S \subseteq S'$ implies $Cn(S) \subseteq Cn(S')$) and idempotent ($Cn(Cn(S)) = Cn(S)$). It *respects* I if $Cn(S)$ is saturated for every S : whenever (Q, c) is a rule instance with $Q \subseteq Cn(S)$ then $c \in Cn(S)$.

Example 13.2 (Derivation closure). Let $Cn_I(S)$ be the set of items having a derivation tree from S : leaves in S , internal nodes instances of rules of I . It is extensive (a one-node tree), monotone, and idempotent (a tree over leaves from $Cn_I(S)$ is grafted onto trees from S). It respects I by the same grafting. For resolution, Chapter 3 writes this as the set of clauses derivable from S .

The closure is the *option closure*: $Cn(S)$ is everything still obtainable from the premises S . A second closure, adapted to redundancy, is introduced in Chapter 14.

13.2 Worlds

Definition 13.3 (Events, histories, state). Fix an initial set $S_0 \subseteq \mathfrak{F}$. An *event* is one of

- $\text{Pop}(f)$: select the item f for processing;
- $\text{Refuse}(f)$: declare f inadmissible as a premise from now on;
- $\text{Bind}(r; f_1, \dots, f_n \vdash f)$: derive f from $f_1, \dots, f_n$ by a rule instance $r = (\{f_1, \dots, f_n\}, f) \in \mathcal{R}$.

A *history* is a finite sequence of events. To each history H attach a triple $\rho(H) = (D, R, P)$ of sets of items, the *extensional state*, by recursion: $\rho(\varepsilon) = (S_0, \emptyset, \emptyset)$ and, writing $\rho(H) = (D, R, P)$,

- $\text{Pop}(f)$ is *well formed* iff $f \in D \setminus (R \cup P)$, and $\rho(H \cdot \text{Pop}(f)) = (D, R, P \cup \{f\})$;
- $\text{Refuse}(f)$ is well formed iff $f \in D \setminus R$, and $\rho(H \cdot \text{Refuse}(f)) = (D, R \cup \{f\}, P)$;
- $\text{Bind}(r; \vec{f} \vdash f)$ is well formed iff $\vec{f} \subseteq P \setminus R$ and $f \notin D$, and $\rho(H \cdot \text{Bind}(r; \vec{f} \vdash f)) = (D \cup \{f\}, R, P)$.

A history is well formed if every event is well formed at its point. Write $D(H), R(H), P(H)$ for the components; D is the set of *derived* items, R the *refused*, P the *popped*. The *available* items are $\text{Av}(H) = D(H) \setminus R(H)$, the *option space* is $\Omega(H) = \text{Cn}(\text{Av}(H))$, and the *provenance* $\pi(H)$ is the set of Bind events occurring in H . A *world* is a well-formed history regarded together with its option space.

A world is the pair that the framework calls (history, option space). The history is append-only: nothing is removed from D or R , and the option space is a derived quantity that can shrink. Two features of the definition matter below. A refused item is never un-refused. And a Bind derives only items not already derived, so D is a set and an item has exactly one recorded justification, the first.

Example 13.4 (The given-clause loop as a history). In the given-clause algorithm of Section 2.2, one iteration selects a clause g from the unprocessed pool ($\text{Pop}(g)$), derives conclusions between g and the processed clauses (a sequence of Bind events), and removes clauses found redundant (Refuse events). The sequence of all events of a run is a well-formed history whose extensional state at each point is the triple (all clauses ever generated, all clauses discarded, all clauses selected). The loop of Chapter 2 is thus a particular strategy for extending histories, and the transition system of Definition 2.6 is the transition system of worlds restricted to that strategy.

13.3 The option space moves only under refusal

Proposition 13.5 (Dynamics of the option space). Let Cn be a closure operator respecting $\perp$, and let H be well formed.

- $\Omega(H \cdot \text{Pop}(f)) = \Omega(H)$ whenever the event is well formed.
- $\Omega(H \cdot \text{Bind}(r; \vec{f} \vdash f)) = \Omega(H)$ whenever the event is well formed.
- $\Omega(H \cdot \text{Refuse}(f)) \subseteq \Omega(H)$ whenever the event is well formed, with equality if and only if $f \in \text{Cn}(\text{Av}(H) \setminus \{f\})$.

Proof. Write $A = \text{Av}(H)$. (a) Pop changes only P , so A is unchanged. (b) The premises $\vec{f}$ lie in A , hence in $\text{Cn}(A)$ by extensivity, and $\text{Cn}(A)$ is saturated, so $f \in \text{Cn}(A)$. The new

set of available items is $A \cup \{f\} \subseteq \text{Cn}(A)$. By monotonicity and idempotence $\text{Cn}(A) \subseteq \text{Cn}(A \cup \{f\}) \subseteq \text{Cn}(\text{Cn}(A)) = \text{Cn}(A)$.

(c) The new available set is $A \setminus \{f\}$ (here $f \in A$ by well-formedness), so containment follows from monotonicity. If $f \in \text{Cn}(A \setminus \{f\})$ then $A \subseteq \text{Cn}(A \setminus \{f\})$ by extensivity, so $\text{Cn}(A) \subseteq \text{Cn}(\text{Cn}(A \setminus \{f\})) = \text{Cn}(A \setminus \{f\})$, which is equality. Conversely if the two closures coincide then $f \in A \subseteq \text{Cn}(A) = \text{Cn}(A \setminus \{f\})$. ■

Corollary 13.6 (Search is movement within a fixed space). *For every well-formed H , $\Omega(H) \subseteq \Omega(\varepsilon) = \text{Cn}(S_0)$, and $\Omega(H) = \text{Cn}(S_0)$ if H contains no Refuse event. In particular, in a refusal-free history every item that will ever be derived already lies in the option space at the start, and progress consists of making items derived (members of D), not of enlarging what is obtainable.*

Proof. Induct on the length of H using Proposition 13.5. The base case is the definition, since $\text{Av}(\varepsilon) = S_0$. ■

The corollary has a practical meaning that is seldom stated. Every time a prover discards an item, it makes a bet about the option space, because discarding is the only operation that can change it; and conversely every efficiency gain from deletion is paid for by the possibility that the bet is wrong. Proposition 13.5(c) says exactly when it is not.

Definition 13.7 (Lossless refusal). A well-formed event $\text{Refuse}(f)$ at H is *lossless* if $\Omega(H \cdot \text{Refuse}(f)) = \Omega(H)$, and *lossless for a goal* $g \in \mathfrak{F}$ if $g \in \Omega(H)$ implies $g \in \Omega(H \cdot \text{Refuse}(f))$.

Proposition 13.5(c) characterizes lossless refusal: *the refused item must already be obtainable from the other available items*. This is the semantic notion of redundancy that the next chapter specialises. A refusal that is lossy for some goals may still be admissible if the goals are not asked for, which is the task-relative notion of Chapter 10 and is the content of the next section.

13.4 The extensional state is the minimal state for all futures

Theorem 13.8 (Future equivalence). *Let H, H' be well-formed histories with $\rho(H) = \rho(H')$. Then for every event e , e is well formed after H if and only if it is after H' , and $\rho(H \cdot e) = \rho(H' \cdot e)$. Consequently, with $\mathcal{F}_{\text{fut}} = \{H \mapsto \rho(H \cdot w) : w \text{ an event sequence}\}$ (defined where $H \cdot w$ is well formed, and by a fixed symbol $*$ elsewhere), one has $\ker \mathcal{F}_{\text{fut}} = \ker \rho$, so ρ is the minimal $\mathcal{F}_{\text{fut}}$ -sufficient representation of well-formed histories (Theorem 10.4).*

Proof. Well-formedness and the state update in Definition 13.3 refer only to the current triple and the event. Hence equal triples treat every event identically, and induction on the length of w gives equal behaviour on all continuations. For the kernel: taking $w = \varepsilon$ shows $\ker \mathcal{F}_{\text{fut}} \subseteq \ker \rho$, and the first part shows $\ker \rho \subseteq \ker \mathcal{F}_{\text{fut}}$. ■

The option space is a function of the extensional state, $\Omega = \text{Cn}(D \setminus R)$, so any representation sufficient for all futures determines it. The theorem has a converse in spirit: a collapse of worlds that identifies two histories with different extensional states changes the set of continuations that are well formed or the states they lead to, so the collapse is not a congruence for the dynamics. The exact formulation follows.

Definition 13.9 (Collapse, two conditions). A *collapse* is a representation $\sigma : \mathcal{W} \rightarrow Q$ on the set $\mathcal{W}$ of well-formed histories. Let $\mathcal{F}$ be a task family on $\mathcal{W}$. σ is $\mathcal{F}$ -*sufficient* if it satisfies the condition of Definition 10.2 for $\mathcal{F}$. σ is *refusal-respecting* if $\ker \sigma \subseteq \ker \text{Av}$, that is, if $\sigma(H) = \sigma(H')$ implies $\text{Av}(H) = \text{Av}(H')$. It is *admissible* for $\mathcal{F}$ if it is both.

By Theorem 10.4, admissibility for $\mathcal{F}$ is sufficiency for $\mathcal{F} \cup \{\text{Av}\}$, and by Theorem 10.7 it is automatic exactly when Av is computable from $\mathcal{F}$. The two conditions are in general independent, and each is violated by natural collapses.

Proposition 13.10 (Independence of the two conditions). *There are a task prov and collapses σ_1, σ_2 of well-formed histories such that σ_1 is $\{\text{prov}\}$ -sufficient but not refusal-respecting, and σ_2 is refusal-respecting but not $\{\text{prov}\}$ -sufficient. Moreover there is a collapse that is sufficient for the verdict task $\text{unsat}(H) = [\perp \in \Omega(H)]$ but not for continuation.*

Proof. Take the task $\text{prov}(H) = \pi(H)$. Let $\sigma_1 = \pi$; it is $\{\text{prov}\}$ -sufficient trivially. It is not refusal-respecting: for $S_0 = \{a, b\}$ with no rules, the histories ε and $\text{Refuse}(a)$ have the same provenance $\emptyset$ and different available sets. Let $\sigma_2 = \text{Av}$, trivially refusal-respecting. It is not $\{\text{prov}\}$ -sufficient: with $S_0 = \{p, q, r\}$ and rules $r_1 = (\{p, q\}, c)$, $r_2 = (\{p, r\}, c)$, the histories $H_1 = \text{Pop}(p), \text{Pop}(q), \text{Bind}(r_1; p, q \vdash c)$ and $H_2 = \text{Pop}(p), \text{Pop}(r), \text{Bind}(r_2; p, r \vdash c)$ have $\text{Av} = \{p, q, r, c\}$ in both and different provenance. For the last statement take $\sigma_3 = \text{unsat}$ and $S_0 = \{a, b\}$ without rules. Neither ε nor $\text{Refuse}(a)$ derives $\perp$, so σ_3 identifies them, yet $\text{Pop}(a)$ is well formed after the first and not after the second. ■

The proof is deliberately elementary. The significance lies in the pattern it exhibits: a representation that is exactly sufficient for the question being asked now (the verdict, the proof, the count) is generally insufficient for *continuation*, and continuation is what a search is. The standing recommendation of the chapter is therefore the following.

A collapse in search is admissible only relative to a declared task and a declared continuation. Preserve $\mathcal{F}$ for the present question (Chapter 10); preserve Av if later events are to behave as they would have; preserve the full extensional state ρ if the search is to be resumed exactly (Theorem 13.8); and preserve provenance π if proofs are to be extracted (Proposition 13.11).

13.5 Provenance is not in the state

Proposition 13.11 (Proofs are not a function of the extensional state). *There are well-formed histories H_1, H_2 with $\rho(H_1) = \rho(H_2)$ and $\pi(H_1) \neq \pi(H_2)$ in which the two recorded justifications of the item c yield derivation trees of c with different sets of leaves. Hence the task “return a derivation of c from S_0 ” is not ρ -sufficient, and by Theorem 13.8 the extensional state, which suffices for all future behaviour of the search, does not suffice for extracting a proof.*

Proof. The histories of the preceding proof have equal $\rho = (\{p, q, r, c\}, \emptyset, \{p, q, r\})$ only if both pop all of p, q, r ; extend H_1 by $\text{Pop}(r)$ and H_2 by $\text{Pop}(q)$. Then $\rho(H_1) = \rho(H_2)$, while the derivation of c read from π has leaves $\{p, q\}$ in H_1 and $\{p, r\}$ in H_2 . ■

This is the world-level counterpart of the residue of Chapter 1: the proof extracted from a run is a function of π , the extensional state is a function of the run, and neither determines

the other. In particular a prover that collapses its state to ρ after every iteration, for memory, can continue indefinitely and can still report that $\perp$ was derived, but cannot say how.

13.6 Outcomes, and two guarantees that must not be conflated

The tasks of the preceding sections ask about the state. A search is also judged by its *outcomes*.

Definition 13.12 (Attainable outcomes). A *continuation* of a well-formed H is an event sequence w with $H \cdot w$ well formed. It is *terminal* if $H \cdot w$ is saturating (Lemma 15.1 in Chapter 15 makes this precise for finite clause universes; in general it means that no Pop or Bind is well formed). The *outcome* of a terminal $H \cdot w$ is refuted if $\perp \in D \setminus R$ and saturated otherwise. The set of *attainable outcomes* of H is

$$\mathcal{C}(H) = \{\text{out}(H \cdot w) : w \text{ a terminal continuation of } H\}.$$

A collapse or refusal q is then judged against a declared outcome task. For refutation the condition is refuted $\in \mathcal{C}(H) \iff \text{refuted} \in \mathcal{C}(q(H))$, which Chapter 14 establishes for the admissible refusals. For repair and diagnosis the condition is too weak: it says that a certificate remains obtainable, not that the distinctions needed to explain or adjust the search remain recorded, and those are the tasks of Chapter 11. The two kinds of requirement correspond to two different representations, the first through Ω and the second through (ρ, π) .

Two properties of an execution are easily conflated and are logically independent. Define a *raw history* to be a sequence of events with no well-formedness requirement, with the state updates of Definition 13.3 applied unconditionally.

Definition 13.13 (Logical soundness and refusal-respect). A raw history is *logically sound* if every item added to D by a Bind event is entailed by the items it was derived from (for an inference system with a semantics, as in Definition 2.1). It is *refusal-respecting* if no event uses, as a premise of a Bind or as the target of a Pop, an item that is in R at that time.

Proposition 13.14 (Independence). *There are raw histories that are logically sound but not refusal-respecting, and raw histories that are refusal-respecting but not logically sound. A well-formed history over a sound inference system is both.*

Proof. Let $S_0 = \{p, p \rightarrow q\}$ with $\rightarrow$ -elimination as the only rule. The raw history $\text{Pop}(p), \text{Pop}(p \rightarrow q), \text{Refuse}(p), \text{Bind}(\rightarrow E; p, p \rightarrow q \vdash q)$ derives only valid consequences and uses the refused p as a premise. For the converse let the rule set contain the unsound instance $(\{p\}, q)$ with $S_0 = \{p\}$; $\text{Pop}(p), \text{Bind}(\{p\} \vdash q)$ respects every refusal and adds q , which is not entailed. For the last statement, well-formedness is the refusal-respecting condition by definition, and logical soundness follows from soundness of the rule instances. ■

Logical soundness is a property of the rule set, assured by a checker for the rules. Refusal-respect is a property of the execution, assured by the gate of Chapter 15. A proof produced by a run that used a refused premise is a valid proof and a violation of the run's recorded restrictions; a run that respects every refusal and applies an unsound rule violates nothing recorded and proves nothing. The lifecycle of Chapter 16 separates the same two things by assigning the first to verifiers and the second to disposition authority.

13.7 What this chapter fixes

(1) A search is a sequence of Pop, Refuse, Bind events on a world (H, Ω) ; only refusal and collapse can change Ω (Proposition 13.5, Corollary 13.6). (2) A refusal is lossless iff the refused item is obtainable from the remaining available items, which is redundancy (Proposition 13.5(c)). (3) A collapse is admissible for a task family iff it is sufficient for the family and refusal-respecting; both conditions can fail independently (Proposition 13.10). (4) The extensional state is minimal for all futures and does not determine proofs (Theorem 13.8, Proposition 13.11).

Exercise 13.1. Show that Proposition 13.5(b) fails if Cn is not required to respect $\perp$. Give a closure operator, an inference system and a Bind event for which Ω strictly increases.

Exercise 13.2. Prove that ρ itself is not minimal for the family of tasks $\{H \mapsto \Omega(H \cdot w)\}_w$ by exhibiting two histories with $\rho(H) \neq \rho(H')$ that agree on every $\Omega(H \cdot w)$. What does this say about the role of P ?

Exercise 13.3. Let $\text{Cn} = \text{Cn}_\perp$ for propositional resolution. Show that $\text{Refuse}(f)$ at H is lossless if and only if f can be derived by resolution from the other available clauses, and use this to show that refusing a clause subsumed by an available clause is in general *not* lossless in this sense. (This is the observation that motivates the closure of Chapter 14.)

Chapter 14

Redundancy as Admissible Collapse

Provers discard clauses. Without deletion, saturation procedures drown in consequences that add nothing, and every serious implementation removes tautologies and subsumed clauses as a matter of course. In the vocabulary of Chapter 13, deletion is Refuse, and Proposition 13.5 says that it is the only event that changes the option space. This chapter proves that for propositional resolution the two standard deletions are admissible, in the strongest sense that fair runs with deletion remain refutationally complete, and shows by a small example that the admissibility of a refusal is a property of the current world and not of the refused item alone.

Throughout, clauses are finite sets of literals as in Chapter 3, $\text{Res}(S)$ denotes the set of clauses derivable from S by resolution (including S itself), and I is propositional resolution.

14.1 A closure adapted to redundancy

The closure Res is the wrong option closure for deletion. If $D \subsetneq C$ and C is refused while D remains available, then $C \in \text{Av}$ is typically not in $\text{Res}(\text{Av} \setminus \{C\})$, because resolution never weakens a clause. So by Proposition 13.5(c) the refusal is lossy for Res , although every use that could be made of C can be made of D (Exercise in Chapter 13). The remedy is to measure options up to redundancy.

Definition 14.1 (Redundancy closure). For a set N of clauses let

$$\uparrow^* N = \{C : C \text{ is a tautology, or } E \subseteq C \text{ for some } E \in N\},$$

and put $\text{Cn}_{\sqsubseteq}(S) = \uparrow^* \text{Res}(S)$.

A clause is in $\text{Cn}_{\sqsubseteq}(S)$ if it is derivable up to subsumption, or is a tautology (which is entailed by everything).

Lemma 14.2 (Lifting through redundancy). *Let N be a set of clauses that is saturated up to redundancy: every resolvent of two members of N lies in $\uparrow^* N$. Then every clause derivable by resolution from $\uparrow^* N$ lies in $\uparrow^* N$.*

Proof. Induct on the derivation. Premises from $\uparrow^* N$ are trivially in it. Let $C = C'_1 \cup C'_2$ be the resolvent on l of $C_1 = C'_1 \cup \{l\}$ and $C_2 = C'_2 \cup \{\bar{l}\}$, both in $\uparrow^* N$.

Case 1: neither premise is a tautology. Then $E_1 \subseteq C_1$ and $E_2 \subseteq C_2$ for some $E_1, E_2 \in N$. If $l \notin E_1$ then $E_1 \subseteq C'_1 \subseteq C$, so $C \in \uparrow^* N$; likewise if $\bar{l} \notin E_2$. Otherwise $E_1 = E'_1 \cup \{l\}$ and $E_2 = E'_2 \cup \{\bar{l}\}$ with $E'_i \subseteq C'_i$. The resolvent $E'_1 \cup E'_2$ of E_1, E_2 lies in $\uparrow^* N$ by saturation up to redundancy, and $E'_1 \cup E'_2 \subseteq C$. A superset of a tautology is a tautology and a superset of a member of N is in $\uparrow^* N$, so $C \in \uparrow^* N$.

Case 2: C_1 is a tautology. Then C_1 contains a complementary pair. If that pair lies inside C'_1 then $C \supseteq C'_1$ is a tautology. Otherwise the pair is $l, \bar{l}$, so $\bar{l} \in C'_1 \subseteq C$ and $C \supseteq C'_2 \cup \{\bar{l}\} = C_2 \in \uparrow^* N$, hence $C \in \uparrow^* N$. The case C_2 tautology is symmetric. ■

Proposition 14.3 ($\text{Cn}_\sqsubseteq$ is an option closure respecting resolution). $\text{Cn}_\sqsubseteq$ is a closure operator, it respects $\mid$ (Definition 13.1), and $\square \in \text{Cn}_\sqsubseteq(S)$ if and only if S is unsatisfiable, for finite S .

Proof. Extensive: $S \subseteq \text{Res}(S)$. Monotone: Res is monotone and $\uparrow^*$ is monotone. Idempotent and respecting $\mid$: $N = \text{Res}(S)$ is closed under resolution, hence saturated up to redundancy. By Lemma 14.2, any resolvent of members of $\uparrow^* N$ lies in $\uparrow^* N$, which is $\text{Cn}_\sqsubseteq(S)$; this is the respect for $\mid$. For idempotence, $\text{Cn}_\sqsubseteq(\text{Cn}_\sqsubseteq(S)) = \uparrow^* \text{Res}(\uparrow^* N)$, and $\text{Res}(\uparrow^* N) \subseteq \uparrow^* N$ by the lemma applied to derivations; applying $\uparrow^*$ gives $\uparrow^* N$ again.

Finally $\square$ is not a tautology and $E \subseteq \square$ forces $E = \square$, so $\square \in \text{Cn}_\sqsubseteq(S)$ iff $\square \in \text{Res}(S)$, which for finite S holds iff S is unsatisfiable by Proposition 3.2 and Theorem 3.4. ■

Hence the framework of Chapter 13 applies with $\Omega = \text{Cn}_\sqsubseteq(\text{Av})$, and the refutability task $\text{unsat}(H) = [\square \in \Omega(H)]$ is the verdict of the run.

14.2 Tautology and subsumption refusal

Theorem 14.4 (Admissible refusals). Let H be a well-formed history, $f \in \text{Av}(H)$ and $A = \text{Av}(H)$. Refusing f is lossless for $\Omega = \text{Cn}_\sqsubseteq(\text{Av})$ in each of the cases

- (i) f is a tautology;
- (ii) f is subsumed: there is $D \in A \setminus \{f\}$ with $D \subseteq f$.

In particular, in both cases $\text{unsat}(H \cdot \text{Refuse}(f)) = \text{unsat}(H)$.

Proof. By Proposition 13.5(c) it suffices to show $f \in \text{Cn}_\sqsubseteq(A \setminus \{f\})$. In case (i), $f \in \uparrow^*(\cdot)$ by definition. In case (ii), $D \in A \setminus \{f\} \subseteq \text{Res}(A \setminus \{f\})$ and $D \subseteq f$, so $f \in \uparrow^* \text{Res}(A \setminus \{f\})$. Proposition 14.3 supplies the closure properties used by Proposition 13.5, and the last claim follows since $\square \in \Omega$ is the verdict. ■

The theorem is the formal sense of the slogan that redundancy is lossless refusal. Note that the witness D must lie in the *current* available set. This is exactly where history enters.

Example 14.5 (Duplicates and the witness that was already refused). Let items be *occurrences* of clauses, so that two occurrences c_1, c_2 may carry the same clause $\{a\}$, and let $S_0 = \{c_1, c_2, c_3\}$ with $c_3 = \{\neg a\}$. The set is unsatisfiable. The refusal $\text{Refuse}(c_1)$ is admissible, since the available c_2 carries the same clause. After it, $\text{Refuse}(c_2)$ is *not* lossless: the available set is $\{c_3\}$, which is satisfiable. A refusal criterion that accepts a witness among all items *seen so far*, available or refused, would approve both steps and destroy the refutation. The criterion of Theorem 14.4 is correct because it quantifies over $\text{Av}(H)$, not over $D(H)$.

This is the clause-level instance of the principle that a refusal is judged by the world in which it is made. Quotienting the occurrences c_1, c_2 to one item with the status “available if and only if some member is available” is the refusal-respecting collapse of Definition 13.9; giving the class the status “refused if some member is refused” is not, and destroys the refutation in the same way.

14.3 Completeness with deletion

The fairness theorem of Chapter 2 (Theorem 2.4) is stated for increasing chains $N_0 \subseteq N_1 \subseteq \dots$ and does not apply to a procedure that deletes. It must be replaced by an argument about *persistent* clauses and the witnesses that justify each deletion. Lossless refusals preserve refutability at each step. A prover must also *find* the refutation, and an infinite sequence of individually admissible refusals could in principle remove every witness in turn. The following theorem shows that for propositional resolution under a fair strategy, it cannot.

Definition 14.6 (Fair run with deletion). A well-formed infinite or terminated sequence of events starting from a finite set S_0 of clauses is a *run with admissible deletion* if every $\text{Refuse}(f)$ event is of the form (i) or (ii) of Theorem 14.4. Its *persistent set* is $N_\infty = \bigcup_i \bigcap_{j \geq i} \text{Av}(H_j)$, where H_j is the prefix of length j . The run is *fair* if every instance of the resolution rule whose two premises lie in N_∞ has its conclusion in $D(H_j)$ for some j .

Theorem 14.7 (Completeness with admissible deletion). *Let S_0 be a finite unsatisfiable set of clauses and let (H_j) be a fair run with admissible deletion. Then $\square \in D(H_j) \setminus R(H_j)$ for some j . Equivalently, $\square$ is persistent.*

Proof. All clauses ever derived contain atoms of S_0 only, and clauses are sets, so the universe of clauses is finite; hence $D_\infty = \bigcup_j D(H_j)$ is finite and N_∞ , being a subset, is finite.

Step 1: every clause in D_∞ is a tautology or contains a member of N_∞ . Let $f \in D_\infty$. If f is persistent, take f itself. If f is a tautology there is nothing to show. Otherwise f is not a tautology and not persistent, so it was refused at some time t , necessarily by a subsumption step, with witness $f_1 \subseteq f$, $f_1 \in \text{Av}(H_t)$ and $f_1 \neq f$, so $f_1 \subsetneq f$. Repeat for f_1 : either f_1 is persistent, or a tautology, or refused later with a witness $f_2 \subsetneq f_1$. The cardinalities strictly decrease along the chain, and a subset of a non-tautology is a non-tautology, so every member is a non-tautology and the chain ends at a clause that is never refused. An unrefused derived clause is available from its derivation onward, hence persistent. So f contains a persistent clause.

Step 2: N_∞ is unsatisfiable. Suppose $v \models N_\infty$. By Step 1, v satisfies every clause of D_∞ , since each is a tautology or a superset of a clause v satisfies. In particular $v \models S_0$, contradicting unsatisfiability of S_0 .

Step 3: N_∞ is saturated up to redundancy. Let E be a resolvent of two members of N_∞ . By fairness $E \in D_\infty$, and by Step 1 $E \in \uparrow^* N_\infty$.

Step 4. By Step 2 and Theorem 3.4, $\square \in \text{Res}(N_\infty)$. By Step 3 and Lemma 14.2, $\text{Res}(N_\infty) \subseteq \uparrow^* N_\infty$, so $\square \in \uparrow^* N_\infty$. Since $\square$ is not a tautology, some $E \in N_\infty$ satisfies $E \subseteq \square$, that is, $E = \square$. So $\square \in N_\infty$. ■

The proof is a short instance of the model-based arguments of Bachmair and Ganzinger [2] specialised to the propositional case, where finiteness of the clause universe replaces

the ordering machinery. Two features are worth recording. Step 1 uses the strictness $f_1 \subsetneq f$, which is available because items are clause sets; with occurrences, a chain $c_1 \rightarrow c_2 \rightarrow \dots$ of equal copies can in principle continue forever, which is why implementations impose a tie-break by age. And Step 2 is purely semantic: it uses no property of the strategy except that every deleted clause was redundant at its time. The role of the history is concentrated in Step 1.

14.4 Admissibility is task-relative

Deletion preserves refutability but not the option space under Res.

Proposition 14.8 (Premature relative to the wrong task). *Let $S_0 = \{p, p \vee q\}$ and $H = \text{Refuse}(p \vee q)$. Under $\text{Cn}_{\sqsubseteq}$ the refusal is lossless. Under $\text{Cn}_{\text{Res}} = \text{Res}$ it is lossy: $p \vee q \in \text{Res}(S_0) \setminus \text{Res}(S_0 \setminus \{p \vee q\})$. So the deletion is admissible for the task “is S_0 refutable” and premature for the task “list every clause derivable from S_0 ”.*

Proof. $p \subseteq p \vee q$ gives the first statement by Theorem 14.4(ii). For the second, $\text{Res}(\{p\}) = \{p\}$ does not contain $p \vee q$. ■

The proposition is the clause-level shadow of Theorem 10.12: a non-injective treatment of the clause set serves some task families and not others, and the family must be named. Table 14.1 summarizes the standard criteria. The third row is a fact about complexity, cited and not proved here.

Criterion for refusing f in A	Decidable in	Status
f is a tautology	linear time	lossless for $\text{Cn}_{\sqsubseteq}$ (Thm. 14.4(i))
Some $D \in A \setminus \{f\}$ has $D \subseteq f$	polynomial time	lossless for $\text{Cn}_{\sqsubseteq}$ (Thm. 14.4(ii)); fair runs complete (Thm. 14.7)
f is entailed by $A \setminus \{f\}$	coNP-complete [8]	lossless for refutability by semantics, but the persistence argument of Thm. 14.7 needs an ordering restriction not developed here
f matches a witness among all items seen, refused or not	linear in the history	not admissible (Example 14.5)

Table 14.1: Redundancy criteria for propositional clause sets. The first two rows are proved in this chapter.

Remark 14.9 (Deletion records and repair obligations (proposed extension)). The proof of Theorem 14.7 uses, for each refused clause, a replacement witness that was available when the refusal was made. A deletion record that retains the refused clause together with its witness is what is needed to reconsider the deletion if the witness is later removed or an initial assumption is altered: the clauses whose only witnesses depend on the altered item are exactly the deletions that generate repair obligations. This is a proposal and is not proved here. Its natural home is the typed dependency structure of Chapter 12, with the refusal record as a new kind of edge, and the mutation experiments of Chapter 18.

14.5 What this chapter fixes

(1) Redundancy is lossless refusal for the option closure adapted to the task: $Cn \sqsubseteq$ for refutation (Proposition 14.3, Theorem 14.4). (2) Fair runs of propositional resolution with tautology and subsumption deletion remain refutationally complete (Theorem 14.7). (3) A refusal is admissible only relative to the current available set, and a criterion that consults refused items is not (Example 14.5). (4) The same deletion is admissible for one task and premature for another (Proposition 14.8).

The refusals of this chapter are decided by the search procedure itself, from information it holds. The next chapter considers refusals and selections that are *guided* by an external signal, and asks what such a signal can and cannot certify.

Exercise 14.1. In Step 1 of Theorem 14.7, show that the chain of witnesses cannot end at a tautology. (The parenthetical in the proof gives the idea; write it as a lemma.)

Exercise 14.2. Show that Theorem 14.7 fails if admissible deletion is replaced by deletion of any clause entailed by the remaining available clauses and the fairness condition is unchanged. (Hint: two mutually entailed clauses.)

Exercise 14.3. Prove that backward subsumption, refusing already processed clauses subsumed by a newly derived clause, is an instance of Theorem 14.4(ii), and that forward subsumption, discarding a newly derived clause that is subsumed, requires the refused clause to be recorded in R if later re-derivation is to be blocked.

Chapter 15

Guidance and the Verifier Gate

The search procedures of Part I choose their next step by fixed rules: oldest clause first, smallest clause first, a weighted mixture. Modern systems replace the rule by a learned model that proposes the next step, a premise to use, a tactic to try, a lemma to add. This chapter asks what such a proposer can change and what it cannot. The answer has two halves. In the world model of Chapter 13, a proposal is an event, and only Refuse and collapse change the option space, so guidance that merely orders Pop and Bind events cannot affect what is obtainable (Section 15.1). Guidance that proposes refusals can, and the second half separates proposals from acceptance: a *gate* that checks each proposed event against a decidable criterion makes the proposer irrelevant for soundness and for completeness, and an *ungated* proposer is shown to be capable of destroying both (Sections 15.2 and 15.3). The last section proves in abstract form that a gate which can see only what the proposer sees cannot discriminate between situations the proposer cannot distinguish (Section 15.4).

15.1 Ordering does not change the option space

Retain the setting of Chapter 14: propositional resolution over a finite set S_0 , worlds with $\Omega = \text{Cn}_{\sqsubseteq}(\text{Av})$, and refusals restricted to the admissible forms of Theorem 14.4.

Lemma 15.1 (Runs are finite and maximal runs are fair). *Every well-formed history over a finite clause universe has length at most $3N$, where N is the number of clauses over the atoms of S_0 . Call a history saturating if no Pop or Bind event is well formed after it. Every history extends to a saturating one, and every saturating history is a fair run in the sense of Definition 14.6.*

Proof. Each Pop adds an item to P , each Refuse adds an item to R , each Bind adds an item to D , and these sets only grow within a set of size N ; the bound follows. Extension to a saturating history is by repeatedly adding well-formed Pop and Bind events, which terminates by the bound. For fairness, let a resolution instance have both premises persistent, that is, available at the end. If its conclusion E were not in D at the end, then Pop of any unpopped available premise would be well formed, contradicting saturation, so both premises are popped and available, and Bind of the instance is well formed, contradicting saturation again. Hence $E \in D$. ■

Theorem 15.2 (Guidance is irrelevant to completeness). *Let S_0 be a finite unsatisfiable set of clauses and let H be any saturating history all of whose Refuse events are admissible in the sense of Theorem 14.4. Then $\square \in D(H) \setminus R(H)$, whatever the order of the Pop and Bind events. In particular this holds for histories produced by any selection function, learned or otherwise, provided it does not stop before saturation.*

Proof. By Lemma 15.1 the history is a fair run with admissible deletion. Apply Theorem 14.7. ■

The theorem is not a statement that guidance is useless. The length of the history, the number of items derived before $\square$ appears, and the memory used all depend strongly on the order. What it excludes is a different claim, sometimes made implicitly: that a better selection function can change which refutations *exist*. In the language of Chapter 13, selection acts within a fixed space (Corollary 13.6). A formal reading of “the model found a proof the baseline could not find” is therefore always a statement about a resource bound, never about the option space, unless the model’s proposals include refusals or collapses.

Remark 15.3 (Early stopping). The hypothesis “does not stop before saturation” cannot be dropped. A guided procedure that stops at a time limit is not a saturating run, and Theorem 15.2 says nothing about it. Its power is the probability, over inputs, that $\square$ has appeared by the time limit, which is a property of the guidance and the distribution of inputs. No such probability is computed in this book.

15.2 The verifier gate

Definition 15.4 (Proposer, gate, gated run). A *proposer* is an arbitrary function Prop from histories to events. It need not be computable, deterministic, or sound; a language model with sampling is an example, understood as a nondeterministic proposer. The *gate* for refutation of clause sets is the decidable predicate Gate(H, e) that holds iff e is well formed after H and, in case $e = \text{Refuse}(f)$, f is a tautology or is subsumed by some $D \in \text{Av}(H) \setminus \{f\}$. A *gated run* starting from ε applies the proposed event if the gate accepts it and discards it otherwise.

Theorem 15.5 (Gate separates the proposer from soundness and completeness). *Let S_0 be a finite set of clauses.*

- (i) *Soundness. For every proposer, every gated run is a well-formed history with admissible refusals, every $f \in D(H)$ is a consequence of S_0 , and $\square \in D(H)$ implies S_0 is unsatisfiable.*
- (ii) *Completeness. If S_0 is unsatisfiable and the proposer is exhaustive, in the sense that at every history at which some Pop or Bind event is well formed it eventually proposes one, then every gated run extends to $\square \in D(H) \setminus R(H)$.*
- (iii) *Necessity of exhaustiveness. There are finite unsatisfiable S_0 and non-exhaustive proposers whose gated runs never derive $\square$.*

Proof. (i) Every applied event satisfies the gate, so is well formed and, if a refusal, admissible. Every $f \in D(H)$ is in S_0 or is the conclusion of a sound rule instance with premises in $D(H)$, so is a consequence of S_0 by induction (Proposition 3.2). If $\square \in D(H)$ then $S_0 \models \square$.

(ii) A gated run with an exhaustive proposer continues until no Pop or Bind is well formed, that is, to a saturating history, since the run is finite by Lemma 15.1 and each stage

at which an event is well formed will eventually see one proposed and, if well formed, accepted. Apply Theorem 15.2.

(iii) Let $S_0 = \{\{a\}, \{\neg a\}\}$ and let the proposer propose only Refuse events that the gate rejects. The run is ε forever; $\Box \notin D(\varepsilon)$. ■

The architectural content of the theorem is Theorem 1.5 of Chapter 1 in the setting of search: assurance attaches to the gate, and the producer may be arbitrarily unreliable without affecting soundness. Completeness is an exhaustiveness property of the producer, and exhaustiveness is cheap to impose: any proposer can be wrapped to fall back to a fixed fair strategy every k th step, at a cost of at most a factor k in the length of the history.

15.3 Ungated proposals

If the gate is omitted for Refuse events, the proposer acquires the one power that changes the option space.

Proposition 15.6 (Ungated refusal can destroy refutability). *There is a finite unsatisfiable S_0 and a well-formed history consisting of a single Refuse event such that $\Box \notin \Omega(H)$, and $\Box$ lies in $D(H')$ for no extension H' of H .*

Proof. Take $S_0 = \{\{a\}, \{\neg a\}\}$ and $H = \text{Refuse}(\{a\})$. Then $\text{Av}(H) = \{\{\neg a\}\}$ is satisfiable, so $\Box \notin \text{Cn}_{\sqsubseteq}(\text{Av}(H)) = \Omega(H)$. By Corollary 13.6 and its proof, no extension enlarges Ω , and every item bound in an extension lies in Ω (Proposition 13.5(b)). Hence $\Box$ is never derived. ■

The example is trivial on purpose. A learned proposer that predicts which clauses are “useless” is making a prediction of exactly this kind, and its errors are irrecoverable in the sense of Proposition 13.5: a refused item stays refused and the option space is smaller for good. The remedy is not to forbid such predictions but to demote them: a prediction of uselessness should lower the priority of an item (ordering, harmless by Theorem 15.2) and not delete it (refusal, harmless only if gated).

	Gate on all events	Gate omitted for refusal
Proposer exhaustive	sound and complete; efficiency depends on proposer (Thm. 15.5)	sound (derived items are consequences); completeness lost if a needed item is refused (Prop. 15.6)
Proposer not exhaustive	sound; may be incomplete (Thm. 15.5(iii))	sound; may be incomplete for either reason

Table 15.1: What the proposer and the gate each control. Soundness of derived items never depends on the proposer, since Bind is always checked as a rule instance; refutational completeness depends on the gate for refusals and on exhaustiveness.

15.4 Self-contained gates cannot discriminate what the proposer cannot

The gate of Definition 15.4 is decidable and external: it reads the history and the clause, which are the same objects the checker of Chapter 1 reads. A different design asks the proposer, or another model sharing its inputs, to judge its own outputs. The structural limit of such a design is a consequence of Proposition 1.12, and is stated here in the abstract form it will be needed in (see also [38]).

Definition 15.7 (Environments, evidence, gate). Let $\mathcal{E}$ be a set of *environments* (the situations in which a proposal will be used), Z a set of *evidence states*, and $z : \mathcal{E} \rightarrow Z$ the *evidence map*. A *correct action* is a function $\alpha : \mathcal{E} \rightarrow \{\text{accept}, \text{reject}\}$. A *gate restricted to z* is a function $G \circ z$ with $G : Z \rightarrow \{\text{accept}, \text{reject}\}$. It is *correct* if $G \circ z = \alpha$.

Theorem 15.8 (Impossibility of self-contained gates). *A correct gate restricted to z exists if and only if α is constant on every fibre of z . In particular, if $e_0, e_1 \in \mathcal{E}$ satisfy $z(e_0) = z(e_1)$ and $\alpha(e_0) \neq \alpha(e_1)$, then no gate restricted to z is correct, however it is constructed and however much computation it uses.*

Proof. This is Proposition 1.12 for the task α and the representation z . For the second statement, a gate $G \circ z$ takes the same value at e_0 and e_1 . ■

Corollary 15.9 (A signal from outside the evidence class is necessary). *Let α be a correct-action function not constant on some fibre of z . Any correct gate must read a quantity not determined by z . Adding to the gate a function of z , such as the proposer's own confidence or an independent run of a model that sees the same inputs, leaves the set of indistinguishable pairs unchanged.*

Proof. A gate reading z and a function $u(z)$ is a function of z ; Theorem 15.8 applies. ■

In proof search, an example is immediate. Let an environment consist of a statement φ together with its status (provable or not), and let the evidence be a natural-language justification produced by a model. If the model produces the same justification for a provable φ_0 and an unprovable φ_1 (as a fluent but wrong argument might), then by Theorem 15.8 no judge that sees only the justification can accept one and reject the other. The kernel check $\text{Chk}(d, \varphi)$ is not subject to the theorem because its evidence is (d, φ) together with the independently defined meaning of Chk , a function of the environment that is not determined by what the model emitted. This is the content of the independence principle of Chapter 1 in the abstract setting.

Remark 15.10 (Grounded does not mean correct). Acceptance by an external check is not acceptance of the intended claim. The kernel verdict is correct for the question “does d check against φ ”, which is the layer O1 of Chapter 9. If the formalization φ of the target τ is wrong (layer O2) or the target does not correspond to the observation (layer O3), a grounded gate accepts a proof of the wrong thing. Grounding removes the self-referential failure; it does not remove the others.

15.5 What this chapter fixes

(1) Ordering guidance acts within a fixed option space and cannot affect completeness for saturating runs (Theorem 15.2). (2) A gate that checks each proposed event against a decidable criterion makes soundness independent of the proposer and reduces completeness to an exhaustiveness condition (Theorem 15.5). (3) An ungated refusal changes the option space irreversibly (Proposition 15.6), so predictions of uselessness should demote, not delete. (4) A gate confined to the evidence the proposer already has cannot discriminate environments that evidence does not distinguish (Theorem 15.8); a grounded gate fixes that failure and no other (Remark 15.10).

Exercise 15.1. Prove the claim in the text that wrapping a proposer to fall back to a fixed fair strategy every k th step yields an exhaustive proposer. State precisely what “fixed fair strategy” must satisfy.

Exercise 15.2. Show that Theorem 15.2 fails for first-order resolution unless the clause universe is replaced by a finiteness assumption, and identify the step of the proof that uses finiteness. (Hint: Lemma 15.1.)

Exercise 15.3. In Theorem 15.8, let $\mathcal{E}$ be finite and α arbitrary. Compute the minimal number of values of an evidence map z' such that a correct gate restricted to z' exists, and relate it to Theorem 10.10.

Chapter 16

Lifecycle and Authority

A search step changes a world. A claim, however, has a life that extends beyond a single search: it is opened as a goal, tied to evidence and obligations, transformed as it is repaired or reformulated, verified, and eventually accepted or rejected. This chapter models that life as a small labelled transition system, proves two safety properties of it, and shows that the most natural-looking simplification, treating a failed verification as a refusal, destroys exactly the repairability that Chapter 11 showed to be essential. The first property is about *what must have happened* before a claim is collapsed into an accepted state. The second is about *who may do it*: verifiers report results and never gain the authority to dispose of a claim.

16.1 The machine

Definition 16.1 (Lifecycle machine). Fix a finite set Obl of *obligations*, for instance the layers O1 to O3 of Chapter 9 together with any declared by the claim. A *configuration* is a triple (q, v, W) where

- $q \in \{\text{open, bound, transformed, verified, verificationFailed, refused, collapsed}\}$ is the *status*;
- $v \in \mathbb{N}$ is the *artifact version*;
- $W \subseteq \text{Obl}$ is the set of obligations verified at version v .

The initial configuration is $(\text{open}, 0, \emptyset)$. Each transition is labelled by an event and an *actor*, either *disp* (a disposition authority) or *ver* (a verifier):

Event (actor)	Source status	Result
Bind (disp)	open	bound, v, W unchanged
Transform (disp)	bound, transformed, verified, verificationFailed	transformed, $v := v + 1, W := \emptyset$
VerifyOK(o) (ver)	bound, transformed	$W := W \cup \{o\}$; status verified if now $W = \text{Obl}$, else unchanged
VerifyFail(o) (ver)	bound, transformed	verificationFailed, v, W unchanged
Collapse (disp)	verified	collapsed
Refuse (disp)	any status other than refused, collapsed	refused

The statuses refused and collapsed are *terminal*: no transition leaves them. The statuses other than the terminal ones are *live*.

The table adopts three design decisions. (1) Every Transform clears W : a verification is a statement about one version of the artifact and does not survive its replacement. (2) Refuse is permitted from every live status, so the disposition authority can always decline to continue, which corresponds to the append-only refusal set of Chapter 13. (3) verificationFailed is a live status: its outgoing transitions are Transform and Refuse.

The open status has no verify transition: a claim must first be bound to the obligations it will be verified against.

16.2 Collapse safety

Theorem 16.2 (Collapse requires current, complete verification). *In every reachable configuration with $q = \text{collapsed}$ one has $W = \text{Obl}$. Moreover in every trace reaching such a configuration, for each $o \in \text{Obl}$ the last event concerning o among $\{\text{VerifyOK}(o), \text{VerifyFail}(o)\}$ is a $\text{VerifyOK}(o)$, and no Transform occurs after the first of these final VerifyOK events.*

Proof. Invariant. Every reachable configuration with $q = \text{verified}$ has $W = \text{Obl}$. This holds because verified is entered only by VerifyOK when the new W equals Obl, and the only transitions that change W from a status verified are Transform, which leaves verified. The status collapsed is entered only by Collapse from verified, with W unchanged, so $W = \text{Obl}$.

Trace claim. Let the trace end in collapsed. Let t be the last Transform in the trace, or the start if none. After t , $W = \emptyset$ at the moment t occurs, and W grows only by VerifyOK events. Since $W = \text{Obl}$ at collapse, every o has a $\text{VerifyOK}(o)$ after t . Between that event and the collapse there is no $\text{VerifyFail}(o)$: a VerifyFail moves to verificationFailed, from which the only exits are Transform and Refuse, and there is no Transform after t , while Refuse is terminal. So the last event concerning o is a $\text{VerifyOK}(o)$, and it comes after the last Transform. ■

The theorem is the lifecycle form of a requirement stated in Chapter 12: an inherited verification is a fact about a particular version. A claim that has been verified, then transformed, and then collapsed without re-verification is excluded by the machine and not merely discouraged. It also interacts with Chapter 9: since Obl includes the layers beyond O1, the machine can enter verified only after an actor outside the kernel has produced $\text{VerifyOK}(\text{O2})$ and $\text{VerifyOK}(\text{O3})$. The kernel is one verifier among several, and it can contribute only $\text{VerifyOK}(\text{O1})$.

16.3 Authority is not verification

Theorem 16.3 (Verifiers do not dispose). *Every transition whose actor is ver has target status in $\{\text{bound}, \text{transformed}, \text{verified}, \text{verificationFailed}\}$, and none has target refused or collapsed. Consequently no sequence of verifier events alone reaches a terminal status, and the set of terminal statuses reachable from a configuration depends on verifier events only through which disposition events become enabled.*

Proof. Inspection of the table: the verifier events VerifyOK and VerifyFail have the stated targets, and the only events with targets refused, collapsed are Refuse and Collapse, both

with actor disp. A sequence of verifier events cannot contain an event leaving a live status through a disposition label, so it stays live. The last sentence restates that Collapse is enabled exactly at verified, a status that verifier events can produce. ■

The separation is the lifecycle counterpart of the gate of Chapter 15. The gate rejects proposals; it does not decide what to propose. Here the verifier reports; it does not decide the fate of the claim. A system in which a verifier's negative report removes a claim from circulation has assigned the verifier a disposition authority that the verifier's own assurance does not warrant, since a failed check is not a refutation (Chapter 1).

16.4 Why a failed verification must stay live

Consider the variant machine $\mathcal{L}'$ in which the target of every $\text{VerifyFail}(o)$ transition is refused instead of $\text{verificationFailed}$.

Proposition 16.4 (Automatic refusal destroys repair). *In the machine of Definition 16.1 there is a trace reaching collapsed that contains a VerifyFail event. In $\mathcal{L}'$ no trace containing a VerifyFail event reaches collapsed.*

Proof. Take $\text{Obl} = \{\text{O1}\}$. The trace $\text{Bind}, \text{VerifyFail}(\text{O1}), \text{Transform}, \text{VerifyOK}(\text{O1}), \text{Collapse}$ is valid: after the failure the status is $\text{verificationFailed}$, whose exits include Transform (to transformed at version 1 with $W = \emptyset$); $\text{VerifyOK}(\text{O1})$ gives $W = \text{Obl}$ and status verified, and Collapse follows. In $\mathcal{L}'$ the same VerifyFail moves to refused, which is terminal. ■

Proposition 16.4 is the lifecycle form of Chapter 11. The repair of a failed result, the replacement of an artifact by a nearby one, is a transition out of $\text{verificationFailed}$. If the failure refuses, there is nothing to repair, and the option space of the claim, in the sense of Chapter 13, has been reduced by an actor who was not authorized to reduce it. Formally: the set of terminal statuses reachable from $\text{verificationFailed}$ is $\{\text{refused}, \text{collapsed}\}$ in $\mathcal{L}$ and $\{\text{refused}\}$ in $\mathcal{L}'$.

Remark 16.5 (Information in a failure). The statement of Theorem 11.13 applies to the failure itself: what is retained in $\text{verificationFailed}$ should include which obligation failed, on which version, and with which witness, because the repair relation of Definition 11.12 reads that information. The machine records only the status; the history carries the rest.

16.5 Reachability and history

The machine is deliberately small enough to be explored exhaustively. With a bound on the version, the configuration space is finite, and the claims above were checked by exhaustive enumeration of all traces up to a fixed length for $|\text{Obl}| = 2$ and 3 (no violations). The proofs given are the general ones; enumeration is a check on the table.

Proposition 16.6 (Histories are not recoverable from configurations). *A live configuration always has an enabled event, so every maximal finite trace ends in a terminal status. Two distinct traces can end in the same configuration, so the final configuration does not determine the history.*

Proof. Refuse is enabled at every live status. The traces Refuse and $\text{Bind}, \text{Refuse}$ both end in $(\text{refused}, 0, \emptyset)$ and differ. This is the analogue of Proposition 13.11 for lifecycles. ■

16.6 Proposed extension: typed memory consolidation

This section is a proposed formalization. It does not describe the guarantees of any implemented memory system, and no property of the retrieval dynamics of such a system is assumed.

The lifecycle machine governs one claim. A system that stores many claims and the traces that produced them has the further problem of *consolidation*: replacing the full history by a smaller memory representation. The elements of the preceding chapters combine into a small calculus for this, in which retrieval guides search, verification is separate from commitment, and consolidation is an admissible collapse.

Definition 16.7 (Memory configuration). A *memory configuration* is $S = (H, M, K, \nu)$ where H is an append-only event history, M a memory representation, K a finite set of *commit capabilities*, and $\nu \in \mathbb{N}$ the *environment version*, which counts the changes to assumptions, policy or checking environment. Items are triples $m = (p, \eta, \varrho)$: payload, retrieval parameters, provenance reference. Events are $\text{Recall}(q, L)$, $\text{Verify}(m, v)$, $\text{Permit}(m)$, $\text{Commit}(m, k)$, $\text{Refuse}(m, r)$, $\text{Reopen}(m, r)$, $\text{Env}(\delta)$, $\text{Transport}(m, w)$ and $\text{Consolidate}(c)$. Fix a family $\mathcal{F}$ of future tasks on histories.

Evidence is indexed by version. Define by recursion on H the sets $\text{Ver}(H)$ and $\text{Perm}(H)$ of items *currently verified* and *currently permitted*: both start empty; $\text{Verify}(m, v)$ adds m to Ver and $\text{Permit}(m)$ adds m to Perm ; $\text{Env}(\delta)$ empties both and increments ν ; $\text{Transport}(m, w)$, applicable immediately after an Env and only when w witnesses that the obligations behind the verification (respectively permission) of m survive that change, restores m to the set in which it stood before the Env ; $\text{Refuse}(m, \cdot)$ removes m from both sets. The witness w plays the role of the interface lemma of Chapter 12: it is a certificate that the consulted set of the verification is disjoint from what the change touched.

The transition rules are:

- $\text{Recall}(q, L)$ may be applied to any configuration; it appends the event and returns a list L of *candidates*. Candidates carry no standing: recall changes neither verification, permission, nor commitment status.
- $\text{Verify}(m, v)$ is applied when v is a witness of the declared verification condition $V_H(m)$ for the current history and version; it appends the event.
- $\text{Permit}(m)$ is applied by the disposition authority when policy at the current version permits commitment of m .
- $\text{Commit}(m, k)$ is applicable only if $m \in \text{Ver}(H) \cap \text{Perm}(H)$ and $k \in K$; it removes k from K and appends the event.
- $\text{Refuse}(m, r)$ appends the event. $\text{Reopen}(m, r)$ is applicable only to a refused item, by the disposition authority, and appends the event with its reason. A reopened item has no current evidence until it is verified and permitted again.
- $\text{Env}(\delta)$ and $\text{Transport}(m, w)$ act on versions as just described.
- $\text{Consolidate}(c)$ replaces M by the pair $(c(H), \varepsilon)$ and is applicable only if $c : \mathcal{H} \rightarrow \mathcal{M}$ is $\mathcal{F}$ -extension-sufficient: $c(H_1) = c(H_2)$ implies $f(H_1 H'') = f(H_2 H'')$ for every $f \in \mathcal{F}$ and every suffix H'' . Subsequent events are appended to the second component of M .

Definition 16.8 (Invariants). A configuration is *well founded* if (I1) every committed m was in $\text{Ver} \cap \text{Perm}$ of the prefix of H immediately before its Commit , that is, had evidence for the version current at commitment; (I2) the capabilities used by the Commit events in

H are pairwise distinct and disjoint from K ; (I3) every $\text{Reopen}(m, r)$ in H is preceded by a $\text{Refuse}(m, \cdot)$; (I4) $M = (c(H'), H'')$ for a factorization $H = H'H''$ and an $\mathcal{F}$ -extension-sufficient c whose consolidation event ends H' , or M is the initial representation.

Theorem 16.9 (Preservation of memory invariants). *If S is well founded and $S \rightarrow S'$ by any rule above, then S' is well founded. Moreover recall never changes the set of committed items.*

Proof. By cases on the rule. I1 concerns the prefix before each Commit and so is untouched by any rule except a new Commit, which satisfies it by applicability. $\text{Commit}(m, k)$ removes k from K while recording it in H , so the used capabilities remain pairwise distinct (since $k \in K$ and K is disjoint from the used set) and disjoint from the new K , giving I2; all other rules leave K and the used set unchanged. Reopen is applicable only after a refusal, which gives I3; the other rules do not add a Reopen. For I4, every rule other than Consolidate appends an event to H and to the second component of M , preserving the factorization; $\text{Consolidate}(c)$ is applicable only for extension-sufficient c and sets $H' = H$, $H'' = \varepsilon$. For the last statement, only Commit changes the set of committed items. ■

Theorem 16.9 fixes what I1 asserts: not that some verification once occurred, but that evidence for the version current at commitment did. An Env event between a verification and a commitment forces either a fresh verification or a transport witness. The cost of the latter is the cost of an interface lemma, and without one the calculus refuses the commitment.

Proposition 16.10 (Memory answers current-history tasks). *If $M = (c(H'), H'')$ with c extension-sufficient, then for each $f \in \mathcal{F}$ the value $f(H'H'')$ is a function of M alone.*

Proof. Fix f and H'' . The map $g : H_1 \mapsto f(H_1H'')$ is constant on the fibres of c by extension-sufficiency, so $g = \bar{g} \circ c$ by Proposition 1.12, and $f(H'H'') = \bar{g}(c(H'))$. The pair $(c(H'), H'')$ determines $\bar{g}$ and its argument. ■

Extension-sufficiency is a right-congruence condition, in the manner of Myhill and Nerode: the coarsest sufficient encoder is the quotient of histories by the relation $H_1 \sim H_2$ iff $f(H_1H'') = f(H_2H'')$ for all $f \in \mathcal{F}$ and all H'' , and by Theorem 10.4 the sufficient encoders are exactly those whose kernel refines it. A consolidation that is sufficient only for tasks on the history at the moment of consolidation is not enough once new events arrive, which is why the weaker condition $\ker c \subseteq \ker \mathcal{F}$ does not appear in the rule.

The refinement relation is one of inclusion: a sufficient encoder retains *at least* the distinctions the declared tasks need and may retain more. Equality of the two relations would be minimality, which is a separate property. If $\mathcal{F}$ contains only proof checking, discarding search history can be sufficient. If it also contains diagnosis, refusal audit or repair after assumption changes, the corresponding distinctions (Chapters 11 and 13) must survive consolidation, and by Theorem 10.12 nothing beyond the declared family can be asserted. The same encoder can be sufficient for the first family and insufficient for the second, and a type that records the family in the encoder's signature makes this visible.

Two limits should be recorded. The use of a capability in K prevents reuse within the calculus only; exactly-once execution across distributed components requires protocol guarantees beyond this model. And the verification predicate V_H is application-specific: for a proof capsule it may be a successful kernel check against a stated statement and axiom environment, for a historical record it may be resolution of provenance and consistency of

replay. A single generic “verified” flag would conceal that difference, which is the point of Chapter 9.

16.7 What this chapter fixes

- (1) A lifecycle machine with versions and per-obligation verification enforces that collapse is preceded by complete verification of the current version (Theorem 16.2). (2) Verifiers report and never dispose: no verifier event reaches a terminal status (Theorem 16.3). (3) A failed verification is a live status, since otherwise the possibility of repair is removed by an actor without the authority to remove it (Proposition 16.4). (4) The final status does not determine the history that produced it (Proposition 16.6).

This completes the account of the process. Part V turns to its product and asks what a verified, collapsed result may be taken to say about the world.

Exercise 16.1. Add a status superseded reached by a new claim replacing an old one, and decide, with a proof, whether Theorem 16.2 and Theorem 16.3 continue to hold when collapsed claims may be superseded.

Exercise 16.2. Replace the clearing of W at every Transform by clearing only those obligations in a set $\text{Aff}(\text{transform})$ determined by the typed dependency graph of Chapter 12. State a condition on Aff under which Theorem 16.2 remains true with “current” reinterpreted, and show by example that without the condition it fails.

Exercise 16.3. Enumerate the configuration space for $|\text{Obl}| = 2$, $v \leq 2$ and verify Theorem 16.2 by exhaustive search over all traces of length at most eight.

Part V

From Verified Theorem to Scientific Claim

Chapter 17

Correspondence

The preceding parts concern what a verified result is and how it is produced, inherited and repaired. This part concerns the step that turns a verified theorem into a statement about the world, the step that Chapter 9 placed at layers O2 and O3 and that no kernel can check. The chapter does three things. It states what a correspondence claim consists of and proves that a claim with no preregistered failure condition is compatible with every observation (Section 17.1). It treats the question of whether parameters can be recovered from observations as a sufficiency question and proves the local and global versions of the answer to be genuinely different (Sections 17.2 and 17.3). And it works one complete example, the Burgers equation, in which a verifiable theorem has an explicit scope boundary that coincides exactly with the failure of a local-to-global argument (Section 17.4).

17.1 Correspondence claims and failure conditions

Definition 17.1 (Correspondence claim). A *correspondence claim* is a tuple $\mathcal{C} = (\varphi, \iota, \tau, \text{Obs}, \text{Fail})$ where

- φ is a formal statement with a checked proof (layer O1 discharged);
- ι is an *interpretation* that assigns to the formal symbols of φ quantities of a described system (layer O2);
- τ is an *operationalization*: a procedure that turns a record of measurements into values of the quantities assigned by ι (layer O3);
- Obs is the set of possible *observation records*; and
- $\text{Fail} \subseteq \text{Obs}$ is the *failure condition*, a decidable set fixed before observation.

The claim *survives* a record $r \in \text{Obs}$ if $r \notin \text{Fail}$ and *fails* on r otherwise.

Proposition 17.2 (Failure conditions carry the empirical content). (a) If $\text{Fail} = \emptyset$, the claim *survives every record*.

- (b) Let $\mathcal{C}$ and $\mathcal{C}'$ differ only in the failure condition, with $\text{Fail}' \subsetneq \text{Fail}$. Then every record on which $\mathcal{C}'$ fails is a record on which $\mathcal{C}$ fails, and any $r \in \text{Fail} \setminus \text{Fail}'$ is a record on which $\mathcal{C}$ fails and $\mathcal{C}'$ survives. In particular a failure condition modified after r was seen can convert failure into survival, and the two claims are different claims although the formal theorem φ is the same.
- (c) The survival relation is a function of the pair (Fail, r) ; it is not a function of r alone, nor of φ alone.

Proof. (a) $r \notin \emptyset$. (b) is immediate from $\text{Fail}' \subseteq \text{Fail}$. (c) Survival is $r \notin \text{Fail}$. Distinct

failure conditions give distinct survival functions, and φ does not appear in the definition of survival. ■

Part (b) is the formal content of the practice of timestamping the failure condition. A theorem together with an interpretation and an operationalization defines what is claimed only up to the failure condition; changing the latter after the fact changes the claim. This is the same observation as Proposition 9.10 in Chapter 9, now on the empirical side of the bridge.

17.2 Identifiability is sufficiency of the observation

A frequent correspondence claim is that parameters θ of a model can be recovered from observations $y = F(\theta)$. In the vocabulary of Chapter 10, let H be the parameter space, $F : H \rightarrow Y$ the observation map as a representation, and $\text{rec} : H \rightarrow H$ the identity, the task of recovering the parameter.

Proposition 17.3 (Identifiability). *The task rec is F -computable if and only if F is injective. For a weaker task $T : H \rightarrow Z$ (a function of the parameter, such as one coordinate or a derived quantity), T is recoverable from F if and only if T is constant on the fibres of F .*

Proof. Proposition 1.12 applied to the representation F . The first statement is the case $T = \text{id}$, for which constancy on fibres means that each fibre is a singleton. ■

Thus a proof that “the observations determine the parameters” is a proof of injectivity, and a proof that a particular quantity is determined is a proof of constancy on fibres. The difficulty in practice is that injectivity is a *global* property while the tools that prove it cheaply are local.

17.3 Local versus global injectivity

Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be C^1 .

Theorem 17.4 (Inverse function theorem). [CITED: ANY ANALYSIS TEXT, E.G. RUDIN, PRINCIPLES OF MATHEMATICAL ANALYSIS, THEOREM 9.24] *If the Jacobian $DF(x_0)$ is invertible, there are open neighbourhoods $U \ni x_0$ and $V \ni F(x_0)$ such that $F|_U : U \rightarrow V$ is a C^1 diffeomorphism. In particular F is injective on U .*

Theorem 17.5 (Hadamard–Caccioppoli). [CITED: HADAMARD 1906; CACCIOPPOLI 1932; SEE KRANTZ AND PARKS, THE IMPLICIT FUNCTION THEOREM, SECTION 6.2] *Let $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be C^1 with $\det DF(x) \neq 0$ for all x . If F is proper, meaning that preimages of compact sets are compact, then F is a C^1 diffeomorphism of $\mathbb{R}^n$ onto $\mathbb{R}^n$. In particular F is globally injective.*

The first theorem is local and requires only a condition at a point. The second is global and requires, in addition to the pointwise condition at every point, a topological condition on the whole map. The hypothesis of properness cannot be dropped, and the following is the standard example.

Proposition 17.6 (Everywhere locally invertible but not injective). *Let $F(x, y) = (e^x \cos y, e^x \sin y)$ on $\mathbb{R}^2$. Then $\det DF(x, y) = e^{2x} > 0$ for all (x, y) , F is injective on every strip $\{y_0 < y < y_0 + 2\pi\}$, and $F(x, y + 2\pi) = F(x, y)$ for all (x, y) . Hence F is a local diffeomorphism at every point, the task of recovering y is not F -computable, and the task of recovering $(x, y \bmod 2\pi)$ is.*

Proof. $DF = \begin{pmatrix} e^x \cos y & -e^x \sin y \\ e^x \sin y & e^x \cos y \end{pmatrix}$ has determinant $e^{2x}(\cos^2 y + \sin^2 y) = e^{2x}$. Writing F as the complex map $z = x + iy \mapsto e^z$, the identity $e^{z+2\pi i} = e^z$ gives periodicity, and $e^z = e^{z'}$ forces $z - z' \in 2\pi i\mathbb{Z}$, so F is injective on any strip of height $< 2\pi$ (and on any open strip of height exactly 2π). The fibres of F are therefore exactly the sets $\{(x, y + 2\pi k) : k \in \mathbb{Z}\}$, and a task is F -computable if and only if it is constant on these sets (Proposition 17.3); the coordinate y is not, and $(x, y \bmod 2\pi)$ is. ■

Corollary 17.7 (Properness fails). *The map F of Proposition 17.6 is not proper. In particular a proof that $\det DF \neq 0$ everywhere, however carefully verified, establishes no identifiability beyond the local statement of Theorem 17.4.*

Proof. If F were proper, Theorem 17.5 would make it injective, contradicting periodicity. Directly, $F^{-1}(\{(1, 0)\}) = \{(0, 2\pi k)\}$ is infinite and discrete, hence not compact. ■

A formally verified lemma of the form “the Jacobian of the observation map is nowhere singular” is exactly the kind of statement that proof assistants discharge well, and it is also exactly the statement that licenses, wrongly, the global conclusion. This is the paradigm of a scope error: the theorem is correct and the correspondence claim built on it is not. The remedy in the sense of Definition 17.1 is to state in the claim which of the two tasks is being asserted, local recovery within a neighbourhood or global recovery, and to attach a failure condition appropriate to it: a record that is consistent with two parameters at distance 2π in y is a failure of the global claim and not of the local one.

17.4 A complete example: the Burgers equation

The inviscid Burgers equation $u_t + u u_x = 0$ is the simplest nonlinear conservation law, and it exhibits the whole pattern of this chapter in a form that can be proved in a few lines. Let $u_0 : \mathbb{R} \rightarrow \mathbb{R}$ be C^1 and let $m = \inf_{\xi} u'_0(\xi)$. Assume $m > -\infty$. Put $t^* = -1/m$ if $m < 0$ and $t^* = \infty$ if $m \geq 0$.

Theorem 17.8 (Classical solutions up to the shock time). *For $0 \leq t < t^*$, the map $X_t(\xi) = \xi + t u_0(\xi)$ is a C^1 bijection of $\mathbb{R}$ with C^1 inverse, and $u(x, t) = u_0(X_t^{-1}(x))$ is a C^1 solution of $u_t + u u_x = 0$ with $u(\cdot, 0) = u_0$.*

Proof. For $t < t^*$ we have $X'_t(\xi) = 1 + t u'_0(\xi) \geq 1 + tm > 0$ when $m < 0$ and $X'_t \geq 1 > 0$ when $m \geq 0$; write $\kappa = \min(1, 1 + tm) > 0$. For $\xi > \eta$, $X_t(\xi) - X_t(\eta) \geq \kappa(\xi - \eta)$ by the mean value theorem, so X_t is strictly increasing with $X_t(\xi) \rightarrow \pm\infty$ as $\xi \rightarrow \pm\infty$; hence it is a bijection of $\mathbb{R}$, and its inverse is C^1 by Theorem 17.4. The function $u(X_t(\xi), t) = u_0(\xi)$ is therefore well defined and C^1 in (x, t) as a composition. Differentiating the identity $u(\xi + t u_0(\xi), t) = u_0(\xi)$ with respect to t at fixed ξ gives $u_t + u_x u_0(\xi) = 0$ at $x = X_t(\xi)$, and $u_0(\xi) = u(x, t)$ there, so $u_t + u u_x = 0$. ■

Note that the proof uses the local theorem at each point and, in addition, a global argument (the lower bound κ , which gives properness) to pass from local to global injectivity, exactly the structure of Theorem 17.5. The bound fails precisely at $t = t^*$.

Theorem 17.9 (No classical solution beyond the shock time). *Let $u_0(x) = -\tanh x$, so $m = -1$ and $t^* = 1$. For every $T > 1$ there is no C^1 solution of $u_t + uu_x = 0$ on $\mathbb{R} \times [0, T]$ with $u(\cdot, 0) = u_0$.*

Proof. Suppose u were such a solution. Along any curve $x'(t) = u(x(t), t)$ one has $\frac{d}{dt}u(x(t), t) = u_t + u_x x' = 0$, so u is constant along such curves. The curve starting at ξ at time 0 carries the value $u_0(\xi)$, hence has constant speed $u_0(\xi)$ and is the line $x = \xi + tu_0(\xi)$ (by uniqueness for the ODE, since u is C^1 and hence locally Lipschitz). For $t > 1$, $X_t(\xi) = \xi - t \tanh \xi$ has $X'_t(0) = 1 - t < 0$ and $X_t(\xi) \rightarrow \pm\infty$ as $\xi \rightarrow \pm\infty$. Since $X'_t(0) < 0$ there are $a < b$ with $X_t(a) > X_t(b)$. Since $X_t(c) \rightarrow +\infty$ as $c \rightarrow \infty$, some $c > b$ has $X_t(c) > X_t(a)$, and the intermediate value theorem gives $\xi_2 \in (b, c)$ with $X_t(\xi_2) = X_t(a)$. Put $\xi_1 = a$; then $\xi_1 \neq \xi_2$ and $X_t(\xi_1) = X_t(\xi_2)$. The two characteristics meet at a time $t \leq T$ at a point where u would equal both $u_0(\xi_1)$ and $u_0(\xi_2)$. But $u_0 = -\tanh$ is injective, a contradiction. ■

The two theorems together say: the formula of Theorem 17.8 is a correct formal result, it can be checked in a proof assistant, and its hypothesis $t < t^*$ is not a technicality but the boundary of the region in which the characteristic map is globally invertible. Beyond t^* the object that corresponds to physical experience is a weak solution selected by an entropy condition (Lax, Oleinik), a different mathematical object about which the above theorems say nothing [11]. A correspondence claim “the verified formula describes the gas” therefore needs a scope entry $t < t^*$ in its ledger (Chapter 9), a failure condition that contains records showing a discontinuity, and the bound $t^* = -1/m$, whose evaluation from data is an O3 obligation.

Example 17.10 (Numerical value). For $u_0 = -\tanh$, $u'_0 = -\text{sech}^2$ attains $m = -1$ at $\xi = 0$, so $t^* = 1$. At $t = 0.5$, $X'_t(\xi) = 1 - 0.5 \text{sech}^2 \xi \geq 0.5$, and X_t is a diffeomorphism. At $t = 1.5$, $X'_t(0) = -0.5 < 0$ and $X_t(\xi) = \xi - 1.5 \tanh \xi$ takes the value 0 at three points, $\xi = 0$ and $\xi = \pm\xi_0$, where $\xi_0 \approx 1.2878$ is the positive root of $\xi = 1.5 \tanh \xi$ (found by bisection). The three characteristics through $x = 0$ at $t = 1.5$ carry the three distinct values $u_0(0) = 0$ and $u_0(\pm\xi_0) = \mp \tanh \xi_0 \approx \mp 0.859$.

17.5 What this chapter fixes

- (1) A correspondence claim consists of a checked formal statement, an interpretation, an operationalization and a preregistered failure condition, and survival is a function of the failure condition (Proposition 17.2). (2) Identifiability of a parameter is injectivity of the observation map, and identifiability of a quantity is constancy on fibres (Proposition 17.3). (3) A pointwise condition such as a nonvanishing Jacobian proves local identifiability only; global identifiability requires a topological hypothesis in addition (Theorems 17.4 and 17.5, Proposition 17.6). (4) In the Burgers equation the boundary of validity of the verified formula is exactly the loss of global invertibility of the characteristic map (Theorems 17.8 and 17.9).

Exercise 17.1. Compute the unique positive root ξ_0 of $\xi = 1.5 \tanh \xi$ to four decimals and verify that $X_{1.5}$ takes the value 0 exactly at $\xi = 0, \pm \xi_0$.

Exercise 17.2. Prove that $F(x, y) = (e^x \cos y, e^x \sin y)$ restricted to $\mathbb{R} \times (0, 2\pi)$ is a diffeomorphism onto $\mathbb{R}^2$ minus the closed positive real axis, and identify what is lost in the corresponding observation task.

Exercise 17.3. State the analogue of Theorem 17.8 for $u_t + f(u)_x = 0$ with f smooth and $f'' > 0$, and identify the quantity that replaces m .

Chapter 18

Mutation of Verified Artifacts

Chapters 11 and 12 derived, from the structure of a checker, predictions about which dependents of a verified development survive a change. A prediction is a claim, and the way to test it is to change the development and see. This chapter formalizes that experiment: what a mutation is, what a preregistered prediction is, how predictions are scored and retained, and why a model that sees a dependency graph without edge types is not merely less accurate but *insufficient* for the task. It also treats the two audit rules that protect the experiment from a failure specific to proof assistants, namely that an artifact can be accepted while resting on a placeholder.

The experimental work that motivated this chapter, a frozen Lean 4 development, a preregistered matrix of twenty mutants over twelve operator classes, and a batch run over all of them, is described in the author’s separate manuscripts [39]. Per-mutant outcome tables are not reproduced here. What is established in this chapter is the mathematics of the experimental design and what the design can and cannot support.

18.1 Specimens, operators, outcomes

Definition 18.1 (Specimen, mutation, blast set). A *specimen* is a well-formed environment Γ in the sense of Section 12.4, together with a set Ax_0 of *declared* axioms. A *mutation operator* is a partial function μ taking a specimen and a target constant to a new environment $\Gamma' = \mu(\Gamma, c)$. The *blast set* of the mutant is

$$B(\Gamma, \Gamma') = \{e \text{ a constant of } \Gamma' : e \text{ does not check in } \Gamma'\},$$

where “checks” is as in Lemma 12.15. The *footprint change* is $\Delta Ax(e) = Ax_{\Gamma'}(e) \triangle Ax_{\Gamma}(e)$ for each constant e that checks in both.

An *operator class* groups operators by the kind of change: replacing a theorem body by another proof of the same type, replacing a definition body, changing a declared type, renaming, reordering, normalizing associativity, and so on.

Definition 18.2 (Prediction, correctness, soundness, precision). A *predictor* is a function $\hat{B}$ from (specimen, operator, target) to a set of constants. On a mutant it is *sound* if $B \subseteq \hat{B}$, *precise* if $\hat{B} \subseteq B$, and *correct* if both. A prediction concerning footprint is a prediction of ΔAx for each surviving constant.

Soundness and precision are independent: a predictor that always returns every constant is sound and imprecise, and one that always returns the empty set is precise whenever $B = \emptyset$ and unsound otherwise. The experimental question is whether a model yields predictions that are both.

18.2 Preregistration and retention

Definition 18.3 (Prediction log). A *prediction log* is an append-only sequence of entries $(id, model, pred, time_p, out, time_o)$, where $pred$ is a prediction made under the model $model$ at time $time_p$, and out is an observed outcome recorded at $time_o$. An entry is *preregistered* if $time_p < time_{mut}$, the time at which the mutant artifact first existed. An entry is *falsified under its model* if $pred \neq out$ (for the exact notion), and *retained* if it remains in the log unchanged. A *model amendment* adds a new model identifier; it never modifies existing entries.

Proposition 18.4 (Retention prevents retroactive correction). *In a prediction log satisfying the append-only rule:*

- (a) *the set of entries falsified under their models is nondecreasing in time;*
- (b) *for each mutant, the score of a model amended after $time_o$ on that mutant is a retrodiction, not a prediction: its $time_p$ is later than $time_o$, so the entry is not preregistered;*
- (c) *the proportion of preregistered predictions that were correct under model m is defined only for mutants whose entries for m precede the mutant artifact, and is therefore not affected by later amendments.*

Proof. (a) Entries are never modified or deleted, and falsification is a property of a single entry. (b) The amended model has $time_p > time_o \geq time_{mut}$. (c) is the definition of preregistration restricted to a model. ■

The proposition states, in the form of an invariant, the practice of preserving a miss as a miss. An amended model may correctly describe the mutants that falsified its predecessor; that correct description is evidence of fit and not of predictive ability. Evidence of predictive ability for the amended model requires *new* preregistered mutants.

18.3 A flat dependency graph is insufficient

The central methodological finding of the experimental work, as reported in the manuscripts cited above, is that a dependency representation that merges the relations of Definition 12.13 into one cannot support the predictions. This is a statement of Chapter 10, and it is provable.

Definition 18.5 (Flat and typed predictors). For the replacement of the body of a constant D by a term v' that checks at D 's type, define

$$\hat{B}_{\text{typed}}(D) = \begin{cases} \emptyset & D \text{ a theorem} \\ \{c : D \in \text{Cons}(c)\} & D \text{ a definition} \end{cases} \quad \hat{B}_{\text{flat}}(D) = \{c : D \in \text{cl}(c)\} \setminus \{D\},$$

with cl the flat dependency closure of Chapter 7 and Cons the consulted set of Definition 12.14.

Proposition 18.6 (Soundness and relation of the predictors). *Assume Convention 12.12. For body replacements that check at their type, $B \subseteq \widehat{B}_{\text{typed}}(D) \subseteq \widehat{B}_{\text{flat}}(D)$, with B the blast set excluding D . Both predictors are sound; $\widehat{B}_{\text{typed}}$ is at least as precise as $\widehat{B}_{\text{flat}}$ on every mutant, and strictly more precise on some.*

Proof. $B \subseteq \widehat{B}_{\text{typed}}$ is Theorem 12.16. $\text{Cons}(c) \subseteq \text{cl}(c)$ gives the second inclusion. For strictness, let D be a definition, T a theorem whose *value* mentions D , and c a constant that mentions T in its value but not otherwise D . Then $D \in \text{cl}(c)$ (flat) while $D \notin \text{Cons}(c)$ if D does not occur in the type of T or c , and c checks after the replacement by Lemma 12.15. ■

The soundness half assumes that the specimen is checked by a kernel satisfying the convention. Both predictors err only in the direction of over-prediction in this setting; a flat predictor that applied some rule *other* than “all reachable constants break”, for instance treating some edge types as safe, is not covered by the proposition, and the next result shows that no rule on the undifferentiated graph is right.

Proposition 18.7 (No predictor on the flat graph is correct). *There are two specimens Γ_1, Γ_2 with the same flat dependency graph, a constant c checking in both that depends on a constant X , and body replacements of X in each, such that $c \notin B(\Gamma_1, \Gamma'_1)$ and $c \in B(\Gamma_2, \Gamma'_2)$. Hence no function of the flat dependency graph alone, and of the identity of the mutated constant, is a correct predictor.*

Proof. Let Γ_1 contain a theorem $X : 0 = 0$ with value `rfl` and a constant $c : 0 = 0$ whose value is X . Let Γ_2 contain a definition $X : \mathbb{N} := 0$ and $c : X = 0$ with value `rfl`. In both specimens the flat graph consists of the single edge $c \rightarrow X$. Replace the body of X in Γ_1 by another proof of $0 = 0$, and in Γ_2 the body 0 by 1 . The first replacement leaves c checking (Lemma 12.15); the second breaks c , since `rfl` : $X = 0$ requires $X \equiv 0$ and $1 \not\equiv 0$. The two flat graphs are isomorphic by a relabelling that preserves the mutated node and the dependent, so a predictor depending only on the flat graph assigns c the same prediction in both. ■

In the vocabulary of Chapter 10, the flat graph is a representation whose kernel identifies mutants with different outcomes, and the outcome task is not served. The typed graph distinguishes the two by the edge label ($\rightarrow_{\text{int}}$ in Γ_1 , $\rightarrow_{\text{sem}}$ in the type of c in Γ_2), and Theorem 12.16 shows that the typed edges are sufficient for the *safe* direction. The author attributes two preregistration misses in the reported matrix (M09 and M16) to the flat model’s conflation of the three edge types, and the retention rule of the preceding section keeps them in the record as falsified under that model.

Remark 18.8 (Exactness is not claimed). Proposition 18.6 establishes soundness of $\widehat{B}_{\text{typed}}$, not precision: a definition in $\text{Cons}(c)$ whose replacement body is convertible with the old need not break anything. Whether a replacement body is convertible with the old one is a question about the checker’s conversion relation, and answering it in general requires running the checker.

18.4 Footprints and the placeholder audit

Proof assistants admit a state that has no analogue in the logical systems of Part I: a declaration may be accepted while resting on a placeholder. In Lean, an elaborator that cannot

complete a proof may insert sorryAx , a constant of every type, and report a warning while the declaration is added to the environment with a value (Chapter 7, Proposition 7.8). Hence the presence of a value for a theorem is not evidence that it has been proved.

Definition 18.9 (Audited verification). A constant c is *audited-verified* in Γ relative to the declared axioms Ax_0 if (i) c checks in Γ , and (ii) $\text{Ax}_\Gamma(c) \subseteq \text{Ax}_0$, where $\text{sorryAx} \notin \text{Ax}_0$ and any axiom of native reduction is counted only if it is in Ax_0 .

Proposition 18.10 (Properties of the audit). (a) If c is *audited-verified* and $e \in \text{cl}(c)$ is a theorem or definition, then $\text{Ax}_\Gamma(e) \subseteq \text{Ax}_0$.

- (b) The audit must be computed on the final environment's closure. A syntactic scan of the declaration c alone, or the observation that c has a value, is not sufficient: the constants c depends on may carry placeholders that c 's own text does not mention.
- (c) Replacing the body of a theorem T by another proof of the same type does not change whether the dependents of T check (Lemma 12.15), but can change whether they are *audited-verified*.

Proof. (a) $\text{cl}(e) \subseteq \text{cl}(c)$ for $e \in \text{cl}(c)$, so $\text{Ax}(e) \subseteq \text{Ax}(c) \subseteq \text{Ax}_0$. (b) Take c whose text mentions only a theorem T with value mentioning sorryAx ; $\text{Ax}(c) \ni \text{sorryAx}$ while the text of c does not. (c) This is Proposition 12.17. ■

The proposition is the reason the audit in the experimental work reads axiom footprints through the environment rather than extracting dependencies from declaration data, and reports footprint changes as a separate outcome from the check.

18.5 A model-level example: associative normalization

The experimental matrix contains an operator class that rewrites a statement by associative normalization. The reported outcome, in the author's words, is that the PRESERVED verdict of Chapter 12 concerns *semantic* correspondence and never survival of the proof term: the same proof breaks, though the claim is logically equivalent. The general form of this is already proved.

Proposition 18.11 (Semantic preservation does not give term survival). Let R be a transformation with $\text{Vd}(R, P) = \text{PRESERVED}$ in a certified fragment and let t be a proof term with $\text{Chk}(t, P)$. If $\text{NF}(R(P)) \neq \text{NF}(P)$ then $\neg \text{Chk}(t, R(P))$.

Proof. Proposition 12.10. ■

Example 18.12 (Commuting a conjunction). $p \wedge q \leftrightarrow q \wedge p$ is a tautology, and the transformation $R(p \wedge q) = q \wedge p$ is PRESERVED in the fragment of Example 12.8. The pair term $\langle h_p, h_q \rangle$ has type $p \wedge q$, and $p \wedge q \not\equiv q \wedge p$ definitionally, since the two types are distinct normal forms; so the term is not a proof of $R(p \wedge q)$, and a new term $\langle h_q, h_p \rangle$ is needed.

18.6 What a matrix of this size can support

A preregistered matrix of n mutants partitions its operator classes into at most n cells. A predictor that agrees with the observed outcomes on all n mutants is indistinguishable, by this matrix, from every other predictor with the same values on those mutants, and

a class represented by a single mutant is represented by a single point: agreement on it cannot separate a model that is right for the class from one that is right for the instance. In the language of Chapter 10, the matrix is a task family, and its closure is exactly the set of functions of the n outcomes. Claims about a class that go beyond the mutants of that class are not supported by the matrix, whatever its size elsewhere. No numerical statistic over the twenty mutants is computed in this book.

18.7 What this chapter fixes

(1) A mutation experiment is a function from mutants to blast sets, and a prediction is sound, precise or correct (Definition 18.2). (2) Retention of falsified predictions is an invariant of an append-only log, and a model amended after the fact is evaluated by retrodiction (Proposition 18.4). (3) The flat dependency graph does not determine the outcome of body replacements, so no flat predictor is correct; typed edges give a sound predictor (Propositions 18.6 and 18.7). (4) Placeholder audits read footprints on the final environment and treat footprint change as a distinct outcome (Proposition 18.10). (5) Semantic preservation of a statement does not carry the proof term (Proposition 18.11).

Exercise 18.1. Prove that in Proposition 18.7 the two specimens can be chosen with literally identical flat graphs, including the label of the mutated node.

Exercise 18.2. State and prove the analogue of Theorem 12.16 for mutations that change the *type* of a constant, and use it to define a typed predictor for type mutations.

Exercise 18.3. For every n , give a predictor that is correct on all n mutants of a given matrix and incorrect on some mutant outside it, and explain what this says about the use of agreement on a fixed matrix as evidence of a model.

Chapter 19

A Negative Result as Method

The preceding chapters describe what a verified result owes the reader. This chapter describes what a *negative* result owes, using as its worked case a reported simulation study in which a proposed decomposition of a physical phenomenon was tested and did not earn its place [40]. The study concerns the onset of motility-induced phase separation and tests whether features named capacity and dissipation add predictive information beyond density. The findings reported there, which are simulation findings and not physical laboratory replication, are three: the simulation engine was validated; the decomposition supplied no incremental predictive value beyond density that survived replication; and an intervention pilot was underpowered, leaving causal content unresolved. Numbers from that study are not reproduced here. The chapter proves what each of the three statements does and does not license, in a form independent of the particular system.

19.1 Two questions that must not be merged

Let θ range over parameters of a simulated system and let $E(\theta)$ be an outcome produced by the engine. Let $b(\theta)$ be a baseline feature (here, density) and $\phi(\theta)$ a proposed additional feature.

Definition 19.1 (Engine validation and decomposition evaluation). *Engine validation* is the claim that E computes the intended model: on a benchmark set $\mathcal{B}$ of problems with independently known answers, E agrees with them to a declared tolerance. This is a claim of the layers O1 and O2 of Chapter 9 for the engine. *Decomposition evaluation* is the claim that the features (b, ϕ) stand in a declared relation to the outcome, here that ϕ adds predictive information about E beyond b .

Proposition 19.2 (The two are independent). *Engine validation neither implies nor is implied by a positive decomposition claim. In particular a validated engine can yield a decomposition that adds nothing, and an engine failing validation can yield a decomposition that appears to add predictive value.*

Proof. Validation concerns the map $\theta \mapsto E(\theta)$ against $\mathcal{B}$. The decomposition claim concerns the joint law of $(E(\theta), b(\theta), \phi(\theta))$ under a sampling distribution of θ . Take an engine that computes $E(\theta) = g(b(\theta))$ exactly: it is validated by construction against any benchmark that g solves, and ϕ adds nothing under every distribution. Take an engine with a bug that

makes $E(\theta) = g(b(\theta)) + \phi(\theta)$ though the model prescribes $g(b(\theta))$: it fails validation and ϕ adds predictive value. ■

The separation is not a courtesy. A negative decomposition result is informative only if the engine is trusted, and a positive one is informative only if the same holds; but neither follows from the other, and a report that merges them cannot be read.

19.2 Incremental predictive value

Fix a loss ℓ and a class $\mathcal{F}_b$ of predictors using b only, and a class $\mathcal{F}_{b,\phi} \supseteq \mathcal{F}_b$ using both. The *incremental value* is

$$\Delta(\mathcal{F}_b, \mathcal{F}_{b,\phi}) = \inf_{f \in \mathcal{F}_b} \mathbb{E} \ell(y, f(b)) - \inf_{f \in \mathcal{F}_{b,\phi}} \mathbb{E} \ell(y, f(b, \phi)) \geq 0,$$

the inequality holding because the second infimum is over a larger class. For the squared loss and unrestricted classes it has a closed form.

Theorem 19.3 (Incremental value under squared loss). *Let y have finite variance and let $\mathcal{F}_b, \mathcal{F}_{b,\phi}$ be all square-integrable functions of b , respectively of (b, ϕ) . Then*

$$\Delta = \mathbb{E}[(\mathbb{E}[y | b, \phi] - \mathbb{E}[y | b])^2],$$

and $\Delta = 0$ if and only if $\mathbb{E}[y | b, \phi] = \mathbb{E}[y | b]$ almost surely.

Proof. The minimizers over the two classes are the conditional expectations. Write $\mu_b = \mathbb{E}[y | b]$ and $\mu_{b\phi} = \mathbb{E}[y | b, \phi]$. Then $y - \mu_b = (y - \mu_{b\phi}) + (\mu_{b\phi} - \mu_b)$ and the cross term has expectation $\mathbb{E}[(\mu_{b\phi} - \mu_b) \mathbb{E}[y - \mu_{b\phi} | b, \phi]] = 0$ since $\mu_{b\phi} - \mu_b$ is a function of (b, ϕ) . Hence $\mathbb{E}(y - \mu_b)^2 = \mathbb{E}(y - \mu_{b\phi})^2 + \mathbb{E}(\mu_{b\phi} - \mu_b)^2$, which is the formula. The second term vanishes iff $\mu_{b\phi} = \mu_b$ a.s. ■

The condition $\mathbb{E}[y | b, \phi] = \mathbb{E}[y | b]$ is the distribution-level form of the factorization criterion of Chapter 1: the task “predict the mean of y ” factors through b . A null result in the sense of the theorem is a statement of b -sufficiency for that task. It is a statement about *conditional means*, and about all functions of b and (b, ϕ) .

Proposition 19.4 (A null in a restricted class is not a null). *Let b, ϕ be independent, mean zero, with finite nonzero variances $\sigma_b^2, \sigma_\phi^2$, and let $y = b\phi$. Let $\mathcal{F}_b^{\text{lin}}$ and $\mathcal{F}_{b\phi}^{\text{lin}}$ be the affine predictors. Then $\Delta^{\text{lin}} = 0$, while the incremental value over unrestricted classes is $\Delta = \sigma_b^2 \sigma_\phi^2 > 0$.*

Proof. $\mathbb{E}[y] = 0$, $\text{Cov}(y, b) = \mathbb{E}[b^2 \phi] = \mathbb{E}[b^2] \mathbb{E}[\phi] = 0$ and $\text{Cov}(y, \phi) = \mathbb{E}[b \phi^2] = 0$. So the best affine predictors with and without ϕ are both the constant 0, and the risks coincide. For unrestricted classes, $\mathbb{E}[y | b] = b \mathbb{E}[\phi] = 0$ and $\mathbb{E}[y | b, \phi] = b\phi$, so by Theorem 19.3 $\Delta = \mathbb{E}[(b\phi)^2] = \sigma_b^2 \sigma_\phi^2$. ■

The proposition shows that “no incremental predictive value” is a statement relative to the predictor class and to the range over which (b, ϕ) was sampled, in the sense of declared task families in Chapter 10. It is exactly the situation of Chapter 11: a representation that is sufficient for one family (affine prediction) is not sufficient for another. A negative result is therefore reported with its class, loss and range, and what it excludes is the incremental value for that class and range.

Replication and preregistration

An estimate $\hat{\Delta}$ computed on a finite sample can be negative, since the fitted larger class need not beat the fitted smaller class on held-out data. A claim that ϕ adds value therefore needs a decision rule: a threshold $\tau > 0$ and a number of independent replicate datasets, fixed in advance (Definition 17.1). The claim *survives replication* if $\hat{\Delta} > \tau$ on every replicate; it *fails* otherwise. By Proposition 17.2, the same data evaluated under a different threshold or a different number of replicates is a different claim.

19.3 Underpowered interventions

Prediction is not intervention. That ϕ adds no predictive value beyond b does not say whether setting ϕ would change the outcome, and the reverse also holds. The causal question requires an intervention experiment, and the following statement shows what a small one can say.

Definition 19.5 (Two-arm intervention estimate). Two arms of n independent runs each set ϕ to two values; outcomes in each arm are independent with variance σ^2 , and let the effect be $\delta = \mu_1 - \mu_0$. Let $\hat{\delta}$ be the difference of sample means, and for $z > 0$ the interval $\text{CI} = [\hat{\delta} - w, \hat{\delta} + w]$ with $w = z\sigma\sqrt{2/n}$. Under normality with known σ and $z = 1.96$, this has coverage 0.95.

Definition 19.6 (Three-valued verdict for an effect). Fix a preregistered margin $\varepsilon > 0$ of practically relevant effect. The verdict is

- SUPPORTED if $\text{CI} \subseteq (\varepsilon, \infty)$ or $\text{CI} \subseteq (-\infty, -\varepsilon)$;
- EXCLUDED_AT_MARGIN if $\text{CI} \subseteq (-\varepsilon, \varepsilon)$;
- UNRESOLVED otherwise.

Theorem 19.7 (What a small intervention can decide). (a) *The three verdicts are mutually exclusive and exhaustive.* (b) *If the interval covers δ , then SUPPORTED implies $|\delta| > \varepsilon$ and EXCLUDED_AT_MARGIN implies $|\delta| < \varepsilon$.* (c) *EXCLUDED_AT_MARGIN is impossible unless $w < \varepsilon$, that is, unless*

$$n > 2\left(\frac{z\sigma}{\varepsilon}\right)^2;$$

and SUPPORTED is impossible unless $|\hat{\delta}| > w$. In particular for $n \leq 2(z\sigma/\varepsilon)^2$ the verdict is never EXCLUDED_AT_MARGIN.

Proof. (a) A nonempty interval contained in $(-\varepsilon, \varepsilon)$ cannot lie in (ε, ∞) or $(-\infty, -\varepsilon)$, and those two are disjoint, so the first two verdicts exclude each other; UNRESOLVED is defined as the complement. (b) If $\delta \in \text{CI}$ and $\text{CI} \subseteq (\varepsilon, \infty)$ then $\delta > \varepsilon$; similarly for the others. (c) $\text{CI} \subseteq (-\varepsilon, \varepsilon)$ requires its length $2w < 2\varepsilon$, and $w = z\sigma\sqrt{2/n} < \varepsilon$ iff $n > 2(z\sigma/\varepsilon)^2$. CI avoiding 0 requires $|\hat{\delta}| > w$. ■

The theorem formalizes “underpowered”: a pilot with $n \leq 2(z\sigma/\varepsilon)^2$ runs per arm cannot decide that the effect is negligible at the margin ε , whatever its outcome, and can support an effect only by observing one larger than w . A pilot of that size that fails to reach SUPPORTED has produced the verdict UNRESOLVED, and that verdict is a result: it excludes the effects outside CI and no others. Reporting it as “no effect” would be the error of treating the failure to certify as certification of failure.

Example 19.8 (Arithmetic). With $z = 1.96$, $\sigma = 1$ and $\varepsilon = 0.5$ the bound is $n > 2 \cdot (1.96/0.5)^2 = 2 \cdot 15.37 \approx 30.7$, so at least 31 runs per arm are required before a null can be declared negligible at that margin. At $n = 10$ per arm, $w = 1.96\sqrt{0.2} \approx 0.877 > 0.5$, so the verdict cannot be EXCLUDED_AT_MARGIN. The numbers illustrate the formula and are not those of any study.

19.4 The ledger of a negative result

Table 19.1 records the claims of the reported study in the vocabulary of this book. The statuses are those reported by the author for the simulations, and they are listed by type of claim and not by magnitude.

Claim	Status	Governing result
The simulation engine implements the intended model	validated against benchmarks (reported)	Definition 19.1
Capacity and dissipation add predictive value beyond density	not supported; null survived replication within the tested class and range (reported)	Thm. 19.3, Prop. 19.4
Intervening on the proposed features changes the outcome	unresolved; pilot underpowered (reported)	Thm. 19.7
Results hold for physical systems	outside the scope of the study; simulation only (reported)	Chapter 17

Table 19.1: Ledger for the reported study. The statuses distinguish validated, not supported, unresolved and out of scope.

The ledger shows four distinct outcomes in a single study, none of which is a plain success or failure. A result of this kind constrains the framework that proposed the decomposition, and the way to keep it as evidence is the way Chapter 18 keeps a miss: retain it in the record, under the model that produced the prediction, and treat later reformulations as new claims requiring new data.

19.5 What this chapter fixes

- (1) Validation of the engine and evaluation of the decomposition are independent claims (Proposition 19.2). (2) Under squared loss, the incremental value of a feature is the second moment of the change in the conditional mean, and a null means b -sufficiency for that task (Theorem 19.3); in a restricted class, a null does not exclude value (Proposition 19.4). (3) A pilot below the sample size $2(z\sigma/\varepsilon)^2$ per arm cannot exclude an effect of size ε , and the verdict UNRESOLVED is a legitimate and informative outcome (Theorem 19.7). (4) A negative result is retained as evidence and is not removed by reformulating the claim.

Exercise 19.1. Show that Theorem 19.3 also holds when ϕ is vector valued, and state what changes if y is vector valued with a quadratic loss $\|y - f\|_M^2$ for a positive definite matrix M .

Exercise 19.2. For two arms of unequal sizes n_0, n_1 show that $w = z\sigma\sqrt{1/n_0 + 1/n_1}$, and find the allocation of a fixed total $n_0 + n_1$ that minimizes w .

Exercise 19.3. Construct, in the setting of Proposition 19.4, a nonlinear class $\mathcal{F}_{b\phi}$ of two parameters under which $\Delta > 0$, and compute it.

Part VI

Synthesis

Chapter 20

What a Verified Result Owes

The book began from a pair of questions: what does a verification assure, and what must a verified result keep in order to remain useful? The chapters between give answers in pieces, each proved where it is stated. This chapter collects them as three principles, converts them into a list of items that a verified result should carry, and states plainly which parts of the book are proved, which are cited, which were checked by computation, and which are proposals.

20.1 Three principles

Principle 1: Assurance attaches to the checker and the bridge, never to the producer, and failure to certify is not certification of failure.

A proof system is a producer together with a decidable checking relation (Definition 1.1); soundness of the producer is irrelevant to the assurance (Theorem 1.5); the assurance of a translated result is the conjunction of the checker's and the translation's (Proposition 1.7); a verified statement is bridged to an observation through layers O1 to O3 of which a kernel can discharge only the first (Chapter 9). A report of preservation has three values, and the third is not a failure (Theorem 12.6). A gate that sees only what the proposer sees cannot discriminate what the proposer cannot (Theorem 15.8).

Principle 2: Sufficiency is always sufficiency for a declared task, and the declaration is part of the result.

A representation serves a task exactly when the task is constant on its fibres (Proposition 1.12, Theorem 10.4); a declared family has a minimal artifact and a closure of served tasks (Theorem 10.7); every non-injective representation fails some task (Theorem 10.12). A certificate is a region and a verdict is a point (Theorems 11.2 and 11.10); repair is a relation, and local repair is impossible from a representation that identifies distant artifacts (Theorems 11.13 and 11.15).

Principle 3: A collapse is admissible only relative to a declared task and the refusals on record.

In a search, the option space is changed only by refusal and collapse (Proposition 13.5); a refusal is lossless iff the refused item is redundant for the declared option closure (Theorem 14.4); fair runs with admissible deletion remain complete (Theorem 14.7); ordering guidance does not change what is obtainable (Theorem 15.2); verifiers report and never dispose, and a failed verification is a live state (Theorems 16.3, Proposition 16.4). The history that a collapse forgets is exactly what repair and audit later require (Proposition 13.11).

The principles are consequences of a single observation, that a function factors through a quotient if and only if it respects the equivalence the quotient imposes. Their content is what follows when the observation is applied to the objects of automated reasoning: runs, certificates, worlds, dependency graphs, observation records.

20.2 The result card

A verified result that is to remain scientifically useful carries, in addition to its statement and derivation, the items of Table 20.1. Each is tied to the chapter where it is defined and to the principle it discharges. The list is not a template to be filled but a set of questions to be answerable.

	Item	Where defined	Principle
1	Statement and checked derivation (O1)	Ch. 1, 7	1
2	Formalization of the target and units (O2)	Ch. 9	1
3	Interpretation, operationalization, preregistered failure condition (O3)	Ch. 17	1
4	Declared task family and the minimal artifact serving it	Ch. 10	2
5	Region of validity: certificate regions or stability margin	Ch. 11	2
6	Typed dependencies and axiom footprint audited on the final environment	Ch. 12, 18	1, 3
7	Search record sufficient for the declared diagnostic and repair tasks (history, refusals, provenance)	Ch. 13, 14	3
8	Lifecycle record separating verification from disposition	Ch. 16	3
9	Retained negative and unresolved outcomes	Ch. 18, 19	1, 3

Table 20.1: The items a verified result owes.

None of the items requires a particular tool. Several, notably items 4, 5 and 7, are cheap to record at the time of the run and impossible to reconstruct afterwards (Proposition 13.11). The bearing of Principle 2 on item 4 is that the card is incomplete without a statement of which tasks the artifact is claimed to serve, since by Theorem 10.12 an artifact that serves all of them is the whole history.

20.3 What is proved, cited, computed and proposed

Proved in the text. The factorization criterion and independence of soundness (Ch. 1); fair derivations and the consequences of Church’s theorem (Ch. 2); propositional resolution completeness by elimination, unification, first-order resolution modulo cited Herbrand and lifting theorems (Ch. 3); Newman’s lemma, termination by interpretation, the critical pair lemma (Ch. 4); the CDCL transition system, RUP, RAT (Ch. 5); congruence closure, Farkas certificates, the Nelson–Oppen counterexample (Ch. 6); the LCF theorem and footprints (Ch. 7); certificate schemes (Ch. 8); dimensional homogeneity and the obligation ledger (Ch. 9); the Galois connection, lower bounds, no free sufficiency, and the realizability criterion and residue theorem for exterior data of a flat sheet, modulo the cited extension and isometric-immersion theorems (Ch. 10); Farkas geometry, margins, RUP monotonicity, relational factorization and locality (Ch. 11); undecidability of preservation, certified fragments, the interface lemma and blast radius (Ch. 12); the world model, redundancy closure, completeness with deletion, gate theorems, the lifecycle machine (Ch. 13–16); identifiability and the Burgers example (Ch. 17); soundness of the typed predictor and insufficiency of the flat one (Ch. 18); incremental value and power (Ch. 19).

Cited and not proved. The theorems of Appendix A.

Assumed as design properties of kernels. Convention 12.12 (consultation of types and values, opacity of theorems); decidability and soundness of real kernels (Chapter 7).

Checked by computation, not mechanized. The 55 critical pairs of the group rewriting system (Ch. 4); preservation of satisfiability by variable elimination and the reconstruction rule on random clause sets (Ch. 5); the Farkas example and margins on a grid (Ch. 11); the RUP derivations of Example 11.11; closure properties of the redundancy closure on random clause sets (Ch. 14); the lifecycle machine by exhaustive enumeration of traces (Ch. 16); the Galois connection of Chapter 10 on small sets. These are checks of statements whose proofs are in the text and are not substitutes for them. No theorem of this book has been formalized in a proof assistant.

Reported, not reproduced. The mutation matrix and its outcomes (Ch. 18) and the study of Chapter 19, both from the author’s separate manuscripts. Statements about them in this book are statements of what was reported; the chapters prove what such reports can and cannot support.

Proposed. Deletion records generating repair obligations (Remark 14.9); the use of the framework for guided proof search beyond the propositional case; the scoped-certificate structure described below.

20.4 Open directions

Mechanization. The lifecycle machine, the interface lemma and the stability results of Chapter 11 are small enough to be formalized. A formalization of the lifecycle in a proof assistant would also be a test of the claims of Chapter 18: the typed predictor makes a prediction about the effect of mutating that very development.

Beyond propositional deletion. Theorem 14.7 uses finiteness of the clause universe. For first-order resolution the persistence argument needs a well-founded ordering on clauses and redundancy defined relative to smaller clauses, as in the model-based completeness theorems of Bachmair and Ganzinger [2]. Combining that machinery with the world model, with refusal records as first-class objects, is a task of the same kind as Chapter 14 at greater technical cost.

Scoped certificates and gluing. A certificate with its scope can be regarded as a local object over the regime in which it was verified, with restriction to a smaller regime (a narrower scope, a subset of the assumptions) and compatibility conditions on overlaps (agreement on shared variables and assumptions). Whether the resulting structure supports a useful gluing theorem, and whether an obstruction to gluing is detected by cohomological invariants, depends on a particular choice of base space and coefficients, which has not been constructed in this book. The proposal is recorded here as a programmatic remark and is not a result.

Certified fragments of practical interest. Example 12.9 gives a complete fragment for propositional rewriting. The corresponding fragments for linear arithmetic, bit-vectors and rewriting modulo theories, with certificates in the sense of Definition 12.5, are the natural next cases.

Exact prediction. Proposition 18.6 gives a sound predictor of the blast set and no more. Exact prediction for definitional unfolding requires deciding convertibility of replacement bodies, which is checker-dependent.

20.5 Conclusion

A verification is a function from a statement and a derivation to a verdict. Everything that makes the verdict *useful*, that it be about the right statement, that it survive what changes, that it be repairable, auditable and extendable, is information that the function does not return and that the run, the dependency graph and the surrounding record contain or lose. The preceding chapters show that this information has precise structure: partitions and their refinements, regions and their margins, options and their refusals, versions and their obligations. A result that carries it can be reused; one that does not can be rechecked and nothing more. The contribution of the book is to say exactly which information, for which task, and what it costs to keep.

The related manuscripts of the author on binding, disposition and foundations [35, 36, 37] supply the vocabulary of worlds, refusal and collapse, and the discussion of conventions in logic that underlies the treatment of kernels in Part II. They are cited for provenance and are not used in any proof.

Appendix A

External Theorems Used

The following results are used in the text and are stated here as they are used. They are cited, not proved. Entries are in the order of first use. Where the text proves a special case (Newman’s lemma, propositional resolution completeness, the Nelson–Oppen counterexample), the proof is in the chapter indicated.

A.1 Church and Turing (Chapter 2, 12).

The set of valid first-order sentences over a signature containing a binary relation symbol is recursively enumerable and not decidable [7, 34].

A.2 Compactness (Chapter 3).

A set of first-order sentences has a model if and only if each finite subset does (Gödel; Mal’cev).

A.3 Herbrand’s theorem (Chapter 3).

A set of universal clauses is unsatisfiable if and only if some finite set of its ground instances is propositionally unsatisfiable [15].

A.4 Lifting lemma (Chapter 3).

If clauses C', D' are ground instances of C, D by a common substitution and E' is a resolvent of C', D' , then C, D have a resolvent E (using a most general unifier) of which E' is an instance [27].

A.5 Birkhoff’s completeness theorem (Chapter 4).

An equation $s \approx t$ holds in all models of an equational theory E if and only if it is derivable from E by the rules of equational logic [4].

A.6 Knuth–Bendix completion (Chapter 4).

Given a finite set of equations and a reduction order, a fair run of the completion procedure that does not fail on an unorientable equation produces a convergent rewrite system generating the same equational theory [17, 16, 3].

A.7 Tseitin transformation (Chapter 12).

Every propositional formula has an equisatisfiable clause form of size linear in the formula, over extra variables [33].

A.8 Cook–Reckhow (Chapter 8).

There exists a propositional proof system in which every tautology has a proof of polynomial size if and only if $\text{NP} = \text{coNP}$ [9].

A.9 Haken (Chapter 5, 8).

Resolution refutations of the clause sets expressing the pigeonhole principle have size exponential in the number of pigeons [13].

A.10 CDCL and resolution (Chapter 5).

Conflict-driven clause learning with nondeterministic decisions and restarts polynomially simulates general resolution [26, 1].

A.11 Nelson–Oppen (Chapter 6).

For quantifier-free theories over disjoint signatures that are stably infinite, satisfiability of the combination is decidable whenever it is for each component, by exchanging equalities between shared variables [24, 32].

A.12 DPLL(T) (Chapter 6).

The abstract lazy architecture is sound and complete relative to a theory solver that returns theory lemmas [25].

A.13 Farkas’ lemma (Chapter 6, 11).

The system $Ax \leq b$ has no real solution if and only if there is $y \geq 0$ with $y^\top A = 0$ and $y^\top b < 0$ [12, 31].

A.14 Minkowski–Weyl (Chapter 11).

A convex cone is polyhedral (defined by finitely many linear inequalities) if and only if it is finitely generated [31].

A.15 Separation of convex sets (Chapter 11).

Two disjoint nonempty convex subsets of $\mathbb{R}^m$, one of them open, are separated by a hyperplane: there are $\lambda \neq 0$ and α with $\lambda^\top x \leq \alpha \leq \lambda^\top y$ for x in the first set and y in the second [28].

A.16 Rice’s theorem (Chapter 12).

Every nontrivial extensional property of partial computable functions, given by programs, is undecidable [29].

A.17 Cook’s theorem (Chapter 14).

Propositional satisfiability is NP-complete, so propositional entailment is coNP-complete [8].

A.18 Data-processing inequality (Chapter 10).

For a Markov chain $X \rightarrow Y \rightarrow Z$, $I(X; Z) \leq I(X; Y)$ [10].

A.19 Buckingham π theorem (Chapter 9).

A physically meaningful relation among n quantities expressed in k independent dimensions can be rewritten as a relation among $n - k$ dimensionless products [5].

A.20 Inverse function theorem (Chapter 17).

If a C^1 map has invertible derivative at a point, it is a C^1 diffeomorphism between neighbourhoods of the point and its image [30].

A.21 Hadamard–Caccioppoli (Chapter 17).

A C^1 map $\mathbb{R}^n \rightarrow \mathbb{R}^n$ with everywhere invertible derivative that is proper is a C^1 diffeomorphism onto $\mathbb{R}^n$ [14, 6, 18].

A.22 Model-based completeness of saturation (Chapter 14, remark).

For ordered resolution with redundancy, saturated unsatisfiable clause sets contain the empty clause [2].

A.23 Kirszbraun's extension theorem (Chapter 10).

A 1-Lipschitz map from a subset of a Euclidean space into a Euclidean space extends to a 1-Lipschitz map on the whole space [20].

A.24 Nash–Kuiper, in Gromov's form (Chapter 10).

Let (V, g) be a compact Riemannian manifold with boundary of dimension n and $q > n$. Every smooth strictly short map $V \rightarrow \mathbb{R}^q$ (one whose pullback of the Euclidean metric is smaller than g) is uniformly approximable by C^1 isometric immersions [22, 21, 19].

A.25 Tarski's quantifier elimination (Chapter 10).

The first-order theory of real closed fields is decidable [23].

Bibliography

- [1] A. Atserias, J. K. Fichte, M. Thurley. Clause-learning algorithms with many restarts and bounded-width resolution. *Journal of Artificial Intelligence Research* 40 (2011), 353–373.
- [2] L. Bachmair, H. Ganzinger. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation* 4(3) (1994), 217–247.
- [3] L. Bachmair, N. Dershowitz, J. Hsiang. Orderings for equational proofs. *Proc. 1st IEEE Symposium on Logic in Computer Science* (1986), 346–357.
- [4] G. Birkhoff. On the structure of abstract algebras. *Proceedings of the Cambridge Philosophical Society* 31 (1935), 433–454.
- [5] E. Buckingham. On physically similar systems; illustrations of the use of dimensional equations. *Physical Review* 4(4) (1914), 345–376.
- [6] R. Caccioppoli. Sugli elementi uniti delle trasformazioni funzionali. *Rendiconti del Seminario Matematico della Università di Padova* 3 (1932), 1–15.
- [7] A. Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics* 58 (1936), 345–363.
- [8] S. A. Cook. The complexity of theorem-proving procedures. *Proc. 3rd ACM Symposium on Theory of Computing* (1971), 151–158.
- [9] S. A. Cook, R. A. Reckhow. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic* 44(1) (1979), 36–50.
- [10] T. M. Cover, J. A. Thomas. *Elements of Information Theory*, 2nd ed. Wiley, 2006.
- [11] L. C. Evans. *Partial Differential Equations*, 2nd ed. American Mathematical Society, 2010.
- [12] J. Farkas. Theorie der einfachen Ungleichungen. *Journal für die reine und angewandte Mathematik* 124 (1902), 1–27.
- [13] A. Haken. The intractability of resolution. *Theoretical Computer Science* 39 (1985), 297–308.
- [14] J. Hadamard. Sur les transformations ponctuelles. *Bulletin de la Société Mathématique de France* 34 (1906), 71–84.
- [15] J. Herbrand. *Recherches sur la théorie de la démonstration*. Thesis, Université de Paris, 1930.
- [16] G. Huet. A complete proof of correctness of the Knuth–Bendix completion algorithm. *Journal of Computer and System Sciences* 23 (1981), 11–21.
- [17] D. E. Knuth, P. B. Bendix. Simple word problems in universal algebras. In: J. Leech (ed.), *Computational Problems in Abstract Algebra*, Pergamon, 1970, 263–297.

- [18] S. G. Krantz, H. R. Parks. *The Implicit Function Theorem: History, Theory, and Applications*. Birkhäuser, 2013.
- [19] Y. Eliashberg, N. Mishachev. *Introduction to the h -Principle*. American Mathematical Society, 2002.
- [20] M. D. Kirszbraun. Über die zusammenziehende und Lipschitzsche Transformationen. *Fundamenta Mathematicae* 22 (1934), 77–108.
- [21] N. H. Kuiper. On C^1 -isometric imbeddings I, II. *Indagationes Mathematicae* 17 (1955), 545–556, 683–689.
- [22] J. Nash. C^1 isometric imbeddings. *Annals of Mathematics* 60 (1954), 383–396.
- [23] A. Tarski. *A Decision Method for Elementary Algebra and Geometry*, 2nd ed. University of California Press, 1951.
- [24] G. Nelson, D. C. Oppen. Simplification by cooperating decision procedures. *ACM Transactions on Programming Languages and Systems* 1(2) (1979), 245–257.
- [25] R. Nieuwenhuis, A. Oliveras, C. Tinelli. Solving SAT and SAT modulo theories: from an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T). *Journal of the ACM* 53(6) (2006), 937–977.
- [26] K. Pipatsrisawat, A. Darwiche. On the power of clause-learning SAT solvers as resolution engines. *Artificial Intelligence* 175(2) (2011), 512–525.
- [27] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM* 12(1) (1965), 23–41.
- [28] R. T. Rockafellar. *Convex Analysis*. Princeton University Press, 1970.
- [29] H. Rogers, Jr. *Theory of Recursive Functions and Effective Computability*. McGraw-Hill, 1967.
- [30] W. Rudin. *Principles of Mathematical Analysis*, 3rd ed. McGraw-Hill, 1976.
- [31] A. Schrijver. *Theory of Linear and Integer Programming*. Wiley, 1986.
- [32] C. Tinelli, M. T. Harandi. A new correctness proof of the Nelson–Oppen combination procedure. In: *Frontiers of Combining Systems (FroCoS 1996)*, Kluwer, 1996, 103–119.
- [33] G. S. Tseitin. On the complexity of derivation in propositional calculus. In: A. O. Slisenko (ed.), *Studies in Constructive Mathematics and Mathematical Logic, Part II*, 1968, 115–125.
- [34] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* (2) 42 (1937), 230–265.

Manuscripts of the author.

- [35] Flyxion. *The Algebra of Binding: Distinction, Interaction, and Admissible Collapse from Elementary Identities to General Systems*. Manuscript, 2026.
- [36] Flyxion. *The Calculus of Disposition*. Manuscript in preparation, 2026.
- [37] Flyxion. *Rebuilding the Foundations of Logic: Sets, Type Theory, and Reproducibility*. Manuscript, 2026.
- [38] Flyxion. *Grounding Before Collapse: Formulation Gaps, Verifier Gates, and the Limits of Self-Contained Judgment*. Manuscript, 2026.

- [39] Flyxion. Verification inheritance and the Lean mutation harness. Manuscripts and tooling, 2026. (Descriptive title.)
- [40] Flyxion. *Capacity, Dissipation, and the Onset of Motility-Induced Phase Separation*. Manuscript, 2026.